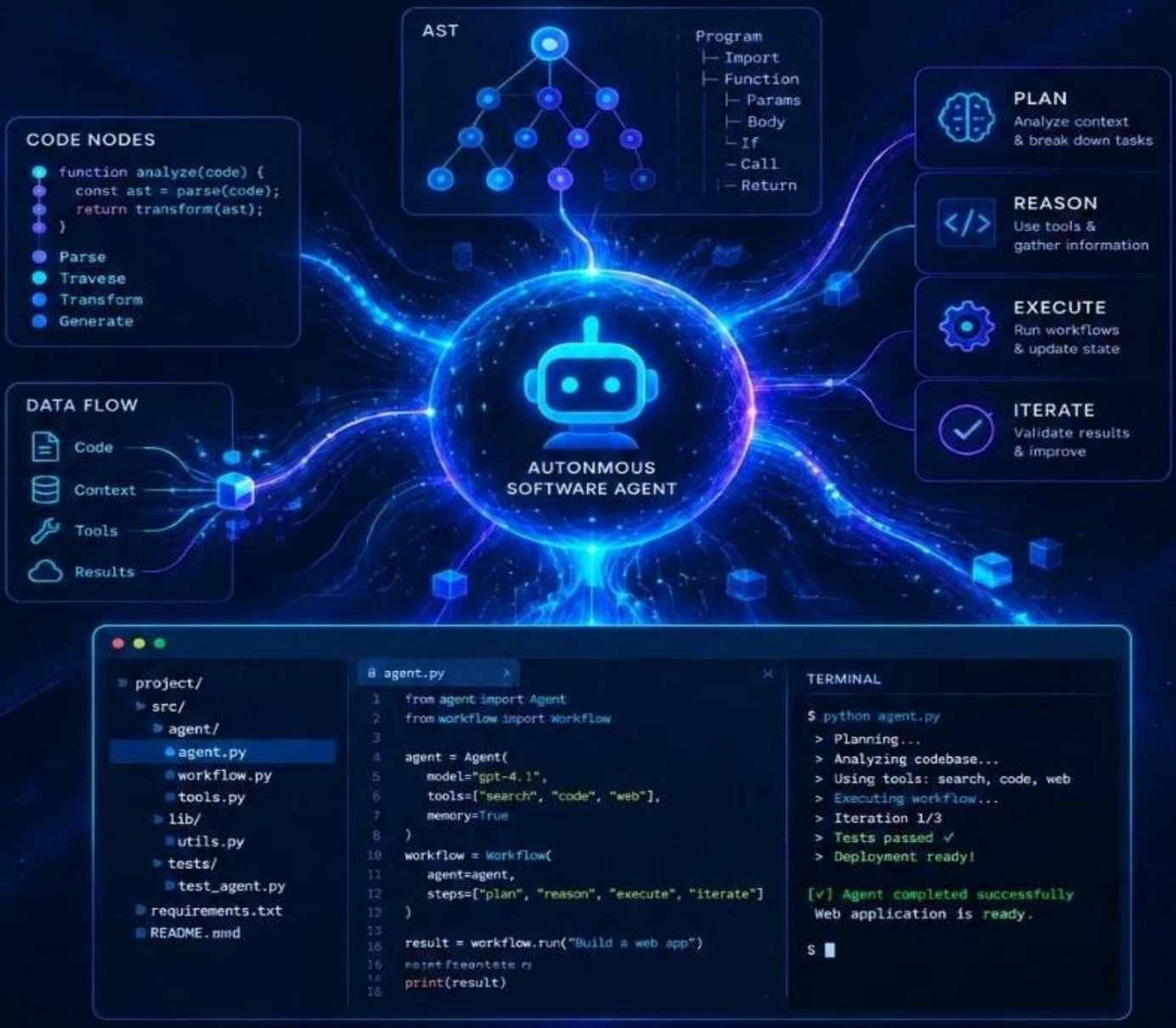
AI
SYSTEMS
DEVELOPMENT
PRACTICES

ENGINEERING THE FUTURE OF SOFTWARE

Mastering Agentic AI Workflows



A PRACTICAL GUIDE FOR MODERN AI ENGINEERS

Author: Mahdieh Mohammadi
Supervisor: Mehrdad Hasanzadeh

TECH PRESS

درباره این کتابچه

این کتابچه با هدف آشنایی با ایجنت‌های هوش مصنوعی و آماده‌سازی خواننده برای استفاده‌ی عملی از آن‌ها تهیه شده است. مطالب از مفاهیم پایه آغاز می‌شوند و به‌تدریج نحوه‌ی عملکرد ایجنت‌ها، انواع و کاربردهای آن‌ها، روش انتخاب ایجنت مناسب، اصول کار با آن‌ها و چگونگی ساخت و مدیریت پروژه با کمک ایجنت‌ها را بررسی می‌کنند.

تلاش شده است مطالب به زبانی ساده، شفاف و کاربردی ارائه شوند تا خواننده، حتی بدون دانش قبلی، بتواند درک مناسبی از ایجنت‌ها به دست آورد و با آگاهی بیشتری از قابلیت‌ها، محدودیت‌ها و ابزارهای آن‌ها استفاده کند.

bitwiti.ir

فهرست مطالب

پیشگفتار

بخش اول: آشنایی با ایجنت‌ها

1. ایجنت چیست و چه کاری انجام می‌دهد؟
2. چرا ایجنت‌ها اهمیت پیدا کرده‌اند؟
3. تفاوت ایجنت با مدل هوش مصنوعی، چت‌بات، دستیار هوش مصنوعی، Coding Assistant و Coding Agent

بخش دوم: ایجنت‌ها چگونه کار می‌کنند؟

1. فرایند کار ایجنت از دریافت درخواست تا ارائه نتیجه
2. نقش مدل هوش مصنوعی در ایجنت
3. هدف‌گذاری، برنامه‌ریزی و اجرای وظایف
4. ابزارها و نحوه استفاده از آن‌ها
5. حافظه و Context
6. چرخه اجرا، بررسی نتیجه و اصلاح خطا
7. میزان استقلال ایجنت و نقش نظارت انسان

بخش سوم: انواع ایجنت‌ها و کاربردهای آن‌ها

1. دسته‌بندی ایجنت‌ها بر اساس کاربرد

bitwibit

2. ایجنت‌های برنامه‌نویسی
3. ایجنت‌های پژوهشی و تحلیل اطلاعات
4. ایجنت‌های آموزشی
5. ایجنت‌های تولید محتوا
6. ایجنت‌های اتوماسیون و انجام کارهای تکراری
7. ایجنت‌های فرعی (Subagents) و نقش آن‌ها در تقسیم وظایف
8. ایجنت‌های تخصصی و چندمنظوره
9. سیستم‌های چندایجنتی و همکاری چند ایجنت

بخش چهارم: انتخاب ایجنت مناسب

1. معیارهای انتخاب ایجنت
2. انتخاب ایجنت مناسب برای انواع پروژه‌ها
3. مقایسه قابلیت‌ها، ابزارها، هزینه و سهولت استفاده
4. انتخاب بین ایجنت‌های رایگان و پولی
5. چه زمانی یک ایجنت کافی است و چه زمانی به چند ایجنت نیاز داریم؟
6. معرفی و بررسی ایجنت‌های پرکاربرد

بخش پنجم: شروع کار با ایجنت‌ها

1. پیش‌نیازهای شروع کار
2. آشنایی با محیط و تنظیمات ایجنت
3. نحوه تعریف وظیفه و نوشتن درخواست مناسب
4. ارائه اطلاعات و Context موردنیاز
5. آشنایی با دستورها و قابلیت‌های ایجنت
6. دستورهای عمومی و دستورهای اختصاصی هر ابزار
7. بررسی خروجی، اصلاح درخواست و ادامه کار

بخش ششم: ساخت پروژه با کمک ایجنت

1. انتخاب مسئله و تعریف هدف پروژه
2. تبدیل ایده به نیازمندی‌های مشخص
3. طراحی ساختار و معماری پروژه
4. تقسیم پروژه به وظایف کوچک و قابل اجرا
5. آماده‌سازی محیط و فایل‌های پروژه
6. توسعه پروژه با کمک ایجنت
7. آزمایش، اشکال‌زدایی و اصلاح کد
8. بررسی کیفیت و آماده‌سازی نسخه نهایی
9. روش‌های افزایش سرعت توسعه با ایجنت

فصل هفتم: مدیریت پروژه با کمک ایجنت

۷. برنامه‌ریزی و تقسیم پروژه به وظایف کوچک
۷. مدیریت Context، فایل‌ها و تغییرات پروژه
۷. مدیریت پیشرفت پروژه و کنترل عملکرد ایجنت
۷. همکاری چند ایجنت و نقش انسان در مدیریت پروژه

فصل هشتم: محدودیت‌ها، خطرات و اشتباهات رایج در استفاده از ایجنت‌ها

۸. محدودیت‌های ایجنت‌ها
۸. خطرات امنیتی و مشکلات اجرایی
۸. اشتباهات رایج در استفاده از ایجنت‌ها
۸. روش‌های کاهش خطا و استفاده ایمن از ایجنت

فصل نهم: اصول حرفه‌ای کار با ایجنت‌ها

۹. نوشتن درخواست‌های دقیق و هدفمند
۹. مدیریت صحیح وظایف و سطح دسترسی
۹. بررسی و ارزیابی خروجی ایجنت
۹. حفظ کنترل و بهبود فرایند کار

bitvisit.ir

فصل دهم: جمع‌بندی و مرجع کاربردی

۱۰. نقشه راه شروع کار با ایجنت‌ها

۱۰. نمونه یک فرایند کاری از ابتدا تا انتها

۱۰. چک‌لیست انتخاب ایجنت و مدیریت پروژه

۱۰. واژه‌نامه اصطلاحات و مرجع دستورها

۱۰. جمع‌بندی نهایی کتابچه

پیشگفتار

هوش مصنوعی در سال‌های اخیر از ابزاری برای پاسخ‌گویی به پرسش‌ها و تولید محتوا فراتر رفته و زمینه را برای توسعه‌ی سیستم‌هایی فراهم کرده است که می‌توانند وظایف مختلف را برنامه‌ریزی کنند، از ابزارهای گوناگون استفاده کنند و در انجام فعالیت‌های پیچیده به کاربران کمک کنند. ایجنت‌های هوش مصنوعی یکی از مهم‌ترین جلوه‌های این تحول هستند و آشنایی با آن‌ها می‌تواند فرصت‌های تازه‌ای برای یادگیری، حل مسئله و توسعه‌ی پروژه‌ها ایجاد کند.

هدف کتابچه

هدف این کتابچه، ارائه‌ی یک راهنمای جامع و کاربردی برای آشنایی با ایجنت‌های هوش مصنوعی و آماده‌سازی خواننده برای استفاده‌ی آگاهانه و مؤثر از آن‌هاست. مطالب از مفاهیم پایه آغاز می‌شوند و به‌تدریج به نحوه‌ی عملکرد ایجنت‌ها، انواع و کاربردهای آن‌ها، انتخاب ابزار مناسب، شیوه‌ی نوشتن دستورها و چگونگی ساخت و مدیریت پروژه با کمک ایجنت‌ها می‌پردازند. همچنین محدودیت‌ها، خطرات و اصول

صحیح استفاده از این ابزارها بررسی می‌شوند تا خواننده بتواند توانایی‌های آن‌ها را در کنار نقاط ضعفشان بشناسد.

این کتابچه برای چه کسانی مناسب است؟

این کتابچه برای افرادی نوشته شده است که می‌خواهند بدون نیاز به دانش قبلی، با ایجنت‌های هوش مصنوعی آشنا شوند و استفاده از آن‌ها را آغاز کنند. دانشجویان، برنامه‌نویسان مبتدی، پژوهشگران، افرادی که قصد توسعه‌ی یک پروژه را دارند و کسانی که به دنبال افزایش سرعت و بهره‌وری در فعالیت‌های خود هستند، می‌توانند از مطالب این کتابچه استفاده کنند.

خواننده پس از مطالعه‌ی این کتابچه چه چیزهایی می‌آموزد؟

انتظار می‌رود خواننده پس از مطالعه‌ی این کتابچه بتواند:

- مفهوم ایجنت و تفاوت آن با سایر ابزارهای هوش مصنوعی را درک کند.
- با نحوه‌ی عملکرد ایجنت‌ها، اجزای اصلی و انواع آن‌ها آشنا شود.
- ایجنت مناسب را بر اساس نیاز و نوع پروژه انتخاب کند.
- درخواست‌ها و دستورهای دقیق‌تری بنویسد و از قابلیت‌های ایجنت بهتر استفاده کند.
- با کمک ایجنت، مراحل طراحی، توسعه، آزمایش و اصلاح یک پروژه را پیش ببرد.
- وظایف پروژه را سازمان‌دهی کند و روند کار ایجنت را مدیریت کند.
- محدودیت‌ها، خطاها و خطرات احتمالی را تشخیص دهد و خروجی‌ها را به‌درستی ارزیابی کند.
- نقش خود را در هدایت، نظارت و تصمیم‌گیری هنگام استفاده از ایجنت بشناسد.

روش پیشنهادی مطالعه

پیشنهاد می‌شود کتابچه به‌ترتیب مطالعه شود؛ زیرا مفاهیم هر بخش، زمینه‌ی درک بهتر بخش‌های بعدی را فراهم می‌کنند. بهتر است خواننده هنگام مطالعه‌ی مثال‌ها، دستورهای پیشنهادی را در صورت امکان در یک ایجنت واقعی آزمایش کند و نتیجه‌ی اجرای آن‌ها را با توضیحات کتابچه مقایسه کند. یادداشت‌برداری از اصطلاحات جدید و تمرین با یک پروژه‌ی کوچک نیز می‌تواند به تثبیت مطالب کمک کند.

همچنین توصیه می‌شود خواننده تنها به حفظ دستورها اکتفا نکند، بلکه دلیل استفاده از هر دستور، شرایط مناسب اجرای آن و روش بررسی نتیجه را نیز بیاموزد. از آنجا

که قابلیت‌ها و دستورهای ایجنت‌ها ممکن است با یکدیگر متفاوت باشند و در طول زمان تغییر کنند، لازم است هنگام کار با یک ابزار مشخص، مستندات و راهنمای رسمی همان ابزار نیز بررسی شود.

ایجنت؛ ابزاری برای توانمندسازی انسان، نه جایگزین او

ایجنت‌های هوش مصنوعی می‌توانند بسیاری از فعالیت‌ها را سریع‌تر کنند، در انجام وظایف پیچیده کمک‌رسان باشند و بخشی از کارهای زمان‌بر را بر عهده بگیرند؛ اما این توانایی‌ها به معنای بی‌نیازی از دانش، تفکر انتقادی و نظارت انسانی نیست. ایجنت ممکن است درخواست را اشتباه تفسیر کند، اطلاعات نادرست ارائه دهد، کد معیوب تولید کند یا تغییری ایجاد کند که با هدف پروژه سازگار نباشد.

بنابراین، رویکرد اصلی این کتابچه آن است که ایجنت را ابزاری برای افزایش توانایی و بهره‌وری انسان بدانیم، نه جایگزینی برای قضاوت، مسئولیت‌پذیری و یادگیری او. استفاده‌ی مؤثر از ایجنت زمانی امکان‌پذیر است که کاربر بتواند هدف را مشخص کند، وظایف را به‌درستی هدایت کند، نتایج را ارزیابی کند و در موقعیت‌های حساس تصمیم نهایی را آگاهانه بگیرد.

در نهایت، هدف این است که خواننده نه‌تنها با ایجنت‌ها آشنا شود، بلکه بتواند با شناخت قابلیت‌ها و محدودیت‌های آن‌ها، به‌صورت هدفمند، مسئولانه و مؤثر از این فناوری در مسیر یادگیری و انجام پروژه‌های خود استفاده کند.

فصل اول: آشنایی با Agent ها

Agent چیست؟ و چه کاری انجام میدهد؟

سیستمی است که از یک یا چند مدل هوش مصنوعی و مجموعه‌ای از قابلیت‌ها و ابزارها استفاده می‌کند تا یک هدف یا وظیفه را دنبال کند. بسته به طراحی سیستم و سطح دسترسی آن، ایجنت می‌تواند درخواست کاربر را تحلیل کند، مراحل انجام کار را مشخص کند، از ابزارهای مناسب استفاده کند، نتیجه‌ی هر مرحله را بررسی کند و در صورت نیاز، مراحل بعدی را تغییر دهد.

به بیان ساده، یک ایجنت فقط برای پاسخ‌دادن به سؤال طراحی نشده است؛ بلکه می‌تواند برای پیش‌بردن یک کار تا رسیدن به نتیجه‌ی موردنظر نیز به کار گرفته شود.

برای مثال، فرض کنیم کاربر می‌خواهد یک برنامه‌ی ساده برای مدیریت وظایف روزانه بسازد. یک مدل هوش مصنوعی می‌تواند کد اولیه‌ی برنامه را پیشنهاد دهد. اما یک ایجنت برنامه‌نویسی، در صورت داشتن ابزارها و دسترسی‌های لازم، ممکن است بتواند:

۱. درخواست کاربر را بررسی کند و نیازمندی‌های پروژه را مشخص کند.

۲. فایل‌های موجود در پروژه را بخواند و ساختار آن‌ها را بررسی کند.

۳. کدهای موردنیاز را ایجاد یا اصلاح کند.

۴. برنامه را اجرا کند و آزمایش‌های مناسب را انجام دهد.

۵. خطاهای شناسایی‌شده را بررسی کند و برای رفع آن‌ها تغییراتی اعمال کند.

۶. نتیجه‌ی کار را به کاربر گزارش دهد.

این مراحل در همه‌ی ایجنت‌ها یکسان نیستند. توانایی انجام آن‌ها به ابزارهای در دسترس، سطح دسترسی، کیفیت مدل، دستورهای کاربر و محدودیت‌های محیط اجرا بستگی دارد. همچنین انجام خودکار یک مرحله لزوماً به معنای درست‌بودن نتیجه‌ی آن نیست.

تفاوت Agent با چت‌بات، مدل هوش مصنوعی، Coding Assistant و Coding Agent

برای درک بهتر مفهوم Agent، باید آن را با چند مفهوم مرتبط مقایسه کنیم. این اصطلاحات به یکدیگر ارتباط دارند، اما از نظر نقش و نحوهٔ انجام وظایف یکسان نیستند.

۱. مدل هوش مصنوعی (AI Model) چیست؟مدل هوش مصنوعی بخشی است که بر اساس آموزش‌های قبلی خود، ورودی را پردازش می‌کند و خروجی تولید می‌کند. برای مثال، یک مدل زبانی می‌تواند به پرسش‌ها پاسخ دهد، متن بنویسد، کد تولید کند یا اطلاعات را تحلیل کند.

تفاوت با Agent: مدل هوش مصنوعی به‌خودی‌خود لزوماً نمی‌تواند ابزارهای خارجی را اجرا کند، فایل‌های پروژه را تغییر دهد یا مراحل انجام یک کار را به‌صورت مستقل

پیگیری کند. Agent می‌تواند از یک مدل به‌عنوان بخش اصلی توانایی پردازش و تصمیم‌گیری خود استفاده کند و آن را با ابزارها و سازوکارهای اجرایی ترکیب کند.

۲. چت‌بات (Chatbot) چیست؟

چت‌بات ابزاری است که امکان گفت‌وگو میان کاربر و یک نرم‌افزار را فراهم می‌کند. چت‌بات‌ها می‌توانند به پرسش‌ها پاسخ دهند، اطلاعات ارائه کنند و بر اساس پیام‌های قبلی، مکالمه را ادامه دهند. برخی از آن‌ها از مدل‌های هوش مصنوعی استفاده می‌کنند و برخی نیز قابلیت اتصال به ابزارها را دارند.

تفاوت با Agent : تمرکز اصلی چت‌بات بر تعامل و پاسخ‌گویی است؛ اما Agent می‌تواند علاوه بر گفت‌وگو، برای انجام یک وظیفه اقدام کند، چند مرحله را پیش ببرد و نتایج اقدامات خود را بررسی کند. البته چت‌بات‌های پیشرفته نیز ممکن است قابلیت‌های Agent را داشته باشند.

۳. دستیار برنامه‌نویسی (Coding Assistant) چیست؟

Coding Assistant ابزاری است که به برنامه‌نویس در فرایند کدنویسی کمک می‌کند. این ابزار می‌تواند کد پیشنهاد دهد، دستورات را تکمیل کند، کدهای موجود را توضیح دهد، خطاها را تحلیل کند و در نوشتن یا اصلاح کد به برنامه‌نویس کمک کند.

تفاوت با :Agent یک دستیار برنامه‌نویسی ممکن است بیشتر بر ارائهٔ پیشنهاد و کمک به کدنویس تمرکز داشته باشد؛ درحالی‌که Coding Agent می‌تواند با دسترسی به فایل‌ها و ابزارهای لازم، وظیفه‌ای را در چند مرحله اجرا کند، کد را تغییر دهد، تست بگیرد و نتیجه را بررسی کند. بااین‌حال، بعضی دستیارهای برنامه‌نویسی نیز همین قابلیت‌ها را دارند و عملاً به‌عنوان Agent عمل می‌کنند.

۴. عامل برنامه‌نویسی (Coding Agent) چیست؟

Coding Agent در واقع نوعی Agent است که برای انجام وظایف مرتبط با توسعهٔ نرم‌افزار طراحی شده است. این ابزار می‌تواند درخواست برنامه‌نویس را بررسی کند، ساختار پروژه را بشناسد، فایل‌ها را بخواند یا تغییر دهد، از ترمینال استفاده کند، تست‌ها را اجرا کند و در صورت مشاهدهٔ خطا برای اصلاح آن تلاش کند.

تفاوتAgent با Coding Agent : در مقابل Agent قرار نمی‌گیرد؛ بلکه زیرمجموعه‌ای از آن است. واژهٔ Agent مفهوم گسترده‌تری دارد و می‌تواند به

عامل‌هایی در حوزه‌های مختلف، مانند پژوهش، تحلیل داده، پشتیبانی مشتری یا برنامه‌نویسی اشاره کند. Coding Agent مشخصاً بر وظایف توسعهٔ نرم‌افزار تمرکز دارد.

فرض کنید می‌خواهیم یک قابلیت ورود کاربران به یک پروژهٔ نرم‌افزاری اضافه کنیم.

•مدل هوش مصنوعی: می‌تواند کد مربوط به ورود کاربران را تولید کند.

•چت‌بات: می‌تواند مراحل پیاده‌سازی را توضیح دهد و به پرسش‌های ما پاسخ دهد.

•Coding Assistant : می‌تواند هنگام نوشتن کد، پیشنهادهای مرتبط ارائه کند.

•Coding Agent: در صورت برخورداری از ابزارها و دسترسی لازم، می‌تواند ساختار پروژه را بررسی کند، فایل‌های موردنیاز را تغییر دهد، تست‌ها را اجرا کند و مشکلات احتمالی را برطرف کند.

تفاوت اصلی این مفاهیم در نقش و نحوهٔ انجام وظایف آن‌هاست. مدل هوش مصنوعی توانایی پردازش و تولید خروجی را فراهم می‌کند؛ چت‌بات بستری برای گفت‌وگو است؛ Coding Assistant به برنامه‌نویس کمک می‌کند؛ و Agent می‌تواند با استفاده از مدل، ابزارها و سازوکارهای اجرایی، وظایف را در چند مرحله پیش ببرد. Coding Agent نیز نوعی Agent است که این قابلیت‌ها را در حوزهٔ برنامه‌نویسی به کار می‌گیرد.

این دسته‌ها مرزهای کاملاً جداگانه‌ای ندارند؛ یک ابزار ممکن است هم‌زمان چت‌بات، دستیار برنامه‌نویسی و Agent باشد. بنابراین، برای تشخیص نوع یک ابزار باید به قابلیت‌های واقعی آن توجه کرد، نه فقط نامی که برایش انتخاب شده است.

چرا ایجنت‌ها اهمیت پیدا کرده‌اند؟

یکی از دلایل اهمیت ایجنت‌ها این است که بسیاری از کارهای واقعی تنها با تولید یک پاسخ به پایان نمی‌رسند. انجام این کارها ممکن است به بررسی اطلاعات، استفاده از چند ابزار، اجرای مراحل مختلف، ارزیابی نتیجه و اصلاح اشتباهات نیاز داشته باشد.

ایجنت‌ها می‌توانند این مراحل را در قالب یک فرایند منسجم‌تر به یکدیگر متصل کنند و در انجام کارهای چندمرحله‌ای به کاربر کمک کنند. برای نمونه، در برنامه‌نویسی می‌توانند در بررسی فایل‌ها، اصلاح کد و اجرای آزمایش‌ها نقش داشته باشند؛ در پژوهش می‌توانند به جمع‌آوری و سازمان‌دهی اطلاعات کمک کنند؛ و در اتوماسیون می‌توانند اجرای برخی وظایف تکراری را تسهیل کنند.

از دیگر مزایای بالقوه‌ی ایجنت‌ها می‌توان به کاهش زمان انجام برخی کارها، کمک به افراد کم‌تجربه، هماهنگ‌کردن ابزارهای مختلف و امکان تمرکز بیشتر کاربر بر تصمیم‌گیری‌های مهم اشاره کرد.

البته استفاده از ایجنت همیشه سریع‌تر، ارزان‌تر یا دقیق‌تر نیست. کارهای مبهم، حساس یا پیچیده ممکن است به نظارت زیاد، اصلاح مکرر و بررسی انسانی نیاز داشته باشند. بنابراین، ارزش ایجنت به میزان تناسب آن با وظیفه و نحوه‌ی استفاده از آن بستگی دارد.

نکته‌ی اصلی این است که ایجنت می‌تواند با استفاده از مدل هوش مصنوعی و ابزارهای مختلف، وظایف چندمرحله‌ای را دنبال کند. با این حال، میزان توانایی و استقلال آن به طراحی سیستم، دسترسی‌ها و محدودیت‌هایش بستگی دارد.

فصل 2: ایجنت‌ها چگونه کار می‌کنند؟

فرایند کار ایجنت از دریافت درخواست تا ارائه‌ی نتیجه

برای درک نحوه‌ی عملکرد ایجنت‌ها، باید بدانیم که انجام یک وظیفه معمولاً تنها به دریافت درخواست و تولید پاسخ محدود نمی‌شود. ایجنت ممکن است لازم باشد درخواست را تحلیل کند، اطلاعات موردنیاز را به دست آورد، مراحل انجام کار را مشخص کند، از ابزارها کمک بگیرد و نتیجه‌ی اقدامات خود را بررسی کند.

این فرایند در بسیاری از ایجنت‌ها به‌صورت یک چرخه انجام می‌شود؛ یعنی سیستم پس از هر اقدام، نتیجه را دریافت می‌کند و بر اساس آن تصمیم می‌گیرد که چه کاری را در ادامه انجام دهد.

به‌طور کلی، فرایند کار یک ایجنت را می‌توان در مراحل زیر خلاصه کرد:

۱. دریافت درخواست: ایجنت درخواست کاربر و اطلاعات همراه آن را دریافت می‌کند.

۲. تحلیل درخواست: سیستم تلاش می‌کند هدف کاربر، محدودیت‌ها و نتیجه‌ی مورد انتظار را مشخص کند.

3. برنامه‌ریزی:در صورت نیاز، وظیفه به چند مرحله‌ی کوچک‌تر تقسیم می‌شود.

۴. انتخاب اقدام: ایجنت مشخص می‌کند که برای پیش‌بردن کار باید پاسخ تولید کند، اطلاعات بیشتری به دست آورد یا از یک ابزار استفاده کند.

۵. اجرای اقدام:اقدام انتخاب‌شده انجام می‌شود؛ برای مثال، یک فایل خوانده می‌شود یا کدی در محیط اجرا اجرا می‌شود.

۶. بررسی نتیجه:خروجی اقدام بررسی می‌شود تا مشخص شود آیا اطلاعات کافی برای ادامه‌ی کار وجود دارد یا خیر

۷. ادامه یا پایان کار: ایجنت ممکن است مرحله‌ی بعدی را اجرا کند، اقدام قبلی را اصلاح کند، اطلاعات بیشتری بخواهد یا نتیجه‌ی نهایی را ارائه دهد.

این مراحل لزوماً همیشه به همین ترتیب یا به‌صورت جداگانه اجرا نمی‌شوند. بعضی وظایف ساده‌اند و به یک یا دو مرحله محدود می‌شوند، درحالی‌که وظایف پیچیده ممکن است به چرخه‌های متعدد نیاز داشته باشند.

مثال کاربردی: فرض کنیم از یک ایجنت برنامه‌نویسی می‌خواهیم خطای ورود کاربران به یک وب‌سایت را بررسی کند. ایجنت ممکن است ابتدا فایل‌های مرتبط را پیدا کند، سپس کد ورود را بخواند، علت احتمالی خطا را مشخص کند، تغییر لازم را اعمال کند و آزمایش‌های مرتبط را اجرا کند. اگر آزمایش‌ها ناموفق باشند، ممکن است دوباره کد را بررسی و اصلاح کند.

نکته‌ی مهم این است که انجام این مراحل به معنی تضمین نتیجه‌ی صحیح نیست. ایجنت ممکن است علت خطا را اشتباه تشخیص دهد یا آزمایشی ناکافی انجام دهد؛ بنابراین، نتیجه‌ی نهایی باید متناسب با اهمیت کار بررسی شود.

نقش مدل هوش مصنوعی در ایجنت

مدل هوش مصنوعی (AI Model) بخش اصلی تحلیل و تولید خروجی در بسیاری از ایجنت‌هاست. مدل می‌تواند درخواست کاربر را تفسیر کند، اطلاعات موجود را بررسی کند، راهکار پیشنهاد دهد و درباره‌ی اقدام بعدی تصمیم بگیرد.

با این حال، مدل و ایجنت یکسان نیستند. مدل قابلیت‌های تحلیل و تولید خروجی را فراهم می‌کند، اما ایجنت با استفاده از مدل و اجزای دیگر، فرایند انجام وظیفه را مدیریت می‌کند.

برای مثال، یک مدل زبانی می‌تواند کد مربوط به ثبت‌نام کاربران را تولید کند. اما اگر قرار باشد کد در یک پروژه‌ی واقعی اصلاح شود، سیستم باید بتواند فایل‌های پروژه را در اختیار مدل قرار دهد، تغییرات پیشنهادی را در محیط مناسب اعمال کند و در صورت امکان نتیجه را آزمایش کند.

در یک ایجنت مجهز به ابزار، معمولاً فرایند به این صورت است:

- مدل درخواست و اطلاعات موجود را تحلیل می‌کند.

- مدل یا منطق کنترلی سیستم، اقدام مناسب را انتخاب می‌کند.
- اگر به ابزار نیاز باشد، سیستم درخواست استفاده از آن ابزار را ایجاد و اجرا می‌کند.
- نتیجه‌ی ابزار دوباره در اختیار مدل یا بخش تصمیم‌گیری قرار می‌گیرد.
- فرایند تا رسیدن به نتیجه، توقف کار یا برخورد با محدودیت ادامه پیدا می‌کند.

در این ساختار، مدل ممکن است تصمیم بگیرد که کدام ابزار لازم است، اما اجرای واقعی ابزار بر عهده‌ی محیط نرم‌افزاری و سازوکار اجرایی ایجنت است.

همچنین همه‌ی ایجنت‌ها از یک مدل یا یک روش تصمیم‌گیری استفاده نمی‌کنند. برخی از آن‌ها مبتنی بر مدل‌های زبانی هستند و برخی دیگر از ترکیب مدل‌های مختلف، قواعد مشخص یا روش‌های تصمیم‌گیری دیگری استفاده می‌کنند.

هدف‌گذاری، برنامه‌ریزی و اجرای وظایف

یکی از قابلیت‌های مهم بسیاری از ایجنت‌ها، تبدیل یک درخواست کلی به مجموعه‌ای از وظایف کوچک‌تر است. این کار به سیستم کمک می‌کند فعالیت‌های پیچیده را مرحله‌به‌مرحله پیش ببرد و ترتیب انجام اقدامات را مشخص کند.

۱. هدف‌گذاری

هدف‌گذاری به معنای مشخص‌کردن نتیجه‌ای است که باید به آن برسیم. هرچه هدف روشن‌تر باشد، ارزیابی نتیجه نیز آسان‌تر خواهد بود.

برای مثال، درخواست «یک وب‌سایت خوب بساز» هدف دقیقی نیست؛ زیرا مشخص نمی‌کند وب‌سایت چه کاربردی دارد، چه امکاناتی باید داشته باشد و چه معیارهایی برای موفقیت آن در نظر گرفته شده است.

در مقابل، درخواست «یک وب‌سایت ساده برای ثبت و پیگیری وظایف روزانه بساز که امکان افزودن، ویرایش، حذف و علامت‌گذاری وظایف انجام‌شده را داشته باشد» هدف مشخص‌تری ارائه می‌کند.

2. برنامه‌ریزی

برنامه‌ریزی (Planning) فرایند تعیین مراحل و اقدامات لازم برای رسیدن به هدف است. ایجنت ممکن است وظیفه‌ی اصلی را به چند وظیفه‌ی کوچک‌تر تقسیم کند و ترتیب انجام آن‌ها را مشخص سازد.

برای ساخت وب‌سایت مدیریت وظایف، برنامه می‌تواند شامل این مراحل باشد:

۱. بررسی نیازمندی‌ها و مشخص‌کردن امکانات اصلی

۲. انتخاب فناوری‌ها و طراحی ساختار پروژه

۳. ایجاد فایل‌ها و پیاده‌سازی بخش‌های اصلی

۴. پیاده‌سازی ذخیره‌سازی اطلاعات

۵. آزمایش امکانات و رفع خطاها

۶. بررسی نهایی و آماده‌سازی پروژه

برنامه‌ی اولیه ممکن است در طول اجرا تغییر کند؛ زیرا اطلاعات جدید یا خطاهای پیش‌بینی‌نشده می‌توانند به اقدامات دیگری نیاز داشته باشند.

اجرای وظایف

پس از مشخص‌شدن اقدامات، ایجنت می‌تواند با توجه به ابزارها و دسترسی‌های خود آن‌ها را اجرا کند. بعضی اقدامات مستقیماً با تولید متن یا کد انجام می‌شوند و برخی دیگر به اجرای عملیات در یک محیط واقعی نیاز دارند.

برای مثال، تولید کد یک تابع با اجرای آن تفاوت دارد. ایجنت ممکن است کدی تولید کند که از نظر ظاهری صحیح به نظر برسد، اما هنگام اجرا با خطا مواجه شود. به همین دلیل، در وظایف برنامه‌نویسی باید میان تولید کد، اجرای کد و آزمایش نتیجه تفاوت قائل شد.

همچنین همه‌ی ایجنت‌ها الزاماً برنامه‌ی کامل و مشخصی را پیش از شروع اجرا تولید نمی‌کنند. برخی از آن‌ها برای هر مرحله برنامه‌ریزی می‌کنند و بر اساس نتیجه‌ی هر اقدام، تصمیم بعدی را می‌گیرند.

ابزارها و نحوه‌ی استفاده از آن‌ها

ابزار (Tool) قابلیتی است که ایجنت از طریق آن می‌تواند عملی مشخص را انجام دهد یا به اطلاعاتی دسترسی پیدا کند که صرفاً با تولید متن در اختیارش نیست.

ابزارها نقش مهمی در تبدیل توانایی تحلیل مدل به اقدامات عملی دارند. نوع ابزارهای موجود مشخص می‌کند که ایجنت در محیط خود چه کارهایی می‌تواند انجام دهد.

انواع ابزارهای رایج

ابزارهای جست‌وجو و دریافت اطلاعات: برای جست‌وجوی اطلاعات، دسترسی به منابع یا دریافت داده‌های موردنیاز استفاده می‌شوند.

ابزارهای کار با فایل: امکان خواندن، ایجاد، ویرایش، جابه‌جایی یا حذف فایل‌ها را فراهم می‌کنند؛ البته فقط در محدوده‌ی دسترسی مجاز.

ابزارهای اجرای کد: برای اجرای برنامه‌ها، اسکریپت‌ها یا کدهای تولیدشده استفاده می‌شوند.

Terminal: محیطی برای اجرای دستورهای متنی سیستم‌عامل و ابزارهای توسعه است. ایجنت برنامه‌نویسی ممکن است از طریق ترمینال وابستگی‌های پروژه را نصب کند، برنامه را اجرا کند یا آزمایش‌ها را انجام دهد.

ابزارهای آزمایش و بررسی کد: برای اجرای تست‌ها، بررسی کیفیت کد و شناسایی برخی خطاها به کار می‌روند.

ابزارهای ارتباط با سرویس‌ها: امکان تعامل با سرویس‌های بیرونی، پایگاه‌های داده، سامانه‌های مدیریت پروژه یا رابط‌های برنامه‌نویسی را فراهم می‌کنند؛ مشروط بر اینکه اتصال و دسترسی لازم وجود داشته باشد.

ایجنت چگونه از ابزار استفاده می‌کند؟

فرض کنیم ایجنت باید اطلاعات یک فایل را بررسی کند. اگر ابزار خواندن فایل در اختیارش باشد، می‌تواند درخواست استفاده از آن را ایجاد کند. سیستم اجرایی، درخواست را بررسی و در صورت مجازبودن اجرا می‌کند. سپس محتوای فایل به‌عنوان نتیجه در اختیار ایجنت قرار می‌گیرد تا بتواند آن را تحلیل کند.

بنابراین، فرایند استفاده از ابزار معمولاً شامل سه قسمت است:

۱. انتخاب ابزار مناسب برای وظیفه

۲. اجرای ابزار در محیط مربوطه

۳. بررسی نتیجه و تصمیم‌گیری درباره‌ی ادامه‌ی کار

داشتن یک ابزار به‌تنهایی به معنای توانایی استفاده‌ی صحیح از آن نیست. ایجنت باید بتواند ابزار مناسب را انتخاب کند و نتیجه‌ی آن را درست تفسیر کند.

اهمیت سطح دسترسی ابزارها

ابزارها می‌توانند قدرت ایجنت را افزایش دهند، اما در صورت دسترسی بیش‌ازحد، خطر ایجاد تغییرات ناخواسته یا آسیب به اطلاعات نیز وجود دارد.

برای مثال، ایجنتی که اجازه‌ی خواندن فایل‌ها را دارد، لزوماً نباید اجازه‌ی حذف آن‌ها را نیز داشته باشد. همچنین اجرای برخی دستورهای ترمینال یا دسترسی به اطلاعات محرمانه ممکن است به تأیید انسان نیاز داشته باشد.

به همین دلیل، دسترسی ابزارها باید متناسب با وظیفه تعیین شود و اقدامات حساس، در صورت نیاز، تحت نظارت کاربر انجام شوند.

حافظه و Context

برای اینکه ایجنت بتواند یک وظیفه را درست دنبال کند، لازم است اطلاعات مرتبط با آن را در اختیار داشته باشد. دو مفهوم مهم در این زمینه، Context و Memory هستند که اگرچه با یکدیگر ارتباط دارند، معنای یکسانی ندارند.

Context چیست؟

Context یا بافت اطلاعاتی، مجموعه‌ی اطلاعاتی است که سیستم در یک مرحله از کار برای تحلیل و تصمیم‌گیری در اختیار دارد. این اطلاعات می‌توانند شامل درخواست فعلی، پیام‌های قبلی، دستورالعمل‌ها، فایل‌های مرتبط با پروژه، نتایج ابزارها و اطلاعات دیگری باشند که برای انجام وظیفه لازم‌اند.

برای مثال، اگر از ایجنت بخواهیم یک تابع را اصلاح کند، اما کد تابع، نحوه‌ی استفاده از آن و خطای رخ‌داده را در اختیارش نگذاریم، ممکن است نتواند راهکار مناسبی ارائه دهد.

در پروژه‌های برنامه‌نویسی، Context مناسب می‌تواند شامل موارد زیر باشد:

- هدف و نیازمندی‌های پروژه
- ساختار فایل‌ها و بخش‌های مرتبط کد
- فناوری‌ها و نسخه‌های مورد استفاده
- تصمیم‌های فنی گرفته‌شده
- خطاهای قبلی و نتایج آزمایش‌ها
- محدودیت‌ها و معیارهای مورد انتظار

Memory یا حافظه چیست؟

حافظه در سیستم‌های ایجنتی به سازوکارهایی گفته می‌شود که اطلاعات را برای استفاده‌ی مجدد نگه می‌دارند. بسته به طراحی ابزار، این اطلاعات ممکن است فقط در طول یک وظیفه یا نشست نگه داشته شوند یا برای استفاده در نشست‌های آینده ذخیره شوند.

برای مثال، یک ایجنت ممکن است بتواند تصمیم‌های قبلی پروژه را در فایل‌های راهنما ذخیره کند تا در ادامه‌ی کار دوباره از آن‌ها استفاده شود.

با این حال، همه‌ی ایجنت‌ها حافظه‌ی پایدار ندارند و نحوه‌ی ذخیره‌سازی اطلاعات در آن‌ها متفاوت است. نگهداری اطلاعات در تاریخچه‌ی گفت‌وگو، فایل‌های پروژه و حافظه‌ی اختصاصی سیستم نیز سازوکارهای متفاوتی هستند.

تفاوت Context و Memory

به‌طور خلاصه، Context به اطلاعاتی اشاره دارد که در مرحله‌ی فعلی برای انجام کار در دسترس سیستم قرار دارند؛ اما Memory به سازوکار نگهداری اطلاعات برای استفاده‌ی مجدد اشاره می‌کند.

اطلاعات ذخیره‌شده در حافظه ممکن است در یک مرحله‌ی بعدی وارد Context شوند. بنابراین، ذخیره‌شدن یک اطلاعات لزوماً به این معنا نیست که مدل در تمام مراحل آن را در اختیار دارد.

محدودیت Context

مدل‌ها معمولاً محدودیتی در مقدار اطلاعاتی دارند که می‌توانند در یک مرحله پردازش کنند. این محدودیت با مفهوم پنجره‌ی Context (Context Window) مرتبط است.

اگر اطلاعات یک پروژه بسیار زیاد باشد، ممکن است نتوان همه‌ی فایل‌ها، گفت‌وگوها و جزئیات را به‌طور هم‌زمان در اختیار مدل قرار داد. در چنین شرایطی، لازم است اطلاعات مرتبط انتخاب شوند، خلاصه‌های دقیق تهیه شوند یا از فایل‌ها و سازوکارهای بازیابی اطلاعات استفاده شود.

برای مدیریت بهتر Context، بهتر است هدف پروژه، تصمیم‌های مهم، وضعیت فعلی کار و وظیفه‌ی بعدی به‌صورت روشن ثبت شوند. این کار به کاهش سردرگمی و جلوگیری از تکرار کارهای قبلی کمک می‌کند.

چرخه‌ی اجرا، بررسی نتیجه و اصلاح خطا

یکی از تفاوت‌های مهم میان تولید یک پاسخ و اجرای یک وظیفه، امکان بررسی نتیجه‌ی اقدامات است. در بسیاری از ایجنت‌ها، فرایند کار پس از اولین اقدام پایان نمی‌یابد؛ بلکه سیستم نتیجه را دریافت می‌کند و بر اساس آن درباره‌ی ادامه‌ی کار تصمیم می‌گیرد.

این چرخه را می‌توان به‌صورت زیر نمایش داد:

درخواست ← تحلیل و برنامه‌ریزی ← اقدام ← مشاهده‌ی نتیجه ← ارزیابی ← اقدام بعدی یا پایان

برای مثال، در یک پروژه‌ی برنامه‌نویسی، ایجنت ممکن است کدی تولید کند و سپس آزمایش‌ها را اجرا کند. اگر آزمایش‌ها با خطا مواجه شوند، خروجی خطا را بررسی می‌کند و برای اصلاح کد تلاش می‌کند. پس از اصلاح نیز ممکن است آزمایش‌ها را دوباره اجرا کند.

در این فرایند، چند مفهوم اهمیت دارند:

مشاهده‌ی نتیجه: دریافت اطلاعات حاصل از اقدام؛ برای مثال، پیام خطای ترمینال یا نتیجه‌ی یک آزمایش.

ارزیابی: بررسی اینکه نتیجه با هدف و معیارهای مورد انتظار سازگار است یا خیر.

اصلاح: تغییر راهکار یا اجرای اقدام دیگری برای برطرف‌کردن مشکل.

تأیید نهایی: بررسی اینکه وظیفه واقعاً انجام شده است و شواهد کافی برای پذیرش نتیجه وجود دارد.

البته نتیجه‌ی موفق یک ابزار همیشه به معنای موفقیت کل وظیفه نیست. برای مثال، ممکن است برنامه بدون خطای اجرایی اجرا شود، اما عملکرد آن با نیاز کاربر مطابقت نداشته باشد. به همین دلیل، معیار ارزیابی باید با هدف واقعی پروژه هماهنگ باشد.

همچنین ایجنت ممکن است در یک چرخه‌ی اصلاح، تغییری ایجاد کند که مشکل دیگری به وجود آورد. بنابراین، تکرار خودکار اقدامات به‌تنهایی تضمین‌کننده‌ی کیفیت نیست. اجرای آزمایش‌های مناسب، بررسی تغییرات و تعیین شرایط توقف برای جلوگیری از تکرار بی‌نتیجه‌ی تلاش‌ها اهمیت دارد.

میزان استقلال ایجنت و نقش نظارت انسان

همه‌ی ایجنت‌ها از نظر میزان استقلال یکسان نیستند. بعضی از آن‌ها برای هر اقدام به راهنمایی یا تأیید کاربر نیاز دارند، درحالی‌که برخی دیگر می‌توانند مجموعه‌ای از وظایف را با دخالت کمتر انجام دهند.

میزان استقلال یک ایجنت به عوامل مختلفی بستگی دارد؛ از جمله طراحی سیستم، نوع وظیفه، ابزارهای موجود، سطح دسترسی، محدودیت‌های اجرایی و قواعدی که کاربر یا توسعه‌دهنده تعیین کرده است.

به‌طور کلی، می‌توان سه حالت رایج را در نظر گرفت:

حالت اول: اجرای هدایت‌شده

کاربر مراحل یا اقدامات اصلی را مشخص می‌کند و ایجنت در انجام هر مرحله کمک می‌کند. این حالت برای افراد مبتدی، کارهای حساس یا وظایفی که به تصمیم‌گیری دقیق نیاز دارند مناسب است.

حالت دوم: اجرای نیمه‌خودکار

ایجنت می‌تواند وظیفه را به مراحل مختلف تقسیم کند و بخشی از آن‌ها را مستقل انجام دهد، اما برای برخی اقدامات مهم، تغییرات حساس یا تصمیم‌های خاص به تأیید کاربر نیاز دارد.

حالت سوم: اجرای خودکار در محدوده‌ی تعیین‌شده

ایجنت می‌تواند مجموعه‌ای از اقدامات را بدون تأیید جداگانه‌ی هر مرحله اجرا کند. با این حال، فعالیت آن همچنان به ابزارها، مجوزها، قواعد و محدودیت‌های محیط وابسته است و نباید آن را به معنای استقلال نامحدود در نظر گرفت.

این سه حالت یک دسته‌بندی توضیحی هستند و لزوماً استاندارد رسمی یا گزینه‌های ثابت همه‌ی ابزارها محسوب نمی‌شوند.

نقش انسان در فرایند کار

حتی زمانی که ایجنت بخش زیادی از وظایف را به‌صورت خودکار انجام می‌دهد، انسان همچنان نقش مهمی دارد. کاربر باید هدف را مشخص کند، محدودیت‌ها را تعیین کند، اطلاعات حساس را محافظت کند و نتیجه را متناسب با اهمیت کار ارزیابی کند.

برای مثال، در یک پروژه‌ی نرم‌افزاری، می‌توان نوشتن یک تابع ساده یا اجرای آزمایش‌های معمول را به ایجنت سپرد؛ اما تغییرات اساسی در معماری پروژه، حذف

اطلاعات مهم یا انتشار نسخه‌ی نهایی بهتر است با بررسی و تصمیم‌گیری آگاهانه انجام شوند.

بهتر است سطح نظارت بر اساس میزان خطر و اهمیت اقدام تعیین شود. هرچه پیامد یک خطا جدی‌تر یا برگشت‌ناپذیرتر باشد، ضرورت بررسی انسانی نیز بیشتر می‌شود

ایجنت‌ها با ترکیب قابلیت‌های مدل هوش مصنوعی، ابزارها و سازوکارهای کنترل اجرا می‌توانند وظایف را در چند مرحله پیش ببرند. آن‌ها درخواست را تحلیل می‌کنند، در صورت نیاز برنامه‌ریزی می‌کنند، اقدام انجام می‌دهند و از نتیجه برای ادامه‌ی کار یا اصلاح خطا استفاده می‌کنند.

در این فرایند، ابزارها امکان انجام عملیات عملی را فراهم می‌کنند، Context اطلاعات موردنیاز مرحله‌ی فعلی را در اختیار سیستم قرار می‌دهد و حافظه می‌تواند به نگهداری اطلاعات برای استفاده‌ی مجدد کمک کند. میزان استقلال نیز به طراحی ایجنت و دسترسی‌های آن وابسته است.

مهم‌ترین نکته این است که توانایی انجام خودکار یک کار با تضمین صحت آن تفاوت دارد. استفاده‌ی مؤثر از ایجنت مستلزم تعریف روشن هدف، فراهم‌کردن اطلاعات مناسب، انتخاب ابزارهای درست و بررسی نتایج است.

فصل3: انواع Agent ها و کاربردهای آنها

دسته‌بندی ایجنت‌ها بر اساس کاربرد

ایجنت‌های هوش مصنوعی را می‌توان بر اساس نوع وظایفی که انجام می‌دهند، ابزارهایی که در اختیار دارند و محیطی که در آن فعالیت می‌کنند، به دسته‌های مختلفی تقسیم کرد. بعضی از ایجنت‌ها برای انجام وظایف مشخصی مانند برنامه‌نویسی طراحی شده‌اند و بعضی دیگر می‌توانند در حوزه‌های گوناگون به کار گرفته شوند.

برای مثال، یک ایجنت برنامه‌نویسی ممکن است بتواند فایل‌های یک پروژه را بررسی کند، کد بنویسد و آزمایش‌ها را اجرا کند؛ درحالی‌که یک ایجنت پژوهشی بیشتر بر جست‌وجوی منابع، گردآوری اطلاعات و تهیه‌ی گزارش تمرکز دارد.

با این حال، دسته‌بندی ایجنت‌ها کاملاً ثابت و انحصاری نیست. ممکن است یک ایجنت چندین قابلیت داشته باشد یا بتوان آن را با ابزارهای جدید برای انجام وظایف متفاوتی به کار گرفت. بنابراین، هنگام شناخت یک ایجنت باید علاوه بر نام و دسته‌ی آن، قابلیت‌های واقعی، ابزارهای قابل‌استفاده و محدودیت‌هایش را نیز بررسی کرد.

در ادامه، مهم‌ترین دسته‌های ایجنت‌ها را بر اساس کاربرد معرفی می‌کنیم.

ایجنت‌های برنامه‌نویسی (Coding Agents)

ایجنت‌های برنامه‌نویسی، سیستم‌هایی هستند که برای کمک به توسعه‌ی نرم‌افزار و انجام وظایف مرتبط با کدنویسی طراحی شده‌اند. این ایجنت‌ها، در صورت داشتن ابزارها و دسترسی‌های لازم، می‌توانند فراتر از پیشنهاد کد عمل کنند و بخشی از فرایند توسعه‌ی یک پروژه را به عهده بگیرند.

قابلیت‌های ایجنت‌های برنامه‌نویسی

بسته به ابزار و محیط اجرا، این ایجنت‌ها می‌توانند فعالیت‌های زیر را انجام دهند:

- بررسی ساختار و فایل‌های یک پروژه
- خواندن و تحلیل کدهای موجود
- نوشتن کد جدید و ایجاد فایل
- اصلاح کدها و توسعه‌ی قابلیت‌های جدید
- جست‌وجو در فایل‌ها برای پیدا کردن بخش‌های مرتبط
- اجرای دستورهای ترمینال و برنامه‌ها
- اجرای تست‌ها و بررسی نتایج
- شناسایی خطاها و تلاش برای رفع آن‌ها
- کمک به بازبینی کد و بررسی تغییرات
- تهیه‌ی مستندات فنی برای پروژه

برای مثال، فرض کنیم می‌خواهیم یک برنامه‌ی ساده برای مدیریت وظایف روزانه بسازیم. می‌توانیم از ایجنت بخواهیم ابتدا ساختار پروژه را مشخص کند، سپس فایل‌های موردنیاز را بسازد، قابلیت افزودن و حذف وظایف را پیاده‌سازی کند و در نهایت با اجرای آزمایش‌های مناسب، عملکرد برنامه را بررسی کند.

البته همه‌ی ایجنت‌های برنامه‌نویسی تمام این امکانات را ندارند. برخی تنها می‌توانند کد پیشنهاد دهند، درحالی‌که برخی دیگر امکان تغییر مستقیم فایل‌ها، اجرای برنامه و استفاده از ابزارهای توسعه را دارند.

ایجنت‌های برنامه‌نویسی برای چه پروژه‌هایی مناسب‌اند؟

این ایجنت‌ها می‌توانند در توسعه‌ی وب‌سایت‌ها، برنامه‌های موبایل، نرم‌افزارهای دسکتاپ، ابزارهای کوچک، اسکریپت‌های خودکارسازی، پروژه‌های تحلیل داده و برخی پروژه‌های هوش مصنوعی کمک‌کننده باشند.

برای مثال، در یک پروژه‌ی سلامت دیجیتال می‌توان از ایجنت برنامه‌نویسی برای ساخت رابط کاربری، پیاده‌سازی بخش ثبت اطلاعات، نوشتن کد پردازش داده‌ها یا تهیه‌ی تست‌ها استفاده کرد. با این حال، در پروژه‌های پزشکی یا پروژه‌هایی که با اطلاعات حساس سروکار دارند، صحت نتایج، امنیت و حریم خصوصی باید به‌طور مستقل بررسی شوند.

*نکته: ایجنت برنامه‌نویسی جایگزین کامل دانش فنی نیست. برای پروژه‌های پیچیده، کاربر باید بتواند معماری کلی، کیفیت کد، امنیت و نتیجه‌ی اجرای برنامه را ارزیابی کند.

ایجنت‌های پژوهشی و تحلیل اطلاعات (Research and Analysis Agents)

ایجنت‌های پژوهشی برای کمک به جست‌وجو، گردآوری، سازمان‌دهی و تحلیل اطلاعات استفاده می‌شوند. هدف آن‌ها این است که فرایند تحقیق را منظم‌تر کنند و به کاربر در تبدیل حجم زیادی از اطلاعات به یک خروجی قابل‌استفاده کمک کنند.

قابلیت‌های ایجنت‌های پژوهشی

بسته به ابزارهای در دسترس، این ایجنت‌ها می‌توانند:

- موضوع پژوهش را به پرسش‌های کوچک‌تر تقسیم کنند.
- منابع مرتبط را جست‌وجو و شناسایی کنند.
- اطلاعات منابع مختلف را استخراج و دسته‌بندی کنند.
- چند منبع را از نظر محتوا و نتیجه با یکدیگر مقایسه کنند.
- نکات اصلی مقالات یا گزارش‌ها را خلاصه کنند.
- الگوها، شباهت‌ها و تفاوت‌های موجود در اطلاعات را شناسایی کنند.

- پیش‌نویس گزارش یا خلاصه‌ی پژوهشی تهیه کنند.

- برای ادامه‌ی تحقیق، پرسش‌ها و مسیرهای احتمالی پیشنهاد دهند.

برای مثال، اگر موضوع پژوهش «کاربرد هوش مصنوعی در تشخیص بیماری‌ها» باشد، ایجنت می‌تواند به جست‌وجوی مقالات مرتبط، دسته‌بندی کاربردها، خلاصه‌سازی یافته‌ها و تهیه‌ی یک گزارش اولیه کمک کند.

با این حال، تولید یک گزارش منسجم به معنای معتبر بودن تمام اطلاعات آن نیست. منابع باید از نظر اعتبار، تاریخ انتشار، روش پژوهش و ارتباط با موضوع بررسی شوند. همچنین ارجاعات علمی و نقل‌قول‌ها باید با منابع اصلی تطبیق داده شوند.

تفاوت جست‌وجو با پژوهش

جست‌وجو معمولاً به یافتن اطلاعات مرتبط با یک پرسش محدود می‌شود؛ اما پژوهش ممکن است به مجموعه‌ای از فعالیت‌ها مانند تعیین پرسش، انتخاب منابع، مقایسه‌ی شواهد، تحلیل نتایج و تدوین گزارش نیاز داشته باشد.

ایجنت پژوهشی می‌تواند چند مرحله از این فرایند را به هم متصل کند، اما جایگزین ارزیابی علمی، روش‌شناسی پژوهش یا مسئولیت پژوهشگر در تأیید نتایج نیست.

ایجنت‌های پژوهشی برای چه پروژه‌هایی مناسب‌اند؟

این ایجنت‌ها برای مرور اولیه‌ی ادبیات علمی، تحقیقات بازار، بررسی رقبا، جمع‌آوری اطلاعات درباره‌ی یک فناوری، تهیه‌ی گزارش‌های تحلیلی و شناخت یک حوزه‌ی تخصصی مفید هستند.

برای نمونه، یک تیم استارتاپی می‌تواند از ایجنت پژوهشی برای شناسایی مشکلات کاربران، بررسی راهکارهای موجود و تهیه‌ی فهرستی اولیه از رقبا استفاده کند. با این حال، برای تصمیم‌گیری نهایی درباره‌ی نیاز واقعی کاربران، همچنان به شواهد معتبر و در صورت لزوم مصاحبه یا تحقیق میدانی نیاز است.

ایجنت‌های آموزشی (Educational Agents)

ایجنت‌های آموزشی برای کمک به فرایند یادگیری طراحی می‌شوند. این سیستم‌ها می‌توانند در توضیح مفاهیم، پاسخ‌گویی به پرسش‌های درسی، طراحی تمرین، بررسی پاسخ‌ها و برنامه‌ریزی مطالعه نقش داشته باشند.

قابلیت‌های ایجنت‌های آموزشی

با توجه به امکانات سیستم، این ایجنت‌ها می‌توانند:

- مفاهیم دشوار را با زبان ساده توضیح دهند.
- برای یک موضوع مثال و تمرین تولید کنند.
- بر اساس سطح فعلی یادگیرنده، مسیر آموزشی پیشنهاد دهند.
- پاسخ‌های کاربر را بررسی و درباره‌ی اشتباهات بازخورد ارائه کنند.
- مطالب طولانی را خلاصه و به یادداشت‌های آموزشی تبدیل کنند.
- از محتوای یک دوره، جزوه یا فایل آموزشی برای پاسخ‌گویی استفاده کنند.
- برنامه‌ی مطالعه و مرور مطالب را پیشنهاد دهند.
- میزان پیشرفت یادگیری را بر اساس اطلاعات موجود پیگیری کنند.

برای مثال، دانشجویی را در نظر بگیریم که در حال یادگیری یادگیری عمیق (Deep Learning) است. یک ایجنت آموزشی می‌تواند محتوای یک جلسه را خلاصه کند، اصطلاحات تخصصی را توضیح دهد، چند سؤال برای ارزیابی درک مطلب بسازد و بر اساس نتیجه‌ی تمرین‌ها، موضوعاتی را برای مرور مجدد پیشنهاد کند.

اگر ایجنت به فایل‌های دوره یا ابزارهای پردازش محتوا دسترسی داشته باشد، می‌تواند بخشی از کارهای مربوط به مطالعه‌ی دوره را نیز انجام دهد. اما برای خلاصه‌سازی دقیق ویدئوها، ممکن است به متن پیاده‌سازی‌شده، زیرنویس یا قابلیت پردازش صوت و تصویر نیاز داشته باشد.

کاربردهای ایجنت‌های آموزشی

این دسته از ایجنت‌ها برای یادگیری زبان، برنامه‌نویسی، دروس دانشگاهی، آمادگی برای آزمون‌ها، یادگیری مهارت‌های تخصصی و سازمان‌دهی دوره‌های آموزشی مفید هستند.

البته ایجنت آموزشی ممکن است پاسخ اشتباه بدهد یا سطح یادگیری فرد را نادرست تشخیص دهد. بنابراین، برای یادگیری مفاهیم پایه و تخصصی باید پاسخ‌ها بررسی شوند و در موضوعات مهم، منابع معتبر نیز مورد استفاده قرار گیرند.

ایجنت‌های تولید محتوا (Content Creation Agents)

ایجنت‌های تولید محتوا برای کمک به طراحی، تولید، بازبینی و سازمان‌دهی محتوای متنی یا رسانه‌ای به کار می‌روند. این ایجنت‌ها می‌توانند در یک مرحله از تولید محتوا کمک کنند یا در صورت دسترسی به ابزارهای مناسب، چند مرحله از فرایند تولید و انتشار را هماهنگ کنند.

قابلیت‌های ایجنت‌های تولید محتوا

از جمله قابلیت‌های آن‌ها می‌توان به موارد زیر اشاره کرد:

- پیشنهاد ایده برای محتوا
- شناخت مخاطب هدف بر اساس اطلاعات ارائه‌شده
- تهیه‌ی تقویم محتوایی
- نوشتن پیش‌نویس مقاله، پست یا متن تبلیغاتی
- بازنویسی و ویرایش متن
- خلاصه‌سازی و تبدیل یک محتوا به قالب‌های دیگر
- پیشنهاد عنوان و ساختار محتوا
- کمک به تولید تصویر یا سایر قالب‌های رسانه‌ای با استفاده از ابزارهای مرتبط
- بررسی و آماده‌سازی محتوا برای انتشار

برای مثال، یک کسب‌وکار کوچک می‌تواند از ایجنت تولید محتوا برای برنامه‌ریزی پست‌های شبکه‌های اجتماعی، تهیه‌ی پیش‌نویس متن‌ها و هماهنگ‌کردن محتوای یک کمپین استفاده کند.

اگر ایجنت به ابزار انتشار متصل باشد و مجوزهای لازم را داشته باشد، ممکن است بتواند محتوا را در زمان مشخص منتشر کند. در غیر این صورت، فقط محتوای آماده‌ی انتشار را تحویل می‌دهد.

تفاوت تولید محتوا با مدیریت کامل محتوا

تولید محتوا به ساخت متن، تصویر یا قالب رسانه‌ای اشاره دارد؛ اما مدیریت محتوا می‌تواند شامل برنامه‌ریزی، زمان‌بندی، انتشار، بررسی بازخورد و اصلاح برنامه‌ی محتوایی نیز باشد.

یک ایجنت ممکن است تنها در تولید محتوا تخصص داشته باشد یا با اتصال به ابزارهای دیگر، بخشی از این چرخه را مدیریت کند. در هر دو حالت، بررسی صحت اطلاعات، تناسب محتوا با مخاطب و رعایت حقوق دیگران بر عهده‌ی کاربر یا تیم مسئول باقی می‌ماند.

ایجنت‌های اتوماسیون و انجام کارهای تکراری (Automation Agents)

ایجنت‌های اتوماسیون برای انجام یا هماهنگ‌کردن وظایف تکراری و فرایندهای چندمرحله‌ای استفاده می‌شوند. هدف آن‌ها کاهش کارهای دستی و اتصال ابزارها و سرویس‌های مختلف به یکدیگر است.

قابلیت‌های ایجنت‌های اتوماسیون

این ایجنت‌ها ممکن است بتوانند:

- اطلاعات را از یک منبع دریافت و در منبع دیگری ثبت کنند.

- داده‌ها را دسته‌بندی و مرتب کنند.

- فرم‌ها یا گزارش‌های مشخصی را بر اساس اطلاعات موجود آماده کنند.

- پیام‌ها یا اعلان‌ها را بر اساس شرایط تعیین‌شده ارسال کنند.

- اطلاعات چند ابزار یا سرویس را با یکدیگر هماهنگ کنند.

- وظایف مشخصی را بر اساس زمان‌بندی یا یک رویداد آغاز کنند.

- درخواست‌های ساده را بررسی و به مسیر مناسب هدایت کنند.

برای مثال، فرض کنیم یک تیم پشتیبانی هر روز اطلاعات درخواست‌های کاربران را از یک فرم دریافت می‌کند. یک فرایند خودکار می‌تواند اطلاعات را دسته‌بندی کند، موارد را در یک صفحه‌ی گسترده ثبت کند و برای درخواست‌های جدید اعلان بفرستد.

اگر در این فرایند از ایجنت استفاده شود، سیستم ممکن است علاوه بر اجرای مراحل ثابت، متن درخواست را تحلیل کند و دسته‌ی مناسب را پیشنهاد دهد. با این حال، همه‌ی اتوماسیون‌ها به ایجنت نیاز ندارند؛ بسیاری از کارهای کاملاً قابل‌پیش‌بینی را می‌توان با قواعد ثابت و ابزارهای خودکارسازی معمولی انجام داد.

چه زمانی ایجنت برای اتوماسیون مناسب است؟

وقتی مراحل کار کاملاً مشخص و ثابت‌اند، یک فرایند خودکار مبتنی بر قواعد ممکن است ساده‌تر، ارزان‌تر و قابل‌اعتمادتر باشد. اما زمانی که درخواست‌ها متنوع‌اند، اطلاعات ساختار ثابتی ندارند یا لازم است بر اساس شرایط متفاوت تصمیم‌گیری شود، استفاده از ایجنت می‌تواند مفید باشد.

برای مثال، انتقال اطلاعات از یک ستون صفحه‌گسترده به ستون دیگر معمولاً به ایجنت نیاز ندارد؛ اما بررسی متن‌های متنوع کاربران و پیشنهاد دسته‌ی مناسب برای هر درخواست ممکن است از قابلیت‌های یک مدل هوش مصنوعی بهره ببرد.

ایجنت‌های فرعی (Subagents) و نقش آن‌ها در تقسیم وظایف

ایجنت‌های فرعی (Subagents) ایجنت‌هایی هستند که برای انجام بخشی از وظایف یک ایجنت اصلی به کار گرفته می‌شوند. در این ساختار، ایجنت اصلی می‌تواند یک وظیفه‌ی بزرگ را به چند وظیفه‌ی کوچک‌تر تقسیم کند و انجام هر بخش را به یک ایجنت فرعی بسپارد. ایجنت‌های فرعی وظایف مشخصی را انجام می‌دهند و نتیجه‌ی کار خود را برای استفاده یا بررسی در اختیار ایجنت اصلی قرار می‌دهند.

نحوه‌ی کار ایجنت‌های فرعی

فرایند کار با ایجنت‌های فرعی معمولاً شامل مراحل زیر است:

تعیین وظیفه‌ی اصلی: ایجنت اصلی درخواست کاربر را بررسی می‌کند و مشخص می‌کند چه کارهایی باید انجام شوند.

تقسیم وظایف: در صورت نیاز، وظیفه‌ی اصلی به چند بخش مستقل و کوچک‌تر تقسیم می‌شود.

واگذاری وظایف: ایجنت اصلی هر بخش را به یک ایجنت فرعی مناسب واگذار می‌کند.

اجرای وظایف: ایجنت‌های فرعی بخش‌های محول‌شده را انجام می‌دهند و نتایج خود را ارائه می‌کنند.

بررسی و ترکیب نتایج: ایجنت اصلی خروجی‌ها را بررسی می‌کند و از آن‌ها برای تکمیل وظیفه‌ی اصلی استفاده می‌کند.

مثال کاربردی

فرض کنیم از یک ایجنت برنامه‌نویسی می‌خواهیم یک وب‌سایت بسازد. ایجنت اصلی می‌تواند بررسی ساختار پروژه را به یک ایجنت فرعی، بررسی خطاهای کد را به ایجنتی دیگر و نوشتن یا اجرای آزمون‌های نرم‌افزاری را به ایجنت فرعی دیگری بسپارد. سپس نتایج را بررسی کند و برای ادامه‌ی توسعه‌ی وب‌سایت تصمیم بگیرد.

مزایای استفاده از ایجنت‌های فرعی

- تقسیم وظایف پیچیده به بخش‌های کوچک‌تر و قابل مدیریت

- امکان انجام هم‌زمان برخی وظایف مستقل

- تمرکز هر ایجنت بر یک مسئولیت مشخص

- کاهش بار کاری ایجنت اصلی در پروژه‌های پیچیده

محدودیت‌ها و نکات مهم

استفاده از ایجنت‌های فرعی همیشه باعث افزایش سرعت یا کیفیت کار نمی‌شود. تعریف نادرست وظایف، تداخل میان مسئولیت‌ها، هزینه‌ی بیشتر پردازش و دشواری بررسی خروجی‌ها می‌توانند عملکرد سیستم را مختل کنند. همچنین، ایجنت اصلی باید نتایج دریافتی را بررسی کند؛ زیرا خروجی ایجنت‌های فرعی نیز ممکن است ناقص یا اشتباه باشد.

نکته‌ی مهم این است که ایجنت فرعی لزوماً یک نوع مستقل از ایجنت نیست، بلکه به نقشی اشاره دارد که یک ایجنت در ساختار کاری سیستم بر عهده می‌گیرد. همچنین، همه‌ی ایجنت‌ها قابلیت ساخت یا به‌کارگیری ایجنت‌های فرعی را ندارند و این امکان به طراحی و قابلیت‌های ابزار مورد استفاده بستگی دارد.

ایجنت‌های تخصصی و چندمنظوره

یکی دیگر از روش‌های دسته‌بندی ایجنت‌ها، توجه به گستره‌ی وظایفی است که برای انجام آن‌ها طراحی شده‌اند. از این نظر، می‌توان بین ایجنت‌های تخصصی و ایجنت‌های چندمنظوره تفاوت قائل شد.

ایجنت تخصصی (Specialized Agent)

ایجنت تخصصی برای یک حوزه یا مجموعه‌ای محدود از وظایف طراحی شده است. چنین ایجنتی ممکن است دستورالعمل‌ها، ابزارها یا روش‌های مشخصی برای انجام آن وظایف داشته باشد.

برای مثال، ایجنتی که برای بررسی و دسته‌بندی درخواست‌های پشتیبانی طراحی شده است، می‌تواند بر شناسایی نوع درخواست، استخراج اطلاعات لازم و پیشنهاد پاسخ تمرکز کند.

مزیت ایجنت تخصصی این است که می‌توان آن را بر اساس نیازهای مشخص یک فرایند تنظیم کرد. با این حال، توانایی آن در انجام وظایف خارج از حوزه‌ی طراحی‌شده ممکن است محدود باشد.

ایجنت چندمنظوره (General-Purpose Agent)

ایجنت چندمنظوره برای انجام طیف گسترده‌تری از وظایف طراحی شده است. چنین ایجنتی ممکن است بتواند در نوشتن متن، تحلیل اطلاعات، برنامه‌نویسی، کار با فایل‌ها یا استفاده از ابزارهای مختلف کمک کند.

برای مثال، یک ایجنت چندمنظوره ممکن است در یک پروژه ابتدا اطلاعات را بررسی کند، سپس گزارشی تهیه کند و در مرحله‌ی بعد، برای ساخت یک ابزار نرم‌افزاری ساده کمک کند؛ البته تنها در صورتی که قابلیت‌ها و ابزارهای لازم را داشته باشد.

گستردگی کاربرد به معنای برتری همیشگی نیست. برای یک کار مشخص، ایجنت تخصصی ممکن است به دلیل داشتن ابزارها یا دستورالعمل‌های متناسب با همان کار، انتخاب مناسب‌تری باشد.

مقایسه‌ی ایجنت‌های تخصصی و چندمنظوره

ایجنت تخصصی معمولاً برای وظیفه‌ای مشخص تنظیم شده است، درحالی‌که ایجنت چندمنظوره برای طیف گسترده‌تری از فعالیت‌ها به کار می‌رود. انتخاب بین این دو به پیچیدگی پروژه، تنوع وظایف، کیفیت موردنیاز و ابزارهای در دسترس بستگی دارد.

برای پروژه‌ای که یک فرایند تکراری و مشخص دارد، ایجنت تخصصی می‌تواند مناسب باشد. برای پروژه‌ای که شامل فعالیت‌های متنوع است، ایجنت چندمنظوره ممکن است انعطاف‌پذیری بیشتری فراهم کند. در برخی پروژه‌ها نیز می‌توان از ترکیب این دو استفاده کرد.

سیستم‌های چندایجنتی و همکاری چند ایجنت (Multi-Agent Systems)

در بعضی پروژه‌ها، یک ایجنت به‌تنهایی برای انجام تمام وظایف مناسب نیست یا تقسیم کار میان چند ایجنت می‌تواند مدیریت فرایند را ساده‌تر کند. در این شرایط، ممکن است از سیستم چندایجنتی استفاده شود.

سیستم چندایجنتی مجموعه‌ای از ایجنت‌هاست که در یک ساختار مشخص با یکدیگر تعامل می‌کنند تا وظایف یک پروژه را انجام دهند. این ایجنت‌ها می‌توانند وظایف متفاوتی داشته باشند و از طریق تبادل اطلاعات، نتایج کار یکدیگر را دریافت کنند.

نحوه‌ی تقسیم کار میان چند ایجنت

برای استفاده از چند ایجنت، ابتدا باید هدف کلی پروژه مشخص شود. سپس وظایف به قسمت‌هایی تقسیم می‌شوند که بتوان آن‌ها را به‌صورت مستقل یا با وابستگی مشخص انجام داد.

برای مثال، در یک پروژه‌ی توسعه‌ی نرم‌افزار می‌توان نقش‌های زیر را تعریف کرد:

ایجنت تحلیل‌گر: نیازمندی‌های پروژه را بررسی و وظایف لازم را مشخص می‌کند.

ایجنت برنامه‌نویس: کد بخش‌های تعیین‌شده را پیاده‌سازی می‌کند.

ایجنت آزمایش‌گر:تست‌های لازم را اجرا و مشکلات مشاهده‌شده را گزارش می‌کند.

ایجنت مستندساز:مستندات مربوط به نحوه‌ی استفاده و ساختار پروژه را آماده می‌کند.

این نقش‌ها صرفاً یک نمونه‌ی توضیحی هستند. در عمل، یک ایجنت ممکن است چند نقش را انجام دهد یا برای هر نقش از ابزارها و مدل‌های متفاوت استفاده شود.

روش‌های همکاری میان ایجنت‌ها

همکاری چند ایجنت می‌تواند به روش‌های مختلفی انجام شود:

- هماهنگ‌کننده و ایجنت‌های اجراکننده: یک ایجنت یا بخش کنترلی، وظایف را تقسیم می‌کند و نتایج را دریافت می‌کند.

- تقسیم وظایف موازی: چند ایجنت بخش‌های مستقلی از کار را هم‌زمان انجام می‌دهند.

- زنجیره‌ی وظایف: خروجی یک ایجنت به‌عنوان ورودی مرحله‌ی بعد استفاده می‌شود.

- بازبینی متقابل: یک ایجنت نتیجه‌ی تولیدشده توسط ایجنت دیگر را بررسی می‌کند.

این الگوها می‌توانند با یکدیگر ترکیب شوند. انتخاب روش مناسب به ساختار پروژه و وابستگی میان وظایف بستگی دارد.

مزایای استفاده از چند ایجنت

در صورت طراحی مناسب، سیستم چندایجنتی می‌تواند تقسیم کار را آسان‌تر کند، امکان انجام هم‌زمان وظایف مستقل را فراهم کند و به تفکیک مسئولیت‌ها کمک کند. برای مثال، در یک پروژه‌ی بزرگ، بررسی منابع و آماده‌سازی مستندات ممکن است هم‌زمان با توسعه‌ی بخش دیگری از پروژه انجام شود.

با این حال، چندایجنتی‌بودن به‌خودی‌خود کیفیت یا سرعت را تضمین نمی‌کند. هماهنگی میان ایجنت‌ها، انتقال صحیح اطلاعات، مدیریت وابستگی‌ها و جلوگیری از تداخل تغییرات می‌تواند دشوار باشد.

چه زمانی به چند ایجنت نیاز داریم؟

استفاده از چند ایجنت زمانی ارزشمند است که وظایف به‌درستی قابل‌تفکیک باشند، هر ایجنت نقش مشخصی داشته باشد و هزینه‌ی هماهنگی از منفعت تقسیم کار کمتر باشد.

برای یک پروژه‌ی کوچک، استفاده از یک ایجنت ممکن است کافی باشد. اضافه‌کردن ایجنت‌های متعدد به چنین پروژه‌ای می‌تواند فرایند را پیچیده‌تر کند و هزینه‌ی بیشتری به همراه داشته باشد.

بنابراین، بهتر است ابتدا ساختار و نیازهای پروژه مشخص شوند و سپس درباره‌ی استفاده از یک یا چند ایجنت تصمیم گرفته شود.

ایجنت‌ها را می‌توان بر اساس کاربرد به دسته‌هایی مانند برنامه‌نویسی، پژوهش و تحلیل اطلاعات، آموزش، تولید محتوا و اتوماسیون تقسیم کرد. همچنین از نظر گستره‌ی کاربرد، می‌توان آن‌ها را تخصصی یا چندمنظوره در نظر گرفت. در پروژه‌های پیچیده‌تر نیز امکان استفاده از سیستم‌های چندایجنتی وجود دارد.

هر دسته برای مجموعه‌ای از وظایف مناسب‌تر است، اما هیچ دسته‌ای برای تمام پروژه‌ها بهترین انتخاب نیست. توانایی‌های واقعی هر ایجنت به طراحی، مدل، ابزارها، دسترسی‌ها و محدودیت‌های آن بستگی دارد.

برای انتخاب درست، باید ابتدا هدف و نیازهای پروژه را مشخص کنیم و سپس بررسی کنیم که کدام ایجنت یا ترکیب ایجنت‌ها می‌تواند آن نیازها را با هزینه، پیچیدگی و سطح نظارت قابل‌قبول برآورده کند.

فصل4: انتخاب ایجنت مناسب

معیارهای انتخاب ایجنت

با افزایش تعداد ایجنت‌های هوش مصنوعی، انتخاب ابزار مناسب به یکی از مراحل مهم شروع هر پروژه تبدیل شده است. ایجنتی که برای برنامه‌نویسی مناسب است، لزوماً بهترین گزینه برای پژوهش علمی، تولید محتوا یا خودکارسازی کارهای روزمره نیست. بنابراین، پیش از انتخاب یک ابزار، باید هدف پروژه، نوع وظایف، امکانات موردنیاز و محدودیت‌های خود را مشخص کنیم.

برای انتخاب ایجنت مناسب، می‌توان معیارهای زیر را در نظر گرفت.

هدف و نوع پروژه

اولین معیار، کاری است که می‌خواهیم انجام دهیم. برای مثال، اگر هدف ما توسعه‌ی یک وب‌سایت باشد، به ایجنتی نیاز داریم که بتواند کدها را بخواند، فایل‌ها را ویرایش کند و در صورت امکان، کدها را اجرا و آزمایش کند. اما برای پژوهش علمی، توانایی جست‌وجوی اطلاعات، بررسی منابع و ارائه‌ی پاسخ‌های مستند اهمیت بیشتری دارد.

قابلیت‌ها و ابزارهای در دسترس

هر ایجنت مجموعه‌ای از قابلیت‌ها و ابزارها را در اختیار دارد. برخی فقط متن تولید می‌کنند، درحالی‌که برخی دیگر می‌توانند فایل‌ها را بخوانند، در اینترنت جست‌وجو کنند، کد اجرا کنند یا به سرویس‌های دیگر متصل شوند.

پیش از انتخاب، باید بررسی کنیم که آیا ابزار موردنظر امکانات لازم برای انجام وظیفه‌ی ما را دارد یا خیر. همچنین، داشتن یک قابلیت در معرفی محصول به این معنا نیست که آن قابلیت در تمام نسخه‌ها یا شرایط استفاده در دسترس است.

میزان استقلال و کنترل کاربر

برخی ابزارها برای هر مرحله به راهنمایی کاربر نیاز دارند؛ برخی دیگر می‌توانند چندین مرحله را با نظارت محدود انجام دهند. هرچه ایجنت اختیار بیشتری داشته باشد، بررسی دسترسی‌های آن و کنترل اقداماتی که انجام می‌دهد اهمیت بیشتری پیدا می‌کند.

برای مثال، اجازه‌ی پیشنهاد تغییرات در کد با اجازه‌ی حذف فایل‌ها یا انتشار مستقیم یک برنامه در محیط واقعی یکسان نیست. بهتر است میزان استقلال ایجنت با اهمیت و حساسیت وظیفه متناسب باشد.

سهولت استفاده و سطح دانش موردنیاز

برخی ایجنت‌ها رابط کاربری ساده‌ای دارند و با نوشتن درخواست به زبان طبیعی می‌توان با آن‌ها کار کرد. برخی دیگر به آشنایی با محیط برنامه‌نویسی، ترمینال، تنظیمات یا مفاهیم فنی نیاز دارند.

ترمینال (Terminal) محیطی متنی برای اجرای دستورها در سیستم‌عامل است. ابزارهایی که از طریق ترمینال کار می‌کنند ممکن است انعطاف بیشتری داشته باشند، اما معمولاً برای کاربران مبتدی به یادگیری بیشتری نیاز دارند.

هزینه و محدودیت استفاده

برخی ابزارها نسخه‌ی رایگان یا آزمایشی دارند و برخی دیگر برای استفاده‌ی گسترده‌تر به اشتراک پولی نیاز دارند. محدودیت تعداد درخواست‌ها، میزان پردازش، دسترسی به مدل‌های پیشرفته و هزینه‌ی اتصال به سرویس‌های خارجی نیز باید بررسی شود.

رایگان بودن یک ابزار به معنای نامحدود بودن آن نیست؛ همچنین، هزینه‌ی واقعی یک پروژه ممکن است علاوه بر اشتراک ایجنت، شامل هزینه‌ی مدل هوش مصنوعی، API، سرور یا سرویس‌های جانبی نیز باشد.

سازگاری با ابزارها و محیط کاری

ایجنت باید بتواند با محیطی که در آن کار می‌کنیم سازگار باشد. برای نمونه، در پروژه‌ای که با Python و Django ساخته شده است، دسترسی مناسب به فایل‌های پروژه، محیط اجرای Python و ابزارهای آزمایش می‌تواند بسیار مفید باشد.

همچنین، اگر هدف ساخت یک سیستم خودکار برای دریافت پیام‌های تلگرام و ثبت اطلاعات در Google Sheets باشد، باید بررسی کنیم که ابزار انتخابی از اتصال به این سرویس‌ها پشتیبانی می‌کند یا امکان پیاده‌سازی چنین اتصالی را فراهم می‌سازد.

امنیت و حفظ حریم خصوصی

اگر پروژه شامل اطلاعات شخصی، اسناد محرمانه، رمزهای عبور یا اطلاعات حساس سازمانی باشد، باید سیاست‌های نگهداری داده، نحوه‌ی دسترسی ایجنت، محل پردازش اطلاعات و امکان محدودکردن دسترسی‌ها بررسی شود.

نباید صرفاً برای راحتی کار، دسترسی کامل به فایل‌ها، حساب‌ها یا اطلاعات محرمانه را در اختیار یک ایجنت قرار داد. اصل مناسب این است که ایجنت فقط به اطلاعات و ابزارهایی دسترسی داشته باشد که برای انجام وظیفه‌ی خود نیاز دارد.

کیفیت و قابلیت بررسی خروجی

ایجنت مناسب باید بتواند خروجی قابل ارزیابی ارائه دهد. برای مثال، در برنامه‌نویسی امکان اجرای آزمون‌ها و مشاهده‌ی خطاها اهمیت دارد؛ در پژوهش، امکان بررسی منابع و تطبیق ادعاها با شواهد مهم است؛ و در اتوماسیون، ثبت مراحل اجرا و مشاهده‌ی نتیجه می‌تواند به شناسایی خطاها کمک کند.

هیچ ایجنتی را نباید صرفاً بر اساس ظاهر جذاب، شهرت یا ادعای سازنده انتخاب کرد. بهتر است آن را با یک وظیفه‌ی کوچک و واقعی آزمایش کنیم و نتیجه را با معیارهای مشخص بسنجیم.

جمع‌بندی معیارهای انتخاب

پیش از انتخاب ایجنت، به این پرسش‌ها پاسخ دهید:

- دقیقاً چه کاری می‌خواهم انجام دهم؟
- چه قابلیت‌هایی برای این کار ضروری هستند؟
- چه مقدار دانش فنی دارم؟
- چقدر هزینه و زمان می‌توانم صرف کنم؟
- چه سطحی از دسترسی و استقلال برای ایجنت مناسب است؟

- چگونه می‌توانم درستی نتیجه را بررسی کنم؟

پاسخ به این پرسش‌ها، گزینه‌های نامناسب را کنار می‌گذارد و انتخاب را ساده‌تر می‌کند.

انتخاب ایجنت مناسب برای انواع پروژه‌ها

پس از مشخص‌کردن معیارهای انتخاب، باید آن‌ها را با نیازهای پروژه‌ی خود تطبیق دهیم. برای این کار، ابتدا مشخص می‌کنیم پروژه به چه نوع فعالیت‌هایی نیاز دارد و سپس ابزارهایی را انتخاب می‌کنیم که این فعالیت‌ها را به‌خوبی پشتیبانی می‌کنند.

پروژه‌های برنامه‌نویسی و توسعه‌ی نرم‌افزار

برای ساخت وب‌سایت، توسعه‌ی اپلیکیشن، اصلاح کد یا بررسی خطاهای نرم‌افزاری، بهتر است از ایجنت‌های برنامه‌نویسی استفاده کنیم. این ابزارها می‌توانند با توجه به قابلیت‌هایشان، فایل‌های پروژه را بررسی کنند، کد بنویسند یا تغییر دهند، دستورهای برنامه‌نویسی را اجرا کنند و در بررسی خطاها کمک کنند.

در این نوع پروژه‌ها، دسترسی به محیط توسعه، سازگاری با زبان برنامه‌نویسی و چارچوب مورد استفاده، امکان اجرای آزمون‌ها و کنترل تغییرات اهمیت زیادی دارد.

پروژه‌های پژوهشی و تحلیل اطلاعات

اگر هدف، بررسی یک موضوع علمی، جمع‌آوری اطلاعات، مقایسه‌ی منابع یا تهیه‌ی گزارش باشد، باید به سراغ ابزارهایی برویم که در جست‌وجو، تحلیل و سازمان‌دهی اطلاعات عملکرد مناسبی دارند.

در این پروژه‌ها، صرفاً تولید یک متن روان کافی نیست. قابلیت دسترسی به منابع معتبر، امکان بررسی ارجاعات، به‌روز بودن اطلاعات و تفکیک شواهد از برداشت‌های احتمالی اهمیت بیشتری دارد.

پروژه‌های آموزشی و یادگیری

برای یادگیری برنامه‌نویسی، مطالعه‌ی یک دوره‌ی آموزشی یا آمادگی برای امتحان، ابزارهایی مفید هستند که بتوانند مفاهیم را متناسب با سطح دانش کاربر توضیح دهند، تمرین طراحی کنند، پاسخ‌ها را بررسی کنند و برای مطالعه برنامه‌ریزی کنند.

در این کاربرد، بهتر است ابزاری انتخاب شود که امکان پرسش‌وپاسخ مرحله‌ای و دریافت بازخورد را فراهم کند؛ نه اینکه فقط پاسخ نهایی را ارائه دهد.

پروژه‌های تولید محتوا

برای تولید مقاله، تدوین محتوای شبکه‌های اجتماعی، تهیه‌ی سناریو یا تنظیم تقویم محتوایی، ابزارهایی مناسب‌اند که در ایده‌پردازی، نگارش، بازنویسی و حفظ لحن موردنظر کمک کنند.

اگر پروژه به انتشار خودکار محتوا نیز نیاز داشته باشد، باید بررسی کنیم که ابزار انتخابی امکان اتصال به سامانه‌ی انتشار یا سرویس موردنظر را دارد یا خیر. توانایی تولید متن به‌تنهایی به معنای توانایی انتشار خودکار آن نیست.

پروژه‌های اتوماسیون و انجام کارهای تکراری

اگر هدف، خودکارسازی فعالیت‌هایی مانند دریافت پیام، دسته‌بندی درخواست‌ها، ثبت اطلاعات در جدول، ارسال اعلان یا ارتباط میان چند سرویس باشد، ابزارهای اتوماسیون و ساخت گردش کار می‌توانند انتخاب مناسبی باشند.

گردش کار (Workflow) مجموعه‌ای از مراحل مشخص است که برای انجام یک فرایند به یکدیگر متصل می‌شوند. در برخی فرایندها، مراحل از پیش تعیین‌شده کافی‌اند؛ اما در کارهایی که به تفسیر درخواست، انتخاب اقدام یا تصمیم‌گیری بر اساس شرایط متغیر نیاز دارند، می‌توان از ایجنت در کنار گردش کار استفاده کرد.

برای مثال، می‌توان سیستمی طراحی کرد که پیام‌های ورودی تلگرام را دریافت کند، موضوع آن‌ها را تشخیص دهد و اطلاعات مربوط را در Google Sheets ثبت کند. اجرای چنین پروژه‌ای به ابزار مناسب، اتصال به سرویس‌ها، تنظیم دسترسی‌ها و آزمایش فرایند نیاز دارد.

پروژه‌های پیچیده و چندمرحله ای

برای پروژه‌هایی که شامل چند فعالیت متفاوت هستند، ممکن است ترکیب چند ابزار یا استفاده از چند ایجنت مفید باشد. برای نمونه، یک پروژه‌ی پژوهشی می‌تواند به جمع‌آوری اطلاعات، تحلیل داده‌ها، نگارش گزارش و بازبینی نهایی نیاز داشته باشد.

بااین‌حال، بهتر است ابتدا بررسی کنیم آیا یک ایجنت با قابلیت‌های کافی می‌تواند تمام مراحل را انجام دهد یا خیر. اضافه‌کردن ابزارهای بیشتر، بدون نیاز واقعی، ممکن است هزینه و پیچیدگی مدیریت پروژه را افزایش دهد.

مقایسه‌ی قابلیت‌ها، ابزارها، هزینه و سهولت استفاده

برای مقایسه‌ی ایجنت‌ها، بهتر است به‌جای انتخاب بر اساس شهرت یا تبلیغات، چند ابزار را با معیارهای یکسان بررسی کنیم.

معیار مقایسه	پرسش اصلی
نوع کاربرد	ابزار برای چه نوع وظایفی طراحی شده است؟
قابلیت‌ها	آیا می‌تواند وظایف موردنیاز پروژه را انجام دهد؟
ابزارهای جانبی	آیا به فایل‌ها، اینترنت، ترمینال یا سرویس‌های دیگر دسترسی دارد؟
سهولت استفاده	آیا کار با آن برای سطح دانش فعلی ما مناسب است؟
هزینه	چه هزینه‌ای برای اشتراک، مصرف مدل یا خدمات جانبی دارد؟
استقلال	چه میزان از کار را می‌تواند بدون دخالت مداوم کاربر انجام دهد؟
امنیت	چگونه می‌توان دسترسی‌ها و اطلاعات حساس را کنترل کرد؟
ارزیابی خروجی	آیا امکان آزمایش، بررسی و پیگیری مراحل اجرا وجود دارد؟

برای انتخاب دقیق‌تر، می‌توان یک وظیفه‌ی یکسان را به چند ابزار داد و خروجی آن‌ها را مقایسه کرد. برای مثال، در ارزیابی ایجنت‌های برنامه‌نویسی می‌توان از آن‌ها خواست یک خطای مشخص را در پروژه‌ای آزمایشی پیدا کنند، علت خطا را توضیح دهند و راه‌حل پیشنهادی خود را با آزمون بررسی کنند.

در این مقایسه باید دقت، میزان تغییرات غیرضروری، کیفیت توضیحات، زمان انجام کار و نیاز به اصلاح انسانی را نیز در نظر بگیریم.

نکته‌ی مهم این است که نتیجه‌ی یک آزمایش کوچک، عملکرد ابزار را در تمام پروژه‌ها تضمین نمی‌کند. بااین‌حال، آزمایش عملی معمولاً اطلاعات مفیدتری از تکیه بر ادعاهای تبلیغاتی در اختیار ما قرار می‌دهد.

انتخاب بین ایجنت‌های رایگان و پولی

یکی از پرسش‌های رایج هنگام شروع کار این است که آیا باید از نسخه‌ی رایگان استفاده کرد یا اشتراک پولی خرید. پاسخ به این پرسش به نوع پروژه، میزان استفاده و امکانات موردنیاز بستگی دارد.

ایجنت‌های رایگان

نسخه‌های رایگان می‌توانند برای آشنایی اولیه، یادگیری، آزمایش ایده‌ها و انجام وظایف محدود مناسب باشند. استفاده از آن‌ها به کاربر کمک می‌کند بدون پرداخت هزینه‌ی اولیه، محیط ابزار را بشناسد و بررسی کند که آیا قابلیت‌های آن با نیازهایش سازگار است یا خیر.

بااین‌حال، ممکن است نسخه‌ی رایگان محدودیت‌هایی در تعداد درخواست‌ها، دسترسی به مدل‌ها، اجرای وظایف طولانی یا استفاده از امکانات پیشرفته داشته باشد.

ایجنت‌های پولی

نسخه‌های پولی ممکن است امکانات بیشتری مانند سقف استفاده‌ی بالاتر، دسترسی به مدل‌های پیشرفته‌تر، قابلیت‌های توسعه‌یافته یا امکانات مناسب برای کارهای حرفه‌ای ارائه دهند. البته پولی بودن یک ابزار لزوماً به معنای بهتر بودن آن برای همه‌ی کاربران نیست.

همچنین، باید میان هزینه‌ی اشتراک و هزینه‌ی مصرف تفکیک قائل شویم. برخی سرویس‌ها بر اساس اشتراک و برخی دیگر بر اساس میزان استفاده یا تعداد درخواست‌های پردازشی هزینه دریافت می‌کنند. در پروژه‌های بزرگ، هزینه‌ی سرویس‌های جانبی نیز ممکن است قابل توجه باشد.

روش پیشنهادی برای انتخاب

۱. ابتدا با نسخه‌ی رایگان یا آزمایشی شروع کنید، اگر چنین امکانی وجود دارد.

۲. یک وظیفه‌ی واقعی و کوچک را با آن انجام دهید.

۳. محدودیت‌هایی را که در عمل با آن‌ها روبه‌رو می‌شوید ثبت کنید.

۴. بررسی کنید آیا نسخه‌ی پولی دقیقاً همان محدودیت‌ها را برطرف می‌کند یا خیر.

۵. هزینه‌ی نهایی را با ارزش زمانی و کیفیتی که به دست می‌آورید مقایسه کنید.

برای مثال، اگر از یک ایجنت فقط برای توضیح مفاهیم برنامه‌نویسی استفاده می‌کنید، شاید نسخه‌ی رایگان نیاز شما را برطرف کند. اما اگر روزانه روی یک پروژه‌ی نرم‌افزاری کار می‌کنید و به بررسی تعداد زیادی فایل، اجرای آزمون‌ها یا انجام وظایف طولانی نیاز دارید، ممکن است پرداخت هزینه برای ابزار مناسب توجیه‌پذیر باشد.

در نهایت، بهتر است تا زمانی که نیاز واقعی خود را نشناخته‌اید، برای خرید چند اشتراک مختلف عجله نکنید.

چه زمانی یک ایجنت کافی است و چه زمانی به چند ایجنت نیاز داریم؟

همه‌ی پروژه‌ها به چند ایجنت نیاز ندارند. گاهی یک ایجنت توانمند که به ابزارهای لازم دسترسی دارد، می‌تواند تمام مراحل موردنیاز را انجام دهد. در مقابل، بعضی پروژه‌ها به چند ایجنت تخصصی یا یک سیستم چندایجنتی نیاز دارند.

استفاده از یک ایجنت مناسب است، زمانی که:

- پروژه کوچک یا متوسط و وظایف آن نسبتاً مشخص هستند.
- یک ابزار می‌تواند تمام قابلیت‌های موردنیاز را فراهم کند.
- هماهنگی میان چند ایجنت ارزش افزوده‌ی قابل توجهی ایجاد نمی‌کند.
- سادگی، هزینه‌ی کمتر و سهولت مدیریت اهمیت دارند.

استفاده از چند ایجنت می‌تواند مفید باشد، زمانی که:

- پروژه به چند تخصص یا مجموعه وظایف متفاوت نیاز دارد.
- بخش‌های مستقل پروژه را می‌توان به‌صورت موازی پیش برد.
- تقسیم وظایف باعث افزایش قابل‌توجه سرعت یا کیفیت می‌شود.
- هماهنگی و بررسی خروجی‌های چند ایجنت قابل مدیریت است.

برای مثال، در یک پروژه‌ی کوچک ساخت وب‌سایت، ممکن است یک Coding Agent بتواند کدنویسی، اصلاح خطاها و اجرای آزمون‌ها را انجام دهد. اما در پروژه‌ای بزرگ‌تر، ممکن است تقسیم کار میان ایجنت‌های متخصص در تحلیل کد، توسعه، آزمایش و بازبینی مفید باشد.

بااین‌حال، چند ایجنت همیشه به معنای عملکرد بهتر نیست. ممکن است خروجی‌های متناقض تولید کنند، وظایف تکراری انجام دهند یا برای هماهنگی به زمان و هزینه‌ی بیشتری نیاز داشته باشند. بنابراین، بهتر است ابتدا با یک ساختار ساده شروع کنیم و تنها در صورت مشاهده‌ی نیاز واقعی، تعداد ایجنت‌ها را افزایش دهیم.

معرفی و بررسی ایجنت‌های پرکاربرد

ابزارهای موجود در حوزه‌ی ایجنت‌ها از نظر هدف، شیوه‌ی استفاده و میزان استقلال با یکدیگر تفاوت دارند. بعضی برای استفاده‌ی مستقیم کاربران طراحی شده‌اند، بعضی در محیط برنامه‌نویسی فعالیت می‌کنند و برخی دیگر امکان ساخت ایجنت‌های اختصاصی و اتصال آن‌ها به سرویس‌های مختلف را فراهم می‌کنند.

در ادامه، چند نمونه از ابزارهای شناخته‌شده را بر اساس کاربرد معرفی می‌کنیم. این فهرست رتبه‌بندی قطعی یا کامل بازار نیست؛ بلکه نقطه‌ی شروعی برای شناخت گزینه‌های مختلف است. قابلیت‌ها، شرایط دسترسی، قیمت و محدودیت‌های این ابزارها ممکن است با گذشت زمان تغییر کنند.

- ChatGPT

ChatGPT یک ابزار عمومی هوش مصنوعی است که می‌تواند در توضیح مفاهیم، نگارش، تحلیل اطلاعات، برنامه‌ریزی، حل مسئله و بسیاری از وظایف روزمره کمک کند. قابلیت‌های در دسترس آن به نسخه، تنظیمات و امکانات فعال‌شده بستگی دارد

این ابزار برای شروع یادگیری، بررسی ایده‌ها و کمک‌گرفتن در مراحل اولیه‌ی پروژه مناسب است. بااین‌حال، برای انجام کارهای تخصصی که به دسترسی مستقیم به فایل‌های پروژه یا اجرای مداوم کد نیاز دارند، باید بررسی کرد که امکانات فعال آن نیاز موردنظر را پوشش می‌دهند یا خیر.

- OpenAI Codex

Codex یک ایجنت برنامه‌نویسی است که برای کمک به توسعه‌ی نرم‌افزار طراحی شده است. بسته به محیط و نسخه‌ی مورد استفاده، می‌تواند در خواندن و تغییر کد، اجرای دستورها، رفع خطاها و انجام وظایف توسعه‌ی نرم‌افزار کمک کند. ابزارهای Codex در محیط‌هایی مانند ترمینال و محیط‌های توسعه در دسترس قرار گرفته‌اند و برخی نسخه‌ها امکان مدیریت وظایف طولانی‌تر و کار با چند ایجنت را نیز فراهم می‌کنند.

این ابزار برای برنامه‌نویسان و افرادی که می‌خواهند بخشی از فرایند توسعه‌ی نرم‌افزار را به ایجنت بسپارند، گزینه‌ای قابل بررسی است. بااین‌حال، خروجی کد و تغییرات ایجادشده همچنان باید بررسی شوند.

- Claude Code

Claude Code یک ایجنت برنامه‌نویسی از شرکت Anthropic است که برای کار با پروژه‌های نرم‌افزاری طراحی شده و از طریق محیط خط فرمان قابل استفاده است. این ابزار می‌تواند با توجه به دسترسی‌ها و تنظیمات، فایل‌های پروژه را بررسی کند، کد را تغییر دهد و در انجام وظایف برنامه‌نویسی کمک کند.

این ابزار برای افرادی مناسب است که می‌خواهند از ایجنت در محیط واقعی توسعه‌ی نرم‌افزار استفاده کنند. از آنجا که کار با محیط خط فرمان و تنظیمات دسترسی ممکن است به دانش فنی نیاز داشته باشد، کاربران مبتدی باید پیش از واگذاری وظایف مهم، نحوه‌ی عملکرد آن را در یک پروژه‌ی آزمایشی یاد بگیرند.

- Cursor

Cursor یک ویرایشگر کد مبتنی بر هوش مصنوعی است که قابلیت‌های کمک به برنامه‌نویسی و کار با ایجنت‌ها را در محیط توسعه فراهم می‌کند. چنین محیطی می‌تواند برای افرادی مناسب باشد که ترجیح می‌دهند هنگام کدنویسی، بررسی و تغییر فایل‌ها را از داخل یک ویرایشگر انجام دهند.

در انتخاب این ابزار باید امکانات نسخه‌ی مورد استفاده، محدودیت‌های مصرف، سازگاری با پروژه و نحوه‌ی کنترل تغییرات را بررسی کرد. همچنین، باید توجه داشت که یک ویرایشگر مجهز به ایجنت با یک مدل هوش مصنوعی به‌تنهایی یکسان نیست؛ ویرایشگر محیط کار را فراهم می‌کند و ایجنت از مدل و ابزارهای در دسترس برای انجام وظایف استفاده می‌کند.

ابزارهای پژوهش و جست‌وجوی اطلاعات

برای پژوهش، می‌توان از ابزارهای هوش مصنوعی مجهز به جست‌وجوی وب و قابلیت بررسی منابع استفاده کرد. ابزارهایی مانند Perplexity در این دسته قابل بررسی هستند.

این ابزارها می‌توانند برای یافتن منابع اولیه، مرور سریع یک موضوع، مقایسه‌ی اطلاعات و تهیه‌ی پیش‌نویس گزارش مفید باشند. بااین‌حال، وجود ارجاع در پاسخ به‌تنهایی اعتبار ادعا را تضمین نمی‌کند. برای پژوهش علمی، باید منبع اصلی را باز کرد، اطلاعات کتاب‌شناختی را بررسی کرد و مطمئن شد که منبع واقعاً از ادعای مطرح‌شده پشتیبانی می‌کند.

- n8n

n8n یک پلتفرم خودکارسازی گردش کار است که امکان اتصال سرویس‌ها، طراحی فرایندهای خودکار و ساخت سیستم‌های مبتنی بر هوش مصنوعی را فراهم می‌کند. در

این محیط می‌توان مراحل از پیش تعریف‌شده را با قابلیت‌های تصمیم‌گیری ایجنت ترکیب کرد و در صورت نیاز، ابزارها و سرویس‌های مختلف را به هم متصل ساخت.

این ابزار برای پروژه‌هایی مانند پردازش پیام‌های ورودی، ثبت اطلاعات در جدول‌ها، ارسال اعلان‌ها و ساخت دستیارهای متصل به سرویس‌های مختلف قابل بررسی است. البته ساخت یک سیستم قابل اعتماد به تنظیم درست مراحل، مدیریت دسترسی‌ها، آزمایش و بررسی خطاها نیاز دارد.

ابزارهای ساخت ایجنت اختصاصی

علاوه بر ابزارهای آماده، پلتفرم‌ها و چارچوب‌هایی نیز وجود دارند که برای ساخت ایجنت‌های اختصاصی استفاده می‌شوند. برای مثال، OpenAI Agents SDK برای توسعه‌دهندگانی طراحی شده است که می‌خواهند با استفاده از کد، ایجنت بسازند و رفتار، ابزارها و نحوه‌ی هماهنگی آن‌ها را کنترل کنند.

این دسته از ابزارها زمانی مفید است که قابلیت‌های آماده پاسخ‌گوی نیاز پروژه نباشند و بخواهیم یک ایجنت با رفتار و امکانات مشخص طراحی کنیم. در مقابل، استفاده از آن‌ها ممکن است به دانش برنامه‌نویسی، آشنایی با API و مدیریت تنظیمات فنی نیاز داشته باشد.

چگونه از میان این ابزارها انتخاب کنیم؟

برای تصمیم‌گیری اولیه می‌توان از راهنمای زیر استفاده کرد:

- برای یادگیری و کارهای عمومی: ابتدا ابزارهای عمومی هوش مصنوعی را بررسی کنید.

- برای توسعه‌ی نرم‌افزار: ابزارهای تخصصی برنامه‌نویسی مانند Codex، Claude Code و Cursor را مقایسه کنید.

- برای پژوهش: ابزارهای جست‌وجو و تحلیل منابع را بر اساس کیفیت ارجاعات و امکان بررسی منابع بسنجید.

- برای خودکارسازی کارها: پلتفرم‌هایی مانند n8n را از نظر اتصال به سرویس‌ها، کنترل مراحل و مدیریت خطا بررسی کنید.

-برای ساخت ایجنت اختصاصی: چارچوب‌ها و ابزارهای توسعه را با توجه به دانش فنی و نیازهای پروژه انتخاب کنید.

پیش از شروع کار با هر ابزار، مستندات رسمی آن را بررسی کنید تا از امکانات فعلی، شرایط دسترسی، هزینه‌ها و محدودیت‌های آن مطلع شوید.

انتخاب ایجنت مناسب باید بر اساس نیاز واقعی پروژه انجام شود، نه صرفاً شهرت ابزار یا تعداد قابلیت‌های آن. هدف پروژه، امکانات موردنیاز، هزینه، سطح دانش فنی، امنیت و امکان بررسی خروجی از مهم‌ترین معیارهای انتخاب هستند.

بهتر است ابتدا با یک ابزار مناسب و یک وظیفه‌ی کوچک شروع کنیم، نتیجه را ارزیابی کنیم و سپس در صورت نیاز به سراغ قابلیت‌های پیشرفته‌تر یا چند ایجنت برویم. در تمام این مراحل، ایجنت وسیله‌ای برای کمک به انجام کار است و مسئولیت بررسی نتیجه و تصمیم‌گیری نهایی همچنان بر عهده‌ی انسان قرار دارد.

فصل5: شروع کار با ایجنت ها

پیش‌نیازهای شروع کار

برای شروع کار با یک ایجنت هوش مصنوعی، لازم نیست از ابتدا دانش تخصصی زیادی داشته باشیم؛ اما باید بدانیم از ایجنت چه انتظاری داریم، چه اطلاعاتی باید در اختیارش قرار دهیم و چگونه نتیجه کار آن را بررسی کنیم. پیش‌نیازهای لازم به نوع ایجنت و کاری که قرار است انجام دهد بستگی دارند.

تعیین هدف و انتخاب ابزار

پیش از انتخاب یا اجرای ایجنت، باید مشخص کنیم چه کاری می‌خواهیم انجام دهیم. برای مثال:

- اگر هدف ما یادگیری یک موضوع است، می‌توانیم از یک ابزار آموزشی یا دستیار هوش مصنوعی کمک بگیریم.

- اگر می‌خواهیم اطلاعات یک موضوع را از منابع مختلف جمع‌آوری و تحلیل کنیم، به ابزاری با قابلیت پژوهش و دسترسی به منابع نیاز داریم.

- اگر قصد داریم یک نرم‌افزار بسازیم یا کدهای موجود را اصلاح کنیم، باید از یک Coding Agent متناسب با محیط برنامه‌نویسی خود استفاده کنیم.

- اگر هدف ما خودکارسازی کارهای تکراری است، باید ابزاری انتخاب کنیم که بتواند به سرویس‌ها و ابزارهای موردنیاز متصل شود.

انتخاب ابزار باید بر اساس نیاز واقعی پروژه انجام شود، نه صرفاً شهرت یا تعداد قابلیت‌های آن.

آماده‌سازی حساب کاربری و محیط اجرا

بسته به ابزار انتخاب‌شده، ممکن است لازم باشد حساب کاربری بسازیم، وارد حساب شویم، نرم‌افزار را نصب کنیم یا تنظیمات اولیه را انجام دهیم.

برای مثال، Codex روش‌های مختلفی برای شروع کار دارد و می‌توان از محیط‌های پشتیبانی‌شده آن استفاده کرد. در مقابل، ابزارهایی مانند Claude Code می‌توانند در محیط توسعه و ترمینال مورد استفاده قرار بگیرند. بنابراین، مراحل راه‌اندازی در همه ایجنت‌ها یکسان نیست.

در ایجنت‌های برنامه‌نویسی، آماده‌سازی محیط ممکن است شامل موارد زیر باشد:

- نصب ابزارها و وابستگی‌های لازم
- انتخاب پوشه یا مخزن پروژه
- آماده‌سازی محیط برنامه‌نویسی
- بررسی نسخه‌های موردنیاز زبان‌ها و کتابخانه‌ها
- تنظیم دسترسی‌های لازم برای خواندن فایل‌ها، ویرایش کد یا اجرای دستورها

برای انجام این مراحل، باید به مستندات رسمی همان ابزار مراجعه کنیم؛ زیرا روش نصب و تنظیمات آن ممکن است با سایر ایجنت‌ها متفاوت باشد.

آشنایی اولیه با پروژه

اگر قرار است ایجنت روی یک پروژه موجود کار کند، بهتر است پیش از شروع، ساختار کلی پروژه، هدف آن و وضعیت فعلی فایل‌ها را بشناسیم.

برای مثال، در یک پروژه پایتون باید بدانیم برنامه چه کاری انجام می‌دهد، فایل اصلی کدام است، کتابخانه‌های موردنیاز کدام‌اند و چگونه می‌توان برنامه را اجرا کرد.

هرچه شناخت ما از پروژه بیشتر باشد، بهتر می‌توانیم درخواست‌های دقیق‌تری ارائه دهیم و خروجی ایجنت را ارزیابی کنیم. البته در بسیاری از ایجنت‌های برنامه‌نویسی می‌توان از خود ابزار نیز خواست که ابتدا ساختار پروژه را بررسی و خلاصه‌ای از آن ارائه کند.

آشنایی با دسترسی‌ها و محدودیت‌ها

بعضی ایجنت‌ها فقط پاسخ تولید می‌کنند؛ اما برخی دیگر می‌توانند فایل بخوانند، فایل جدید بسازند، کد را تغییر دهند، دستور اجرا کنند یا به ابزارهای خارجی متصل شوند.

پیش از استفاده باید بدانیم ایجنت به چه منابعی دسترسی دارد و برای انجام چه اقداماتی به تأیید ما نیاز دارد. دسترسی بیشتر لزوماً به معنای انتخاب بهتر نیست؛ دسترسی باید با نیاز پروژه متناسب باشد.

همچنین نباید اطلاعات محرمانه، رمزهای عبور، کلیدهای دسترسی یا داده‌های حساس را بدون بررسی سیاست‌های امنیتی و حریم خصوصی ابزار در اختیار آن قرار دهیم.

آشنایی با محیط و تنظیمات ایجنت

پس از انتخاب و راه‌اندازی ایجنت، باید با محیط کاری و تنظیمات اصلی آن آشنا شویم. ظاهر و امکانات محیط‌ها متفاوت است، اما معمولاً چند مفهوم در بسیاری از ابزارها مشترک هستند.

محیط گفتگو یا ورود درخواست

این بخش جایی است که کاربر درخواست خود را وارد می‌کند، اطلاعات تکمیلی می‌دهد و نتیجه کار را دریافت می‌کند.

در ابزارهای ساده‌تر، کاربر معمولاً با نوشتن متن با ایجنت تعامل می‌کند. در ایجنت‌های برنامه‌نویسی، علاوه بر نوشتن درخواست، ممکن است بتوان پوشه پروژه را انتخاب کرد، فایل‌ها را مشخص کرد یا نتیجه اجرای دستورها را مشاهده کرد.

فضای کاری پروژه

فضای کاری یا Workspace محیطی است که ایجنت در آن وظیفه خود را انجام می‌دهد. در ایجنت‌های برنامه‌نویسی، این فضا معمولاً شامل فایل‌های کد، پوشه‌ها، تنظیمات و سایر اجزای پروژه است.

برای مثال، هنگامی که یک Coding Agent به پوشه مشخصی دسترسی دارد، ممکن است بتواند فایل‌های آن پوشه را بررسی کند یا تغییراتی در آن‌ها انجام دهد. بااین‌حال، دامنه دسترسی دقیق به ابزار و تنظیمات آن بستگی دارد.

پیش از شروع کار، باید مطمئن شویم پوشه یا پروژه صحیح انتخاب شده است؛ زیرا اجرای درخواست در محیط اشتباه می‌تواند به تغییر فایل‌های نامرتبط منجر شود.

تنظیمات مدل و نحوه اجرا

برخی ابزارها اجازه می‌دهند مدل هوش مصنوعی یا نحوه اجرای وظیفه را انتخاب کنیم. در بعضی محیط‌ها نیز تنظیماتی برای میزان دسترسی، درخواست تأیید پیش از اجرای عملیات یا شیوه ادامه کار وجود دارد.

این تنظیمات در همه ابزارها یکسان نیستند. برای مثال، در بعضی ایجنت‌های برنامه‌نویسی می‌توان حالت برنامه‌ریزی یا بررسی را از حالت اجرای تغییرات جدا کرد. در حالت برنامه‌ریزی، ابزار ابتدا راهکار پیشنهادی را ارائه می‌کند و در صورت تأیید، کار وارد مرحله اجرا می‌شود.

برای پروژه‌های جدید یا حساس، بهتر است ابتدا از حالتی استفاده کنیم که امکان بررسی برنامه پیشنهادی و تأیید اقدامات را فراهم می‌کند.

بررسی نتیجه و تغییرات

در ایجنت‌هایی که روی فایل‌ها یا پروژه‌ها کار می‌کنند، باید بدانیم نتیجه فعالیت را از کجا مشاهده کنیم. این نتیجه ممکن است شامل متن تولیدشده، فایل‌های جدید، تغییرات کد، گزارش آزمایش‌ها یا پیام‌های خطا باشد.

صرف مشاهده پیام «انجام شد» برای اطمینان از موفقیت کافی نیست. باید بررسی کنیم که آیا خروجی واقعاً نیاز ما را برآورده می‌کند و آیا تغییرات ایجادشده با هدف پروژه سازگار هستند یا خیر.

نحوه تعریف وظیفه و نوشتن درخواست مناسب

یکی از مهم‌ترین مهارت‌ها در کار با ایجنت‌ها، توانایی تعریف دقیق وظیفه است. ایجنت برای رسیدن به نتیجه مناسب، باید بداند چه کاری از آن انتظار داریم، چه اطلاعاتی در اختیار دارد و نتیجه مطلوب چه ویژگی‌هایی باید داشته باشد.

اجزای یک درخواست مناسب

یک درخواست مناسب می‌تواند شامل اجزای زیر باشد:

هدف: مشخص می‌کند چه نتیجه‌ای می‌خواهیم.

زمینه یا توضیح مسئله: اطلاعاتی درباره پروژه، شرایط فعلی یا علت انجام کار ارائه می‌دهد.

وظایف: مشخص می‌کند ایجنت باید چه اقداماتی انجام دهد.

محدودیت‌ها: تعیین می‌کند چه کارهایی نباید انجام شوند یا چه شرایطی باید رعایت شوند.

قالب خروجی: مشخص می‌کند نتیجه باید به چه شکلی ارائه شود.

معیار موفقیت: توضیح می‌دهد از کجا متوجه می‌شویم کار به‌درستی انجام شده است.

لازم نیست هر درخواست کوتاهی تمام این اجزا را داشته باشد؛ اما هرچه وظیفه پیچیده‌تر و حساس‌تر باشد، مشخص‌کردن این موارد اهمیت بیشتری پیدا می‌کند.

مقایسه درخواست مبهم و درخواست دقیق

فرض کنیم می‌خواهیم ایجنت به ما در ساخت یک برنامه ساده کمک کند.

درخواست مبهم:

{ برای من یک برنامه مدیریت کارها بساز}

این درخواست هدف کلی را بیان می‌کند، اما جزئیات مهمی مانند امکانات برنامه، نوع خروجی و محدودیت‌ها مشخص نیستند. در نتیجه، ایجنت ممکن است بر اساس برداشت خودش تصمیم‌هایی بگیرد که با انتظار ما هماهنگ نباشند.

درخواست دقیق‌تر:

{ یک برنامه ساده با پایتون برای مدیریت کارهای روزانه طراحی کن. کاربر باید بتواند کار جدید اضافه کند، کارهای ثبت‌شده را مشاهده کند و وضعیت هر کار را به انجام‌شده تغییر دهد. فعلاً از رابط گرافیکی استفاده نکن و برنامه را در محیط ترمینال اجراپذیر نگه دار. ابتدا ساختار پیشنهادی را توضیح بده و پس از تأیید من، کد را پیاده‌سازی کن. در پایان، روش اجرا و چند آزمایش برای بررسی عملکرد برنامه را ارائه بده }.

در درخواست دوم، هدف، امکانات، محدودیت‌ها، ترتیب اجرا و معیارهای بررسی روشن‌تر هستند.

درخواست را به وظایف کوچک تقسیم کنید

برای پروژه‌های پیچیده، بهتر است تمام کارها را در یک درخواست بزرگ جمع نکنیم. تقسیم پروژه به مراحل کوچک‌تر به ما کمک می‌کند که نتیجه هر مرحله را بررسی کنیم و پیش از ادامه، خطاها یا برداشت‌های اشتباه را اصلاح کنیم.

برای مثال، ساخت یک برنامه را می‌توان به مراحل زیر تقسیم کرد:

۱. بررسی مسئله و مشخص‌کردن نیازمندی‌ها

۲. پیشنهاد ساختار برنامه

۳. پیاده‌سازی قابلیت اصلی

۴. اجرای آزمایش‌ها

۵. رفع خطاها

۶. بررسی نهایی و مستندسازی

در هر مرحله باید وظیفه موردانتظار را مشخص کنیم و پس از مشاهده نتیجه، درباره ادامه کار تصمیم بگیریم.

تعیین محدوده تغییرات

در درخواست‌های مربوط به پروژه‌های موجود، مشخص‌کردن محدوده تغییرات اهمیت زیادی دارد.

برای مثال، به‌جای اینکه بگوییم «کد را بهتر کن»، می‌توانیم بنویسیم:

{ تابع محاسبه میانگین را بررسی کن و خطاهای احتمالی آن را مشخص کن. فقط همین تابع و بخش‌های مستقیماً مرتبط را تغییر بده. سایر قسمت‌های برنامه را بدون هماهنگی تغییر نده. پیش از اصلاح، توضیح بده چه مشکلی پیدا کرده‌ای }.

چنین درخواستی محدوده کار را روشن می‌کند و احتمال تغییرات غیرضروری را کاهش می‌دهد.

درخواست را در صورت نیاز اصلاح کنید

تعامل با ایجنت معمولاً با یک درخواست پایان نمی‌یابد. ممکن است پس از مشاهده نتیجه، متوجه شویم بخشی از درخواست درست درک نشده یا خروجی به اصلاح نیاز دارد.

در این شرایط، به‌جای تکرار کامل درخواست، می‌توانیم مشخص کنیم چه چیزی باید تغییر کند و چه بخش‌هایی باید ثابت بمانند.

برای مثال:

{ ساختار کلی مناسب است، اما در نسخه فعلی امکان حذف کارها وجود ندارد. این قابلیت را اضافه کن و سایر امکانات را تغییر نده. سپس آزمایش‌های مربوط به افزودن، مشاهده، تکمیل و حذف کارها را اجرا کن }.

این روش به هدایت تدریجی ایجنت کمک می‌کند؛ بااین‌حال، نتیجه هر اصلاح همچنان باید بررسی شود.

ارائه اطلاعات و Context موردنیاز

Context یا زمینه، به اطلاعاتی گفته می‌شود که ایجنت برای درک درست درخواست و انجام وظیفه در اختیار دارد. این اطلاعات می‌توانند شامل توضیحات کاربر، فایل‌ها، سابقه گفتگو، ساختار پروژه، دستورالعمل‌ها و نتایج مراحل قبلی باشند.

اگر درخواست به‌تنهایی اطلاعات کافی نداشته باشد، ایجنت ممکن است برای تکمیل بخش‌های نامشخص از فرضیات خود استفاده کند. بنابراین، مدیریت Context بخش مهمی از کار با ایجنت‌هاست.

چه اطلاعاتی باید ارائه کنیم؟

بسته به نوع پروژه، اطلاعات موردنیاز متفاوت است. موارد زیر از نمونه‌های رایج هستند:

-هدف پروژه: پروژه برای حل چه مسئله‌ای ساخته می‌شود؟

- وضعیت فعلی: تاکنون چه کارهایی انجام شده است؟

- فایل‌ها و منابع: کدام فایل‌ها، اسناد یا داده‌ها برای انجام وظیفه لازم‌اند؟

- تصمیم‌های قبلی : چه انتخاب‌هایی انجام شده و چه مواردی نباید تغییر کنند؟

- محدودیت‌ها: چه فناوری‌ها، روش‌ها یا منابعی باید استفاده شوند یا کنار گذاشته شوند؟

- معیارهای ارزیابی: نتیجه باید چه ویژگی‌هایی داشته باشد؟

لازم نیست تمام اطلاعات پروژه را در هر درخواست تکرار کنیم. بهتر است اطلاعات ضروری و مرتبط را در دسترس ایجنت قرار دهیم و موارد نامرتبط را وارد Context نکنیم.

ارائه فایل‌ها و منابع مرتبط

در بسیاری از ابزارها می‌توان فایل‌ها را بارگذاری کرد، مسیر آن‌ها را مشخص کرد یا اجازه دسترسی به پوشه پروژه را داد.

برای مثال، اگر از ایجنت بخواهیم کد یک تابع را اصلاح کند، ارائه همان فایل و فایل‌های مرتبط معمولاً مفیدتر از ارائه مجموعه بزرگی از فایل‌های نامرتبط است.

در پروژه‌های پژوهشی نیز باید مشخص کنیم که ایجنت از کدام منابع استفاده کند و آیا لازم است اطلاعات را از منابع رسمی و معتبر استخراج کند. اگر ارائه منبع یا ارجاع ضروری است، این شرط باید صریحاً در درخواست ذکر شود.

حفظ اطلاعات مهم بین مراحل

در پروژه‌های چندمرحله‌ای، ممکن است لازم باشد تصمیم‌ها، وضعیت پیشرفت و وظایف باقی‌مانده ثبت شوند تا در مراحل بعدی دوباره مورد استفاده قرار گیرند.

برای این کار می‌توان از مستندات پروژه، فایل‌های یادداشت، گزارش پیشرفت یا قابلیت‌های حافظه خود ابزار استفاده کرد. برخی Coding Agentها نیز از فایل‌های دستورالعمل پروژه پشتیبانی می‌کنند؛ برای نمونه، در Claude Code می‌توان از فایل `CLAUDE.md` برای ثبت دستورالعمل‌ها و اطلاعات مرتبط با پروژه استفاده کرد.

این قابلیت‌ها در ابزارهای مختلف متفاوت‌اند و باید مطابق مستندات رسمی هر ابزار تنظیم شوند.

مراقبت از Context بیش‌ازحد یا ناقص

ارائه اطلاعات بیشتر همیشه به معنای نتیجه بهتر نیست. اگر اطلاعات غیرضروری فراوانی ارائه کنیم، ممکن است نکات اصلی در میان جزئیات گم شوند. از طرف دیگر، حذف اطلاعات ضروری نیز می‌تواند باعث برداشت اشتباه شود.

بنابراین، بهتر است Context را به‌صورت هدفمند آماده کنیم: اطلاعات لازم را ارائه دهیم، تصمیم‌های مهم را ثبت کنیم و در صورت تغییر شرایط پروژه، اطلاعات قبلی را به‌روزرسانی کنیم.

آشنایی با دستورها و قابلیت‌های ایجنت

دستورها و قابلیت‌ها به ما کمک می‌کنند شیوه تعامل با ایجنت را متناسب با نوع کار تنظیم کنیم. در اینجا باید میان دو مفهوم تفاوت قائل شویم:

درخواست یا Prompt: متنی است که کاربر برای توضیح هدف، وظیفه و شرایط موردنظر به ایجنت ارائه می‌کند.

دستور یا Command: در بعضی ابزارها، دستور شکل مشخص و ازپیش‌تعریف‌شده‌ای دارد و برای اجرای یک قابلیت خاص استفاده می‌شود.

برای مثال، نوشتن «این تابع را بررسی کن و خطاهای آن را توضیح بده» یک درخواست متنی است؛ اما در ابزارهایی که از دستورهای اختصاصی پشتیبانی می‌کنند، ممکن است فرمان مشخصی برای مشاهده وضعیت، بررسی تنظیمات یا مدیریت جلسه وجود داشته باشد.

قابلیت‌های رایج در ایجنت‌ها

بسته به ابزار، ممکن است با قابلیت‌های زیر روبه‌رو شویم:

- خواندن و تحلیل فایل‌ها
- جست‌وجو در اطلاعات یا منابع
- ایجاد و ویرایش فایل
- نوشتن یا اصلاح کد
- اجرای دستورها در ترمینال
- اجرای آزمایش‌ها و بررسی خطاها
- استفاده از ابزارها و سرویس‌های خارجی
- ثبت یا بازیابی اطلاعات مربوط به پروژه
- تقسیم وظایف بین چند ایجنت یا عامل فرعی

نباید فرض کنیم همه ایجنت‌ها تمام این قابلیت‌ها را دارند. برخی ابزارها تنها در محیط گفتگو فعالیت می‌کنند، درحالی‌که بعضی دیگر به فایل‌ها و ابزارهای اجرایی دسترسی دارند. حتی در یک ابزار واحد نیز قابلیت‌های در دسترس می‌توانند به تنظیمات، نسخه، مجوزها و محیط اجرا بستگی داشته باشند.

استفاده از قابلیت مناسب برای هر مرحله

بهتر است برای هر مرحله از پروژه، قابلیتی را انتخاب کنیم که مستقیماً به انجام همان وظیفه کمک می‌کند.

برای مثال، در یک پروژه برنامه‌نویسی:

- برای شناخت پروژه، از قابلیت خواندن و تحلیل فایل‌ها استفاده می‌کنیم.
- برای مشخص‌کردن راهکار، از ایجنت می‌خواهیم برنامه پیشنهادی ارائه کند.
- برای پیاده‌سازی، اجازه ویرایش فایل‌های مرتبط را می‌دهیم.
- برای بررسی عملکرد، آزمایش‌های مناسب را اجرا می‌کنیم.
- برای ارزیابی نهایی، تغییرات و نتایج آزمایش‌ها را بررسی می‌کنیم.

به این ترتیب، به‌جای واگذاری مبهم تمام کارها، روند اجرا قابل‌پیگیری‌تر می‌شود.

دستورهای عمومی و دستورهای اختصاصی هر ابزار

دستورهای ایجنت‌ها را می‌توان به‌طور کلی در دو گروه بررسی کرد: درخواست‌های عمومی که در ابزارهای مختلف قابل استفاده‌اند و دستورهای اختصاصی که به ساختار یک ابزار مشخص وابسته هستند.

درخواست‌های عمومی

دستورهای عمومی، درخواست‌هایی هستند که کاربر با زبان طبیعی می‌نویسد تا هدف، وظیفه یا نتیجه موردانتظار خود را برای ایجنت مشخص کند. این درخواست‌ها به ابزار خاصی محدود نیستند؛ البته اجرای آن‌ها به قابلیت‌ها و دسترسی‌های ایجنت بستگی دارد.

۱. بررسی ساختار پروژه (Explore Project)

نمونه دستور:

{ ساختار پروژه را بررسی کن، فایل‌ها و پوشه‌های مهم را معرفی کن و توضیح بده هرکدام چه نقشی دارند. فعلاً هیچ فایلی را تغییر نده }.

کاربرد: برای آشنایی اولیه با یک پروژه موجود، پیش از شروع توسعه یا اصلاح کد.

۲. پیدا کردن اطلاعات موردنیاز (Search for Information)

نمونه دستور:

{ در فایل‌های پروژه جست‌وجو کن و تمام بخش‌هایی را که به احراز هویت کاربران مربوط هستند پیدا کن. مسیر فایل‌ها و نقش هر بخش را توضیح بده }.

کاربرد: برای پیدا کردن فایل‌ها، توابع، تنظیمات یا اطلاعات مشخص در یک پروژه؛ به شرط آنکه ایجنت به فایل‌های موردنظر دسترسی داشته باشد.

۳. توضیح خطای برنامه (Explain an Error)

نمونه دستور:

{ این پیام خطا را به زبان ساده توضیح بده، علت‌های احتمالی آن را مشخص کن و بگو برای تشخیص علت اصلی به چه اطلاعاتی نیاز داریم }.

کاربرد: برای فهمیدن خطاهای برنامه‌نویسی و یادگیری روش تشخیص مشکل.

۴. پیشنهاد چند راهکار ((Suggest Alternatives

نمونه دستور:

{ برای پیاده‌سازی این قابلیت سه روش پیشنهاد بده. مزایا، معایب، پیچیدگی و پیش‌نیازهای هر روش را مقایسه کن. فعلاً هیچ‌کدام را اجرا نکن }.

کاربرد: برای بررسی گزینه‌های مختلف پیش از انتخاب راهکار نهایی.

۵. ایجاد فایل جدید (Create a File)

نمونه دستور:

{ در پوشه اصلی پروژه یک فایل README.md ایجاد کن و هدف پروژه، پیش‌نیازها، روش نصب و نحوه اجرا را در آن بنویس. ابتدا بررسی کن که آیا این فایل از قبل وجود دارد یا خیر }.

کاربرد: برای ایجاد فایل‌های کد، مستندات، تنظیمات یا سایر فایل‌های موردنیاز؛ در صورتی که ایجنت اجازه ایجاد فایل داشته باشد.

۶. بررسی وابستگی‌های پروژه (Check Dependencies)

نمونه دستور:

{ وابستگی‌های پروژه را بررسی کن، کتابخانه‌های مورد استفاده و نقش هرکدام را توضیح بده و مشخص کن آیا موردی وجود دارد که برای اجرای برنامه ضروری باشد اما نصب نشده باشد. پیش از نصب یا حذف هر کتابخانه از من تأیید بگیر}.

کاربرد: برای شناخت کتابخانه‌ها و ابزارهای موردنیاز و کاهش مشکلات ناشی از تنظیمات ناقص محیط.

۷. بهبود خوانایی کد (Improve Readability)

نمونه دستور:

{ خوانایی این کد را بررسی کن. نام‌گذاری متغیرها، ساختار توابع و بخش‌های پیچیده را مشخص کن و پیشنهادهای اصلاحی بده. عملکرد برنامه را تغییر نده }.

کاربرد: برای قابل‌فهم‌ترکردن کد و آسان‌ترکردن نگهداری آن.

۸. تهیه گزارش پیشرفت (Progress Report)

نمونه دستور:

{ بر اساس کارهای انجام‌شده، گزارشی شامل وظایف تکمیل‌شده، وظایف باقی‌مانده، مشکلات فعلی و مراحل پیشنهادی بعدی تهیه کن. مواردی را که هنوز بررسی نشده‌اند، به‌عنوان تکمیل‌شده معرفی نکن }.

کاربرد: برای پیگیری وضعیت پروژه و مشخص‌کردن ادامه مسیر.

۹. بررسی امنیت اولیه (Security Review)

نمونه دستور:

{ این بخش از پروژه را از نظر مشکلات امنیتی رایج بررسی کن. موارد مشکوک را همراه با محل وقوع، خطر احتمالی و راهکار پیشنهادی گزارش بده. فعلاً هیچ تغییری اعمال نکن }.

کاربرد: برای شناسایی اولیه مشکلات امنیتی احتمالی. این بررسی جایگزین ارزیابی امنیتی تخصصی نیست.

۱۰. تهیه چک‌لیست (Create a Checklist)

نمونه دستور:

{ برای آماده‌سازی این پروژه جهت تحویل، یک چک‌لیست شامل آزمایش‌ها، بررسی کد، مستندات، تنظیمات و نکات امنیتی تهیه کن }.

کاربرد: برای اطمینان از اینکه مراحل مهم پیش از تحویل یا انتشار پروژه فراموش نمی‌شوند.

۱۱. ادامه کار از مرحله قبلی (Continue Previous Work)

نمونه دستور:

{ با توجه به مستندات پروژه و وضعیت فعلی فایل‌ها، مشخص کن آخرین مرحله تکمیل‌شده چیست و چه وظایفی باقی مانده‌اند. ابتدا وضعیت را گزارش بده و پس از تأیید من، کار بعدی را شروع کن }.

کاربرد: برای ادامه پروژه‌های چندمرحله‌ای، به‌ویژه زمانی که کار در چند جلسه انجام می‌شود. این درخواست زمانی دقیق‌تر عمل می‌کند که اطلاعات مراحل قبلی در دسترس ایجنت باشد.

۱۲. بررسی تأثیر تغییرات (Assess Change Impact)

نمونه دستور:

«پیش از تغییر این تابع، بخش‌هایی از پروژه را که ممکن است تحت تأثیر قرار بگیرند شناسایی کن. وابستگی‌های مرتبط و آزمایش‌هایی را که باید پس از تغییر اجرا شوند، فهرست کن.»

کاربرد: برای کاهش خطر ایجاد خطا در بخش‌های دیگر پروژه هنگام اصلاح کد

نکته: نمونه‌های بالا الگوهای پیشنهادی برای نوشتن درخواست هستند، نه دستورهای اختصاصی یک ابزار. می‌توان آن‌ها را با توجه به هدف پروژه تغییر داد. همچنین درخواست‌کردن یک عملیات به معنای تضمین اجرای موفق آن نیست؛ ایجنت باید قابلیت و دسترسی لازم را داشته باشد و نتیجه کار نیز بررسی شود.

مزیت این روش آن است که برای استفاده از آن لزوماً به حفظ‌کردن دستورهای فنی نیاز نداریم. بااین‌حال، کیفیت نتیجه به وضوح درخواست، اطلاعات موجود و قابلیت‌های واقعی ابزار وابسته است.

دستورهای اختصاصی

برخی ابزارها علاوه بر درخواست‌های متنی، فرمان‌های مشخصی دارند که برای انجام عملیات خاص طراحی شده‌اند. این فرمان‌ها ممکن است با علامت `/`، گزینه‌های خط فرمان یا شیوه‌های دیگری فراخوانی شوند.

برای نمونه، در Claude Code دستور `/init` برای ایجاد فایل `CLAUDE.md` و آماده‌سازی دستورالعمل‌های مرتبط با پروژه استفاده می‌شود. همچنین دستورهایی برای مدیریت یا ادامه جلسه و بررسی مجوزها وجود دارند.

در ابزارهای دیگر، نام، ساختار و عملکرد فرمان‌ها ممکن است کاملاً متفاوت باشد. بنابراین، نباید دستور اختصاصی یک ابزار را بدون بررسی در ابزار دیگری اجرا کنیم.

اسلش (Slash) و کاربرد آن در دستورهای ایجنت‌ها

علامت `/` که اسلش (Slash) نام دارد، در برخی ابزارهای هوش مصنوعی برای شروع دستورهای اختصاصی استفاده می‌شود. این علامت به ابزار کمک می‌کند تشخیص دهد که عبارت واردشده یک دستور مشخص است، نه یک درخواست معمولی به زبان طبیعی.

برای مثال، در ابزار Claude Code دستورهای زیر وجود دارند:

- `/help` برای مشاهده راهنما و دستورهای قابل استفاده

- `/clear` برای پاک‌کردن زمینه گفتگوی فعلی

- `/compact` برای فشرده‌سازی زمینه گفتگو و خلاصه‌ترکردن اطلاعات آن

در بعضی ابزارها، با تایپ علامت `/` می‌توان فهرستی از دستورهای موجود را مشاهده کرد.

نکته: اسلش به‌تنهایی دستور نیست، بلکه در ابزارهایی که از این شیوه پشتیبانی می‌کنند، برای شناسایی دستورهای اختصاصی به کار می‌رود. همه ایجنت‌ها از این روش استفاده نمی‌کنند و کاربرد هر دستور به ابزار موردنظر بستگی دارد.

چگونه دستورهای رسمی هر ابزار را پیدا کنیم؟

برای شناخت دستورهای یک ایجنت، بهتر است این مراحل را دنبال کنیم:

۱. وارد مستندات رسمی ابزار شویم.

۲. بخش‌های مربوط به شروع کار، راهنمای کاربر، مرجع دستورها یا تنظیمات را پیدا کنیم.

۳. نام و کاربرد هر دستور را از توضیحات رسمی بررسی کنیم.

۴. مشخص کنیم دستور چه ورودی‌هایی می‌پذیرد و چه نتیجه‌ای ایجاد می‌کند.

۵. ابتدا آن را در محیطی کم‌خطر و روی یک پروژه آزمایشی امتحان کنیم.

۶. پیش از اجرای دستورهایی که فایل‌ها را تغییر می‌دهند، دستورهای سیستمی اجرا می‌کنند یا به منابع حساس دسترسی دارند، اثر احتمالی آن‌ها را بررسی کنیم.

به خاطر داشته باشیم که مستندات رسمی هر ابزار، مرجع اصلی برای اطلاع از نام دقیق دستورها، کاربرد آن‌ها و محدودیت‌هایشان است.

بررسی خروجی، اصلاح درخواست و ادامه کار

آخرین مرحله در شروع کار با ایجنت‌ها، یادگیری نحوه ارزیابی نتیجه است. ایجنت ممکن است پاسخی ارائه کند که از نظر ظاهری کامل به نظر برسد، اما هنوز بخشی از نیازمندی‌ها را برآورده نکند یا دارای خطا باشد. بنابراین، نتیجه باید بر اساس معیارهای مشخص بررسی شود.

مقایسه نتیجه با هدف اولیه

پس از دریافت خروجی، باید آن را با درخواست اولیه مقایسه کنیم و به این پرسش‌ها پاسخ دهیم:

- آیا هدف اصلی محقق شده است؟
- آیا تمام نیازمندی‌های ضروری رعایت شده‌اند؟
- آیا محدودیت‌های تعیین‌شده حفظ شده‌اند؟
- آیا اطلاعات یا فایل‌های موردنیاز ایجاد شده‌اند؟
- آیا تغییری خارج از محدوده درخواست رخ داده است؟
- آیا نتیجه با معیارهای موفقیت سازگار است؟

اگر پاسخ برخی پرسش‌ها منفی باشد، باید مشخص کنیم چه بخشی نیازمند اصلاح است.

بررسی و آزمایش خروجی

روش بررسی به نوع وظیفه بستگی دارد. برای یک متن پژوهشی، باید صحت اطلاعات، اعتبار منابع و درستی ارجاعات بررسی شوند. برای یک برنامه نرم‌افزاری، لازم است کد، عملکرد برنامه و نتایج آزمایش‌ها ارزیابی شوند.

در پروژه‌های برنامه‌نویسی، بررسی می‌تواند شامل اجرای برنامه، اجرای تست‌ها، بررسی پیام‌های خطا و مشاهده تغییرات فایل‌ها باشد. البته موفقیت در یک آزمایش محدود، تضمین نمی‌کند که برنامه در تمام شرایط بدون خطا کار خواهد کرد.

ارائه بازخورد دقیق

اگر نتیجه مناسب نبود، بهتر است به‌جای گفتن «درستش کن»، مشکل را مشخص کنیم.

برای مثال:

{ در نسخه فعلی، برنامه هنگام واردکردن فهرست خالی با خطا مواجه می‌شود. این حالت را بررسی و اصلاح کن. سپس برنامه را با سه ورودی متفاوت آزمایش کن و نتیجه هر آزمایش را گزارش بده. سایر بخش‌ها را تغییر نده، مگر اینکه برای رفع این خطا ضروری باشند }.

این درخواست مشخص می‌کند مشکل چیست، چه تغییری انتظار می‌رود و چگونه باید نتیجه بررسی شود.

ثبت تغییرات و حفظ نسخه قابل‌بازگشت

هنگامی که ایجنت روی فایل‌ها یا کدها تغییر ایجاد می‌کند، بهتر است بتوانیم تغییرات را بررسی و در صورت نیاز به نسخه قبلی بازگردیم. استفاده از ابزارهای کنترل نسخه مانند Git می‌تواند در پروژه‌های نرم‌افزاری به پیگیری تغییرات کمک کند.

پیش از تأیید تغییرات مهم، باید بدانیم چه فایل‌هایی تغییر کرده‌اند و آیا این تغییرات با هدف پروژه سازگارند. همچنین بهتر است پیش از عملیات پرخطر، نسخه پشتیبان یا امکان بازگشت مناسب داشته باشیم.

تصمیم‌گیری درباره ادامه کار

پس از بررسی نتیجه، معمولاً سه انتخاب داریم:

۱. پذیرش نتیجه: اگر خروجی نیازمندی‌ها را برآورده کرده و بررسی‌های لازم را گذرانده باشد، می‌توانیم به مرحله بعد برویم.

2. درخواست اصلاح: اگر بخشی از خروجی نادرست یا ناقص باشد، مشکل را دقیق توضیح می‌دهیم و اصلاح آن را درخواست می‌کنیم.

3. بازنگری در روش: اگر راهکار پیشنهادی با مسئله تناسب نداشته باشد، باید هدف، محدودیت‌ها یا برنامه اجرا را دوباره بررسی کنیم.

این فرایند نشان می‌دهد که کار با ایجنت تنها به نوشتن درخواست و دریافت پاسخ محدود نیست؛ بلکه شامل هدایت، ارزیابی و تصمیم‌گیری نیز می‌شود.

برای شروع کار با ایجنت‌ها، ابتدا باید ابزار متناسب با هدف خود را انتخاب کنیم و محیط لازم را آماده سازیم. سپس با محیط کار، تنظیمات و دسترسی‌های آن آشنا

شویم، وظیفه را به‌صورت روشن تعریف کنیم و اطلاعات موردنیاز را در اختیار ایجنت قرار دهیم.

اصل مهم: برای کار مؤثر با ایجنت، فقط دانستن اینکه چه درخواستی بنویسیم کافی نیست؛ باید بدانیم چه اطلاعاتی ارائه کنیم، چه دسترسی‌هایی بدهیم، نتیجه را چگونه بسنجیم و چه زمانی لازم است مسیر کار را تغییر دهیم.

فصل6: ساخت پروژه با کمک ایجنت ها

یکی از کاربردهای مهم ایجنت‌های هوش مصنوعی، کمک به ساخت و توسعه پروژه‌هاست. ایجنت می‌تواند در مراحل مختلف یک پروژه، از بررسی ایده اولیه و طراحی ساختار تا نوشتن کد، آزمایش، رفع خطا و تهیه مستندات به ما کمک کند.

بااین‌حال، ساخت یک پروژه موفق صرفاً به نوشتن چند درخواست و دریافت کد از ایجنت محدود نمی‌شود. برای رسیدن به نتیجه مطلوب، باید مسئله را درست تعریف کنیم، نیازمندی‌ها را مشخص سازیم، مراحل اجرا را برنامه‌ریزی کنیم و خروجی هر مرحله را مورد بررسی قرار دهیم.

انتخاب مسئله و تعریف هدف پروژه

اولین مرحله در ساخت هر پروژه، مشخص‌کردن مسئله‌ای است که می‌خواهیم حل کنیم. پیش از آنکه از ایجنت بخواهیم چیزی بسازد، باید بدانیم چرا به آن نیاز داریم، چه افرادی قرار است از نتیجه استفاده کنند و چه مشکلی باید برطرف شود.

مسئله پروژه چیست؟

مسئله پروژه، نیاز یا مشکلی است که قرار است با کمک پروژه برطرف شود. برای مثال، ممکن است دانشجویی برای مدیریت برنامه مطالعه خود مشکل داشته باشد، یک پژوهشگر به ابزاری برای مرتب‌سازی اطلاعات نیاز داشته باشد یا یک مرکز درمانی بخواهد فرایندی تکراری را خودکار کند.

هر ایده‌ای لزوماً یک مسئله مشخص و قابل‌حل نیست. برای تبدیل ایده به پروژه، باید بتوانیم نیاز اصلی را به‌صورت روشن توضیح دهیم.

برای مثال:

- ایده کلی: ساخت یک اپلیکیشن سلامت

- مسئله مشخص: افراد هنگام ثبت روزانه فعالیت بدنی خود، اطلاعات را در چند محل مختلف یادداشت می‌کنند و به‌سختی می‌توانند روند فعالیتشان را مشاهده کنند.

- هدف اولیه: ساخت ابزاری ساده برای ثبت فعالیت روزانه و نمایش خلاصه هفتگی آن.

در مثال دوم، مسئله و نتیجه موردانتظار روشن‌تر هستند و می‌توان بر اساس آن‌ها مراحل بعدی را طراحی کرد.

هدف پروژه را مشخص کنید

هدف پروژه باید بیان کند که در پایان کار چه نتیجه‌ای انتظار داریم. بهتر است هدف تا حد امکان مشخص، قابل‌فهم و قابل‌ارزیابی باشد.

برای مثال، به‌جای اینکه بگوییم «می‌خواهم یک برنامه خوب بسازم»، می‌توانیم بگوییم:

{ می‌خواهم یک برنامه تحت وب بسازم که کاربران بتوانند فعالیت روزانه خود را ثبت کنند، فهرست فعالیت‌ها را مشاهده کنند و خلاصه‌ای از فعالیت‌های هفته جاری را ببینند }.

این هدف به ایجنت کمک می‌کند محدوده پروژه را بهتر درک کند.

مخاطب و محیط استفاده را مشخص کنید

پیش از شروع توسعه، باید بدانیم چه کسانی از پروژه استفاده خواهند کرد و در چه شرایطی به آن دسترسی دارند.

برای مثال، ممکن است برنامه برای استفاده شخصی طراحی شود یا قرار باشد چند کاربر از طریق مرورگر به آن دسترسی داشته باشند. این تفاوت بر انتخاب فناوری، ساختار برنامه، نیاز به پایگاه داده و شیوه نگهداری اطلاعات تأثیر می‌گذارد.

از ایجنت برای بررسی اولیه کمک بگیرید

در این مرحله می‌توانیم از ایجنت بخواهیم ایده را تحلیل کند، پرسش‌های مهم را مشخص سازد و درباره امکان‌پذیری پروژه پیشنهاد بدهد.

نمونه درخواست:

«می‌خواهم ابزاری برای ثبت و پیگیری فعالیت‌های روزانه طراحی کنم. ایده را از نظر مسئله اصلی، کاربران هدف، قابلیت‌های ضروری، دشواری‌های احتمالی و راه‌های ساده‌سازی بررسی کن. اگر اطلاعات مهمی مشخص نیست، پیش از پیشنهاد راهکار از من سؤال بپرس. فعلاً کدی تولید نکن.»

این درخواست کمک می‌کند پیش از ورود به مرحله پیاده‌سازی، ابهام‌های اصلی مشخص شوند.

نکته: پیشنهادهای ایجنت درباره نیاز کاربران یا امکان‌پذیری پروژه باید در صورت لزوم با بررسی واقعی کاربران، منابع معتبر یا متخصصان تأیید شوند. ایجنت به‌تنهایی نمی‌تواند ثابت کند که یک ایده در دنیای واقعی نیازمند یا موفق خواهد بود.

تبدیل ایده به نیازمندی‌های مشخص

پس از مشخص‌شدن هدف، باید تعیین کنیم پروژه دقیقاً چه کارهایی باید انجام دهد. به این مشخصات، نیازمندی‌های پروژه گفته می‌شود.

نیازمندی‌ها مانند نقشه راهنمای ساخت پروژه عمل می‌کنند و به ما کمک می‌کنند مشخص کنیم چه چیزی باید ساخته شود و نتیجه نهایی چگونه ارزیابی خواهد شد.

نیازمندی‌های عملکردی

نیازمندی‌های عملکردی، قابلیت‌ها و کارهایی را مشخص می‌کنند که سیستم باید انجام دهد.

برای مثال، در برنامه ثبت فعالیت روزانه، نیازمندی‌های عملکردی می‌توانند شامل موارد زیر باشند:

- ثبت فعالیت جدید
- مشاهده فهرست فعالیت‌ها
- ویرایش اطلاعات ثبت‌شده
- حذف یک فعالیت
- نمایش خلاصه فعالیت‌های هفتگی

هر قابلیت باید تا حد امکان روشن باشد. برای نمونه، «ثبت فعالیت» می‌تواند شامل واردکردن نام فعالیت، تاریخ، مدت‌زمان و توضیحات باشد.

نیازمندی‌های غیرعملکردی

نیازمندی‌های غیرعملکردی به ویژگی‌های کیفی و شرایط عملکرد سیستم مربوط می‌شوند؛ مانند امنیت، سرعت، سهولت استفاده، قابلیت نگهداری و سازگاری با محیط اجرا.

برای مثال:

- برنامه باید در مرورگرهای موردنظر قابل استفاده باشد.
- پیام‌های خطا باید برای کاربر قابل‌فهم باشند.

- اطلاعات واردشده باید مطابق قواعد تعریف‌شده اعتبارسنجی شوند.

- ساختار کد باید به‌گونه‌ای باشد که توسعه قابلیت‌های بعدی امکان‌پذیر باشد.

این موارد بسته به نوع پروژه اهمیت متفاوتی دارند. برای یک نمونه آموزشی ساده، ممکن است برخی الزامات در حد پایه کافی باشند؛ اما برای سامانه‌ای که اطلاعات حساس کاربران را ذخیره می‌کند، امنیت و حریم خصوصی اهمیت بسیار بیشتری خواهند داشت.

تعیین محدوده پروژه

محدوده پروژه مشخص می‌کند چه قابلیت‌هایی در نسخه فعلی ساخته می‌شوند و چه قابلیت‌هایی به آینده موکول خواهند شد.

برای مثال، در نسخه اولیه برنامه ثبت فعالیت می‌توانیم ثبت، مشاهده و حذف فعالیت‌ها را پیاده‌سازی کنیم؛ اما قابلیت‌هایی مانند یادآوری هوشمند، اتصال به ساعت هوشمند و تحلیل پیشرفته را به مراحل بعد موکول کنیم.

محدوده مشخص مانع از آن می‌شود که پروژه در همان مراحل اولیه بیش‌ازحد بزرگ و پیچیده شود.

تعیین معیارهای پذیرش

معیار پذیرش، شرطی روشن است که نشان می‌دهد یک قابلیت تا چه اندازه درست پیاده‌سازی شده است.

برای نمونه، برای قابلیت ثبت فعالیت می‌توان معیارهای زیر را تعریف کرد:

- کاربر بتواند فعالیتی با اطلاعات معتبر ثبت کند.

- فعالیت جدید پس از ثبت در فهرست نمایش داده شود.

- ورود اطلاعات ضروری به‌صورت خالی پذیرفته نشود.

- پس از بارگذاری مجدد صفحه، اطلاعات مطابق طراحی برنامه حفظ شوند.

معیارهای پذیرش به ایجنت و کاربر کمک می‌کنند به‌جای قضاوت کلی درباره خوب‌بودن خروجی، آن را بر اساس شرایط مشخص بررسی کنند.

تهیه سند نیازمندی‌ها با کمک ایجنت

نمونه درخواست:

{ بر اساس هدف پروژه، سند نیازمندی‌های نسخه اولیه را تهیه کن. نیازمندی‌های عملکردی و غیرعملکردی را جداگانه بنویس، محدوده پروژه را مشخص کن و برای هر قابلیت معیار پذیرش تعریف کن. موارد نامشخص را به‌عنوان پرسش مطرح کن و بدون تأیید من فرضیات مهم را قطعی در نظر نگیر }.

خروجی این مرحله باید پیش از شروع توسعه بررسی و در صورت نیاز اصلاح شود.

طراحی ساختار و معماری پروژه

پس از مشخص‌شدن نیازمندی‌ها، باید تصمیم بگیریم پروژه چگونه سازمان‌دهی شود و اجزای آن چه ارتباطی با یکدیگر داشته باشند. این مرحله را طراحی ساختار و معماری پروژه می‌نامیم.

معماری پروژه به انتخاب‌ها و اصول کلی ساخت سیستم مربوط می‌شود؛ درحالی‌که ساختار پروژه نحوه سازمان‌دهی فایل‌ها، پوشه‌ها، ماژول‌ها و اجزای آن را مشخص می‌کند.

انتخاب فناوری مناسب

در پروژه‌های نرم‌افزاری باید فناوری‌هایی را انتخاب کنیم که با نیازمندی‌ها، سطح دانش، منابع و محدودیت‌های پروژه سازگار باشند.

برای مثال، در یک برنامه ساده ممکن است پایتون برای منطق اصلی برنامه کافی باشد. اگر برنامه به رابط وب نیاز داشته باشد، می‌توان فناوری‌های سمت کاربر و سمت سرور را نیز بررسی کرد.

ایجنت می‌تواند گزینه‌های مختلف را پیشنهاد دهد؛ اما انتخاب نهایی باید با توجه به پیچیدگی پروژه، امکانات محیط اجرا، هزینه نگهداری و دانش تیم انجام شود.

نمونه درخواست:

{ بر اساس نیازمندی‌های این پروژه، سه گزینه برای فناوری‌های مورد استفاده پیشنهاد بده. برای هر گزینه، مزایا، محدودیت‌ها، پیش‌نیازهای یادگیری و دشواری پیاده‌سازی را توضیح بده. در پایان، گزینه مناسب‌تری برای یک پروژه آموزشی کوچک پیشنهاد کن و دلیل انتخابت را بنویس }.

طراحی اجزای اصلی

پیش از نوشتن کد، بهتر است اجزای اصلی سیستم مشخص شوند.

برای نمونه، یک برنامه تحت وب ممکن است از اجزای زیر تشکیل شود:

- رابط کاربری: بخشی که کاربر با آن تعامل می‌کند.

- منطق برنامه: بخشی که قواعد و عملیات اصلی را اجرا می‌کند.

- پایگاه داده: بخشی که اطلاعات را ذخیره و بازیابی می‌کند.

- لایه ارتباطی:بخشی که در صورت نیاز، ارتباط میان رابط کاربری و منطق سرور را برقرار می‌کند.

این تقسیم‌بندی در همه پروژه‌ها یکسان نیست؛ یک برنامه ساده ممکن است به تمام این اجزا نیاز نداشته باشد.

طراحی ساختار فایل‌ها و پوشه‌ها

ساختار فایل‌ها باید به‌گونه‌ای باشد که پیدا کردن، اصلاح و توسعه بخش‌های مختلف آسان باشد.

نمونه درخواست:

{ بر اساس فناوری انتخاب‌شده، ساختار پیشنهادی پوشه‌ها و فایل‌های پروژه را طراحی کن. برای هر فایل توضیح بده چه مسئولیتی دارد و چرا به آن نیاز داریم. فعلاً فقط ساختار را پیشنهاد بده و هیچ فایلی ایجاد نکن }.

در این مرحله، بهتر است پیش از ایجاد تعداد زیادی فایل یا پوشه، طرح پیشنهادی را بررسی کنیم. ساختار بیش‌ازحد پیچیده برای پروژه کوچک می‌تواند نگهداری آن را دشوار کند.

مستندسازی تصمیم‌های معماری

تصمیم‌های مهم معماری بهتر است در یک سند ثبت شوند. این سند می‌تواند شامل فناوری‌های انتخاب‌شده، دلیل انتخاب آن‌ها، ساختار کلی پروژه و محدودیت‌های فنی باشد.

ثبت این اطلاعات کمک می‌کند در مراحل بعدی، ایجنت یا اعضای تیم تصمیم‌های قبلی را در نظر بگیرند و بدون دلیل ساختار پروژه را تغییر ندهند.

تقسیم پروژه به وظایف کوچک و قابل اجرا

پس از طراحی ساختار، باید پروژه را به وظایف کوچک‌تر تقسیم کنیم. هر وظیفه باید هدفی مشخص داشته باشد و بتوان نتیجه آن را جداگانه بررسی کرد.

تقسیم وظایف باعث می‌شود اجرای پروژه قابل‌کنترل‌تر شود و در صورت بروز مشکل، پیدا کردن مرحله‌ای که خطا در آن رخ داده آسان‌تر باشد.

چرا باید وظایف را کوچک کنیم؟

اگر از ایجنت بخواهیم تمام یک نرم‌افزار را با یک درخواست بزرگ بسازد، ممکن است بخش‌هایی را ناقص اجرا کند، جزئیات را نادیده بگیرد یا تغییراتی ایجاد کند که بررسی آن‌ها دشوار باشد.

در مقابل، تقسیم پروژه به مراحل کوچک به ما اجازه می‌دهد خروجی هر مرحله را ارزیابی کنیم و سپس به مرحله بعد برویم.

نمونه تقسیم یک پروژه

برای برنامه ثبت فعالیت روزانه، وظایف می‌توانند به شکل زیر تقسیم شوند:

۱. آماده‌سازی محیط و ساختار پروژه

۲. طراحی مدل اطلاعات فعالیت

۳. پیاده‌سازی قابلیت ثبت فعالیت

۴. پیاده‌سازی نمایش فهرست فعالیت‌ها

۵. اضافه‌کردن قابلیت ویرایش و حذف

۶. اعتبارسنجی ورودی‌ها

۷. نوشتن و اجرای آزمایش‌ها

۸. بررسی رابط کاربری و رفع مشکلات

۹. تهیه راهنمای نصب و اجرا

این ترتیب یک نمونه پیشنهادی است؛ در پروژه‌های واقعی، ترتیب وظایف به فناوری‌ها و وابستگی‌های میان اجزا بستگی دارد.

مشخص‌کردن ورودی و خروجی هر وظیفه

برای هر وظیفه باید مشخص کنیم ایجنت چه اطلاعاتی دریافت می‌کند و در پایان چه نتیجه‌ای باید ارائه دهد.

برای مثال:

- وظیفه: پیاده‌سازی قابلیت ثبت فعالیت

- ورودی: سند نیازمندی‌ها، ساختار پروژه و مدل اطلاعات

- خروجی: کد مربوط به ثبت فعالیت و آزمایش‌های مرتبط

- معیار پذیرش: ثبت اطلاعات معتبر و نمایش نتیجه، همراه با مدیریت ورودی‌های نامعتبر

این شیوه باعث می‌شود وظایف از حالت کلی و مبهم خارج شوند.

ثبت وضعیت وظایف

می‌توانیم وظایف را در یک فهرست ساده با وضعیت‌های «انجام‌نشده»، «در حال انجام»، «نیازمند بررسی» و «تکمیل‌شده» ثبت کنیم.

نکته مهم این است که پایان اجرای ایجنت الزاماً به معنای تکمیل موفق وظیفه نیست. یک وظیفه زمانی باید تکمیل‌شده محسوب شود که خروجی آن بررسی شده و معیارهای پذیرش را برآورده کرده باشد.

آماده‌سازی محیط و فایل‌های پروژه

پیش از شروع پیاده‌سازی، باید محیطی آماده کنیم که ایجنت بتواند در محدوده مشخص‌شده روی پروژه کار کند.

آماده‌سازی محیط توسعه

محیط توسعه شامل ابزارها و تنظیماتی است که برای ساخت، اجرا و آزمایش پروژه به آن‌ها نیاز داریم.

بسته به نوع پروژه، این موارد ممکن است شامل زبان برنامه‌نویسی، ویرایشگر کد، کتابخانه‌ها، پایگاه داده، ابزارهای آزمایش و سیستم کنترل نسخه باشند.

در این مرحله باید مطمئن شویم ابزارهای ضروری نصب شده‌اند و نسخه‌های موردنیاز با یکدیگر سازگار هستند.

ایجاد یا بررسی مخزن پروژه

در پروژه‌های نرم‌افزاری، استفاده از سیستم کنترل نسخه مانند Git به ثبت تغییرات و پیگیری سابقه پروژه کمک می‌کند.

اگر پروژه از قبل وجود دارد، بهتر است ابتدا وضعیت فایل‌ها و تغییرات فعلی بررسی شود. اگر پروژه جدید است، می‌توان ساختار اولیه و مخزن آن را آماده کرد.

پیش از آنکه ایجنت تغییرات گسترده‌ای ایجاد کند، باید بدانیم نسخه اولیه پروژه در چه وضعیتی قرار دارد و چگونه می‌توان تغییرات را بررسی یا در صورت لزوم بازگرداند.

آماده‌سازی فایل‌های راهنما

در پروژه‌های پیچیده‌تر، می‌توان فایل‌هایی برای توضیح هدف پروژه، ساختار پوشه‌ها، روش نصب، نحوه اجرای آزمایش‌ها و استانداردهای کدنویسی تهیه کرد.

این فایل‌ها به ایجنت کمک می‌کنند اطلاعات ضروری را در مراحل مختلف در دسترس داشته باشد. برای نمونه، بعضی Coding Agentها از فایل‌های دستورالعمل پروژه پشتیبانی می‌کنند؛ اما نام، قالب و نحوه خواندن این فایل‌ها به ابزار بستگی دارد.

تعیین محدوده دسترسی ایجنت

پیش از اجرا باید مشخص کنیم ایجنت به چه فایل‌ها و ابزارهایی دسترسی دارد و چه اقداماتی نیازمند تأیید هستند.

برای شروع، بهتر است دسترسی‌ها را به حد موردنیاز محدود کنیم. همچنین نباید بدون بررسی، اجازه اجرای عملیات مخرب، حذف فایل‌های مهم یا دسترسی به اطلاعات حساس را بدهیم.

نمونه درخواست:

{ ابتدا محیط و ساختار فعلی پروژه را بررسی کن. فایل‌های مهم، وابستگی‌ها، روش اجرای برنامه و وضعیت آزمایش‌ها را گزارش بده. فعلاً هیچ فایلی را تغییر نده و هیچ دستور تغییردهنده‌ای اجرا نکن }.

این درخواست کمک می‌کند پیش از شروع تغییرات، شناخت اولیه‌ای از وضعیت پروژه به دست آوریم.

توسعه پروژه با کمک ایجنت

پس از آماده‌سازی محیط و تأیید نیازمندی‌ها، می‌توانیم وارد مرحله توسعه شویم. در این مرحله، ایجنت بر اساس وظایف تعریف‌شده، به پیاده‌سازی قابلیت‌های پروژه کمک می‌کند.

شروع با یک قابلیت اصلی

بهتر است ابتدا یکی از قابلیت‌های ضروری و نسبتاً مستقل را پیاده‌سازی کنیم. این کار کمک می‌کند روش کار ایجنت، ساختار پروژه و نحوه ارزیابی خروجی را در مقیاس کوچک‌تر یاد بگیریم.

برای مثال، در برنامه ثبت فعالیت روزانه می‌توانیم ابتدا قابلیت ثبت و مشاهده فعالیت‌ها را بسازیم و سپس به سراغ قابلیت‌های دیگر برویم.

ارائه درخواست متناسب با وظیفه

در هر مرحله، درخواست باید به وظیفه همان مرحله محدود باشد و اطلاعات موردنیاز را در اختیار ایجنت قرار دهد.

نمونه درخواست:

{ اکنون قابلیت ثبت فعالیت را پیاده‌سازی کن. از ساختار و فناوری‌های تأییدشده پروژه استفاده کن. ابتدا فایل‌های مرتبط را بررسی کن، سپس تغییرات لازم را انجام بده. در پایان، فایل‌های تغییرکرده، نحوه عملکرد قابلیت و آزمایش‌های انجام‌شده را گزارش کن. سایر قابلیت‌ها را در این مرحله پیاده‌سازی نکن }.

این درخواست هم هدف را مشخص می‌کند و هم محدوده کار را محدود نگه می‌دارد.

بررسی تغییرات پس از هر مرحله

پس از اجرای وظیفه، باید بررسی کنیم چه فایل‌هایی ایجاد یا تغییر کرده‌اند، آیا ساختار پروژه حفظ شده است و آیا قابلیت جدید درست کار می‌کند.

اگر ابزار امکان نمایش تفاوت نسخه‌ها یا تغییرات فایل‌ها را دارد، می‌توانیم از آن برای بررسی دقیق‌تر استفاده کنیم.

بهتر است تغییرات مهم را پیش از رفتن به مرحله بعد بررسی کنیم؛ زیرا ادامه توسعه بر پایه کد نادرست می‌تواند اصلاح مشکلات را دشوارتر کند.

ادامه توسعه بر اساس نتایج

پس از تأیید یک قابلیت، می‌توانیم به سراغ وظیفه بعدی برویم. اگر مشکلی وجود داشته باشد، ابتدا همان مشکل را برطرف می‌کنیم.

این روش توسعه تدریجی به ما کمک می‌کند پروژه را در مراحل کوچک پیش ببریم و کنترل بیشتری بر کیفیت نهایی داشته باشیم.

نقش یادگیری کاربر در توسعه

اگر هدف، یادگیری برنامه‌نویسی نیز باشد، نباید فقط به تولید کد توسط ایجنت اکتفا کنیم. بهتر است از آن بخواهیم منطق راهکار، دلیل انتخاب روش‌ها و نحوه عملکرد بخش‌های مهم را توضیح دهد.

برای مثال:

{ کدی را که نوشته‌ای توضیح بده. مسئولیت هر تابع، ورودی‌ها و خروجی‌های آن و دلیل استفاده از این روش را مشخص کن. سپس سه سؤال آموزشی درباره کد مطرح کن تا مطمئن شوم منطق آن را فهمیده‌ام}.

به این ترتیب، ایجنت علاوه بر کمک به ساخت پروژه، می‌تواند در فرایند یادگیری نیز نقش داشته باشد.

آزمایش، اشکال‌زدایی و اصلاح کد

پس از پیاده‌سازی قابلیت‌ها، باید بررسی کنیم که برنامه مطابق نیازمندی‌ها کار می‌کند. این مرحله یکی از مهم‌ترین بخش‌های توسعه نرم‌افزار است؛ زیرا تولید کد به‌تنهایی تضمین نمی‌کند که برنامه صحیح باشد.

آزمایش عملکرد برنامه

آزمایش نرم‌افزار به ما کمک می‌کند رفتار برنامه را در شرایط مختلف بررسی کنیم.

برای نمونه، در برنامه ثبت فعالیت می‌توانیم این حالت‌ها را آزمایش کنیم:

- ثبت یک فعالیت با اطلاعات معتبر
- ثبت فعالیت بدون واردکردن اطلاعات ضروری
- مشاهده فهرست فعالیت‌ها
- ویرایش یک فعالیت موجود
- حذف یک فعالیت
- بررسی رفتار برنامه هنگام دریافت ورودی نامعتبر

آزمایش‌ها باید با نیازمندی‌ها و معیارهای پذیرش ارتباط داشته باشند.

استفاده از ایجنت برای شناسایی خطا

اگر برنامه خطایی داشته باشد، می‌توانیم پیام خطا، مراحل بازتولید مشکل و کدهای مرتبط را در اختیار ایجنت قرار دهیم.

نمونه درخواست:

{ هنگام ثبت فعالیت بدون واردکردن نام آن، برنامه با خطا مواجه می‌شود. علت را بررسی کن، توضیح بده مشکل در کدام بخش است و راهکار اصلاحی پیشنهاد بده. پس از اصلاح، حالت ورودی خالی و یک ورودی معتبر را آزمایش کن }.

ارائه اطلاعات دقیق درباره خطا به ایجنت کمک می‌کند مسئله را بهتر بررسی کند. اگر اطلاعات کافی وجود نداشته باشد، ممکن است ایجنت بر اساس حدس راهکاری ارائه دهد که مشکل اصلی را برطرف نکند

تفاوت اصلاح خطا با پنهان‌کردن آن

هدف از رفع خطا، برطرف‌کردن علت اصلی مشکل است، نه صرفاً حذف پیام خطا یا جلوگیری از نمایش آن.

برای مثال، اگر برنامه هنگام دریافت ورودی نامعتبر متوقف شود، راهکار مناسب ممکن است اعتبارسنجی ورودی و نمایش پیام مناسب باشد. نادیده‌گرفتن خطا بدون بررسی علت آن می‌تواند مشکلات دیگری ایجاد کند.

بنابراین، باید از ایجنت بخواهیم علت خطا را توضیح دهد و پس از اصلاح، عملکرد برنامه را دوباره آزمایش کند.

بررسی نتیجه آزمایش‌ها

اگر ایجنت اعلام کرد آزمایش‌ها موفق بوده‌اند، باید مشخص کنیم چه آزمایش‌هایی اجرا شده‌اند و آیا واقعاً نیازمندی‌های اصلی را پوشش می‌دهند.

مواردی که باید بررسی شوند عبات‌اند از:

- کدام آزمایش‌ها اجرا شدند؟

- کدام آزمایش‌ها موفق یا ناموفق بودند؟

- آیا بعضی آزمایش‌ها اصلاً اجرا نشدند؟

- آیا خطا یا هشدار مهمی باقی مانده است؟

- آیا آزمایش‌های انجام‌شده برای اطمینان از قابلیت موردنظر کافی هستند؟

موفقیت آزمایش‌های موجود به معنای نبود تمام خطاهای احتمالی نیست؛ ممکن است برخی شرایط هنوز بررسی نشده باشند.

اصلاح تدریجی و جلوگیری از ایجاد خطاهای جدید

پس از هر اصلاح، باید مطمئن شویم مشکل اصلی برطرف شده و قابلیت‌های قبلی همچنان درست کار می‌کنند. گاهی تغییر یک بخش باعث ایجاد مشکل در بخش دیگری می‌شود.

به همین دلیل، بهتر است پس از اصلاحات مهم، آزمایش‌های مرتبط دوباره اجرا شوند و تغییرات جدید بررسی شوند.

بررسی کیفیت و آماده‌سازی نسخه نهایی

پس از تکمیل قابلیت‌ها و انجام آزمایش‌ها، باید پروژه را برای استفاده یا ارائه آماده کنیم. این مرحله فراتر از اطمینان از اجرای برنامه است و شامل بررسی کیفیت، امنیت، مستندات و آمادگی پروژه برای تحویل می‌شود.

بررسی تطابق با نیازمندی‌ها

در این مرحله، پروژه را با نیازمندی‌ها و معیارهای پذیرش اولیه مقایسه می‌کنیم.

برای هر قابلیت مشخص می‌کنیم که آیا کامل شده، ناقص است یا هنوز نیاز به بررسی دارد. قابلیت‌های ضروری که برآورده نشده‌اند باید پیش از اعلام تکمیل پروژه تعیین تکلیف شوند.

بررسی کیفیت کد و ساختار پروژه

در پروژه‌های نرم‌افزاری، کد باید تا حد امکان خوانا، قابل‌فهم و قابل‌نگهداری باشد. همچنین بهتر است از پیچیدگی غیرضروری و تکرار بی‌دلیل جلوگیری شود.

می‌توانیم از ایجنت بخواهیم مشکلات احتمالی ساختار کد، نام‌گذاری، تکرار، وابستگی‌ها و مستندات را بررسی کند. بااین‌حال، پیشنهادهای آن باید با نیازهای واقعی پروژه سنجیده شوند؛ هر تغییر پیشنهادی الزاماً ضروری نیست.

بررسی امنیت و حریم خصوصی

اگر پروژه با اطلاعات کاربران یا داده‌های حساس سروکار دارد، باید دسترسی‌ها، ذخیره‌سازی اطلاعات، اعتبارسنجی ورودی‌ها و احتمال افشای داده‌ها بررسی شوند.

برای پروژه‌های سلامت، این موضوع اهمیت ویژه‌ای دارد؛ زیرا اطلاعات پزشکی و هویتی ممکن است حساس باشند. نباید اطلاعات واقعی و قابل‌شناسایی بیماران را بدون مجوز و بررسی الزامات امنیتی در اختیار ایجنت یا سرویس‌های خارجی قرار داد.

بررسی امنیت باید متناسب با خطرات پروژه انجام شود و برای سامانه‌های حساس، به ارزیابی تخصصی نیز نیاز است.

تهیه مستندات نهایی

مستندات به کاربر یا توسعه‌دهنده دیگری کمک می‌کنند پروژه را بشناسد، اجرا کند و در آینده توسعه دهد.

مستندات نهایی می‌توانند شامل موارد زیر باشند:

- معرفی و هدف پروژ
- فهرست قابلیت‌ها
- پیش‌نیازهای نصب
- روش راه‌اندازی و اجرا
- نحوه استفاده
- روش اجرای آزمایش‌ها
- محدودیت‌های شناخته‌شده
- مشکلات باقی‌مانده و پیشنهادهای توسعه آینده

برای پروژه‌های نرم‌افزاری، تهیه فایل راهنما مانند `README.md` می‌تواند نقطه شروع مناسبی باشد.

آماده‌سازی برای تحویل یا انتشار

پیش از تحویل پروژه باید فایل‌های غیرضروری، اطلاعات محرمانه، تنظیمات حساس و وابستگی‌های لازم بررسی شوند. همچنین باید مشخص باشد که پروژه در چه محیطی آزمایش شده و چه محدودیت‌هایی دارد.

اگر قرار است تغییرات در یک مخزن مشترک ثبت شوند، استفاده از شاخه جداگانه، ثبت تغییرات در Commitها و بررسی تغییرات پیش از ادغام می‌تواند به کنترل کیفیت کمک کند. مستندات رسمی GitHub نیز بر بررسی تغییرات و بازبینی آن‌ها پیش از ادغام در پروژه تأکید می‌کنند.

تعریف دقیق نسخه نهایی

پروژه زمانی آماده تحویل است که معیارهای تعیین‌شده را برآورده کند، آزمایش‌های ضروری انجام شده باشند و محدودیت‌های مهم آن مستند شده باشند.

نباید صرفاً به این دلیل که ایجنت اعلام کرده کار تمام شده است، پروژه را کامل تلقی کنیم. تصمیم نهایی باید بر اساس شواهد، آزمایش‌ها و نیازمندی‌های پروژه گرفته شود.

روش‌های افزایش سرعت توسعه با ایجنت

ایجنت‌ها می‌توانند در بسیاری از مراحل توسعه زمان انجام کار را کاهش دهند؛ اما استفاده نامناسب از آن‌ها ممکن است باعث افزایش خطا، تغییرات غیرضروری یا دوباره‌کاری شود. برای استفاده مؤثر، باید فرایند کار را به‌گونه‌ای طراحی کنیم که هم سرعت و هم قابلیت بررسی حفظ شوند.

آماده‌کردن Context پیش از شروع

وقتی هدف، ساختار پروژه، فناوری‌ها، محدودیت‌ها و وضعیت فعلی مشخص باشند، ایجنت برای فهمیدن شرایط به رفت‌وبرگشت کمتری نیاز خواهد داشت.

بهتر است اطلاعات مهم پروژه را در مستندات مناسب ثبت کنیم و هنگام شروع هر وظیفه، اطلاعات مرتبط را در اختیار ایجنت قرار دهیم.

استفاده از درخواست‌های دقیق و مرحله‌ای

درخواست‌های دقیق احتمال برداشت اشتباه را کاهش می‌دهند. همچنین تقسیم وظایف بزرگ به مراحل کوچک‌تر باعث می‌شود نتیجه هر مرحله قابل‌بررسی باشد.

برای افزایش سرعت، لازم نیست همه مراحل را به‌صورت دستی و طولانی توضیح دهیم؛ می‌توانیم از الگوهای درخواست استفاده کنیم و آن‌ها را متناسب با وظیفه تغییر دهیم.

استفاده مجدد از الگوها و مستندات

اگر در پروژه‌های مختلف وظایف مشابهی داریم، می‌توانیم الگوهای درخواست، چک‌لیست‌ها و دستورالعمل‌های تکرارشونده را نگهداری کنیم.

برای مثال، یک الگوی ثابت برای بررسی کد می‌تواند شامل بررسی خطاهای احتمالی، خوانایی، امنیت، آزمایش‌ها و گزارش تغییرات باشد. این روش از فراموش‌شدن مراحل مهم جلوگیری می‌کند.

خودکارسازی کارهای تکراری

بعضی وظایف، مانند اجرای آزمایش‌ها، بررسی قالب کد یا ساخت گزارش‌های مشخص، قابلیت خودکارسازی دارند. در صورت پشتیبانی ابزار و آماده‌بودن محیط اجرا، می‌توان این مراحل را به بخشی از فرایند توسعه تبدیل کرد

بااین‌حال، هر فرایند خودکاری باید متناسب با پروژه تنظیم و نتیجه آن بررسی شود. خودکارسازی یک کار نادرست، صرفاً باعث تکرار سریع‌تر همان خطا خواهد شد.

واگذاری هم‌زمان وظایف مستقل

اگر چند وظیفه وابستگی مستقیمی به یکدیگر نداشته باشند، ممکن است بتوان آن‌ها را به‌صورت هم‌زمان اجرا کرد؛ برای مثال، یک وظیفه برای بررسی مستندات و وظیفه دیگری برای بررسی ساختار فایل‌ها.

در پروژه‌های پیچیده‌تر، ابزارهایی که از عامل‌های فرعی یا اجرای چند وظیفه پشتیبانی می‌کنند، می‌توانند به تقسیم کار کمک کنند. بااین‌حال، اجرای هم‌زمان همیشه سریع‌تر نیست؛ زیرا وظایف وابسته، تعارض فایل‌ها و نیاز به هماهنگی می‌توانند هزینه اضافی ایجاد کنند.

انتخاب سطح مناسب نظارت

لازم نیست کاربر تمام جزئیات کم‌خطر را به‌صورت دستی انجام دهد؛ اما اقداماتی که ممکن است اطلاعات را حذف کنند، امنیت را تحت تأثیر قرار دهند یا تغییرات گسترده ایجاد کنند، باید با دقت بیشتری بررسی شوند.

برای مثال، می‌توان بررسی اولیه فایل‌ها را به ایجنت سپرد، اما پیش از حذف فایل‌های مهم یا ادغام تغییرات گسترده، نتیجه را بررسی کرد و تأیید لازم را داد.

جلوگیری از دوباره‌کاری

اگر ایجنت بدون اطلاع از تصمیم‌های قبلی پروژه کار کند، ممکن است راهکاری تکراری پیشنهاد دهد یا ساختارهای موجود را بی‌دلیل تغییر دهد.

ثبت تصمیم‌ها، وضعیت وظایف، تغییرات اخیر و مشکلات حل‌نشده به کاهش این مشکل کمک می‌کند. همچنین بهتر است پیش از واگذاری یک وظیفه جدید، مشخص کنیم چه کارهایی قبلاً انجام شده‌اند.

استفاده از کنترل نسخه

ابزارهایی مانند Git امکان ثبت تغییرات و بررسی سابقه پروژه را فراهم می‌کنند. ثبت تغییرات در مراحل کوچک و معنادار، مقایسه نسخه‌ها و امکان بازگشت به وضعیت قبلی را آسان‌تر می‌کند.

این روش به‌ویژه زمانی مفید است که ایجنت چندین فایل را تغییر می‌دهد یا قرار است راهکارهای مختلفی آزمایش شوند.

اندازه‌گیری سرعت در کنار کیفیت

هدف از استفاده از ایجنت فقط افزایش سرعت تولید خروجی نیست. اگر خروجی سریع تولید شود اما خطاهای زیادی داشته باشد یا زمان زیادی برای اصلاح آن صرف شود، ممکن است بهره‌وری واقعی افزایش پیدا نکند.

بنابراین، بهتر است علاوه بر زمان اجرا، کیفیت خروجی، تعداد خطاها، میزان دوباره‌کاری و زمان لازم برای بررسی را نیز در نظر بگیریم.

ساخت پروژه با کمک ایجنت، فرایندی مرحله‌ای است که از تعریف مسئله و تعیین هدف آغاز می‌شود و با طراحی، توسعه، آزمایش و آماده‌سازی نسخه نهایی ادامه پیدا می‌کند.

در این فرایند، ایجنت می‌تواند در تحلیل مسئله، تهیه نیازمندی‌ها، طراحی ساختار، نوشتن کد، رفع خطا و مستندسازی کمک کند. بااین‌حال، نتیجه مطلوب زمانی به دست می‌آید که وظایف روشن باشند، اطلاعات ضروری در اختیار ایجنت قرار گیرند و خروجی هر مرحله بررسی شود

اصول کلیدی این بخش عبارت‌اند از:

۱. پیش از ساخت، مسئله و هدف پروژه را روشن کنید.

۲. نیازمندی‌ها و معیارهای پذیرش را مشخص کنید.

۳. پیش از توسعه، ساختار و فناوری‌های مناسب را انتخاب کنید.

۴. پروژه را به وظایف کوچک و قابل‌بررسی تقسیم کنید.

۵. پیش از اعطای دسترسی، محیط و فایل‌های پروژه را بررسی کنید.

۶. پس از هر مرحله، تغییرات و نتایج را ارزیابی کنید.

۷. کد را آزمایش کنید و علت خطاها را بررسی کنید.

۸. امنیت، مستندات و محدودیت‌های پروژه را پیش از تحویل در نظر بگیرید.

۹. از ایجنت برای افزایش بهره‌وری استفاده کنید، اما مسئولیت ارزیابی و تصمیم‌گیری نهایی را حفظ کنید.

اصل اساسی: ایجنت می‌تواند بخش بزرگی از کار اجرایی را انجام دهد، اما موفقیت پروژه به تعریف درست مسئله، تصمیم‌گیری آگاهانه، بررسی مستمر و مسئولیت‌پذیری انسان وابسته است.

فصل7: مدیریت پروژه با کمک ایجنت

برنامه‌ریزی و تقسیم پروژه به وظایف کوچک

استفاده از ایجنت به‌تنهایی تضمین نمی‌کند که یک پروژه به‌درستی پیش برود. برای رسیدن به نتیجه مطلوب، باید پروژه را به مراحل مشخص تقسیم کرد، ترتیب انجام کارها را تعیین کرد و وظایف هر مرحله را به‌روشنی توضیح داد. این کار به ایجنت کمک می‌کند تا به‌جای انجام تغییرات پراکنده، بر اساس یک مسیر مشخص فعالیت کند.

برای برنامه‌ریزی پروژه، ابتدا باید هدف نهایی را تعیین کنیم و سپس مشخص کنیم که برای رسیدن به آن، چه کارهایی باید انجام شوند. برای مثال، اگر هدف ساخت یک اپلیکیشن سلامت باشد، می‌توان مراحل پروژه را به این صورت در نظر گرفت:

۱. مشخص‌کردن مسئله و نیاز کاربران

۲. تعیین امکانات اصلی اپلیکیشن

۳. طراحی ساختار و انتخاب فناوری‌های مناسب

۴. پیاده‌سازی امکانات به‌صورت مرحله‌ای

۵. آزمایش عملکرد و برطرف‌کردن خطاها

۶. بررسی نهایی و آماده‌سازی نسخه قابل استفاده

پس از مشخص‌شدن مراحل، هر مرحل باید به وظایف کوچک‌تر و قابل بررسی تقسیم شود. برای مثال، به‌جای درخواست «یک اپلیکیشن سلامت کامل بساز»، می‌توان ابتدا از ایجنت خواست ساختار پروژه را طراحی کند، سپس یک قابلیت مشخص را پیاده‌سازی کند و بعد از آزمایش آن، به سراغ قابلیت بعدی برود.

نکته مهم: بهتر است پیش از شروع تغییرات گسترده، از ایجنت بخواهیم برنامه پیشنهادی خود را ارائه دهد. پس از بررسی و تأیید برنامه، اجرای وظایف را آغاز کنیم. این روش احتمال تغییرات غیرضروری و انحراف از هدف اصلی را کاهش می‌دهد.

نمونه درخواست به ایجنت:

{ هدف من ساخت یک اپلیکیشن ساده برای ثبت عادت‌های سلامت کاربران است. پروژه را به مراحل کوچک و قابل اجرا تقسیم کن. برای هر مرحله، وظایف، پیش‌نیازها و معیارهای تکمیل را مشخص کن. فعلاً فقط برنامه را ارائه بده و هیچ فایلی را تغییر نده }.

مدیریت Context، فایل‌ها و تغییرات پروژه

ایجنت برای انجام درست وظایف به اطلاعات کافی درباره پروژه نیاز دارد. این اطلاعات می‌تواند شامل هدف پروژه، ساختار فایل‌ها، تصمیم‌های قبلی، فناوری‌های مورد استفاده، مشکلات شناخته‌شده و کارهایی باشد که تاکنون انجام شده‌اند. مجموعه اطلاعات مرتبطی که ایجنت برای انجام وظیفه از آن استفاده می‌کند، Context نام دارد.

اگر اطلاعات مهم در اختیار ایجنت قرار نگیرد، ممکن است تصمیم‌های قبلی را نادیده بگیرد، راه‌حل ناسازگاری پیشنهاد دهد یا بخشی از کد را به شکلی تغییر دهد که با سایر قسمت‌های پروژه هماهنگ نباشد.

برای مدیریت بهتر Context، لازم است اطلاعات مهم پروژه به شکلی منظم ثبت شوند. برای مثال، می‌توان یک فایل README برای معرفی پروژه و نحوه اجرای آن، یک فایل مستندات برای ثبت تصمیم‌های مهم و یک فهرست وظایف برای پیگیری مراحل انجام کار داشت. البته ساختار مناسب به اندازه و نوع پروژه بستگی دارد و برای پروژه‌های کوچک، مستندات ساده نیز کافی است.

مدیریت فایل‌ها و تغییرات نیز اهمیت زیادی دارد. پیش از اعمال تغییرات مهم، بهتر است وضعیت فعلی پروژه بررسی و در صورت نیاز نسخه پشتیبان تهیه شود. ابزارهایی مانند Git امکان ثبت تغییرات و بازگشت به نسخه‌های قبلی را فراهم می‌کنند. همچنین، پیش از پذیرفتن تغییرات ایجنت باید مشخص شود کدام فایل‌ها تغییر کرده‌اند و آیا این تغییرات با هدف پروژه سازگار هستند یا خیر.

نمونه درخواست به ایجنت:

{ ابتدا ساختار پروژه و فایل‌های مرتبط را بررسی کن. هدف پروژه، فناوری‌های استفاده‌شده، تصمیم‌های مهم و وضعیت فعلی آن را خلاصه کن. سپس مشخص کن برای انجام وظیفه بعدی چه اطلاعاتی در اختیار داری و چه اطلاعاتی هنوز لازم است. تا پیش از تأیید من، هیچ فایلی را تغییر نده }.

مدیریت پیشرفت پروژه و کنترل عملکرد ایجنت

در طول اجرای پروژه، صرفاً دانستن اینکه ایجنت مشغول انجام کار است کافی نیست؛ باید بتوانیم تشخیص دهیم چه مقدار از کار انجام شده، چه مشکلاتی وجود دارد و آیا نتیجه به هدف موردنظر نزدیک شده است یا خیر.

برای این کار، بهتر است وظایف پروژه را با وضعیت مشخص ثبت کنیم؛ برای مثال:

انجام‌نشده: وظیفه هنوز شروع نشده است.

در حال انجام: ایجنت در حال کار روی وظیفه است.

نیازمند بررسی: ایجنت کار را انجام داده، اما نتیجه باید بررسی شود.

تکمیل‌شده: نتیجه بررسی شده و معیارهای تعیین‌شده را برآورده می‌کند.

متوقف‌شده: ادامه کار به اطلاعات، تصمیم یا رفع یک مشکل نیاز دارد.

نکته مهم این است که گزارش ایجنت درباره تکمیل یک وظیفه، به‌تنهایی به معنای موفقیت آن نیست. برای مثال، ممکن است ایجنت اعلام کند که یک قابلیت برنامه‌نویسی را پیاده‌سازی کرده است، اما آن قابلیت در عمل درست کار نکند. بنابراین، هر وظیفه باید معیار مشخصی برای تکمیل داشته باشد و نتیجه آن بر اساس شواهد قابل بررسی ارزیابی شود.

همچنین، بهتر است پس از پایان هر مرحله، تغییرات انجام‌شده، فایل‌های تحت تأثیر، آزمایش‌های اجراشده و مشکلات باقی‌مانده بررسی شوند. اگر نتیجه با انتظار ما مطابقت نداشت، باید پیش از شروع مرحله بعدی، مشکل را مشخص و اصلاح کنیم.

نمونه درخواست به ایجنت:

{ گزارشی از وضعیت فعلی پروژه ارائه بده. وظایف انجام‌شده، وظایف باقی‌مانده، تغییرات اعمال‌شده، آزمایش‌های اجراشده و مشکلات حل‌نشده را جداگانه مشخص کن. برای هر مورد، فقط زمانی ادعای تکمیل یا موفقیت کن که شواهد کافی در اختیار داشته باشی }.

همکاری چند ایجنت و نقش انسان در مدیریت پروژه

در پروژه‌های پیچیده، گاهی می‌توان از چند ایجنت برای انجام بخش‌های مختلف کار استفاده کرد. برای مثال، یک ایجنت می‌تواند اطلاعات موردنیاز را پژوهش کند، ایجنت دیگری ساختار پروژه را طراحی کند و ایجنتی دیگر در پیاده‌سازی یا بررسی کد کمک کند. در بعضی ابزارها، یک ایجنت اصلی وظایف را میان ایجنت‌های فرعی تقسیم می‌کند و نتایج آن‌ها را جمع‌بندی می‌کند.

بااین‌حال، استفاده از چند ایجنت همیشه بهتر یا سریع‌تر نیست. اگر وظایف به‌خوبی تقسیم نشده باشند، ممکن است ایجنت‌ها کارهای تکراری انجام دهند، پیشنهادهای متناقض ارائه کنند یا تغییرات ناسازگار در فایل‌های مشترک ایجاد کنند. بنابراین، باید وظایف هر ایجنت، محدوده دسترسی و نحوه تحویل نتیجه به‌روشنی مشخص شود. همچنین، نتایج باید پیش از ادغام در پروژه اصلی بررسی شوند.

مهم‌تر از همه، مسئولیت نهایی تصمیم‌گیری و تأیید نتیجه بر عهده انسان است. ایجنت می‌تواند برنامه پیشنهاد دهد، اطلاعات تحلیل کند، کد بنویسد و خطاها را شناسایی کند؛ اما انسان باید تشخیص دهد که آیا راه‌حل انتخاب‌شده با هدف پروژه، نیاز کاربران، محدودیت‌ها و ملاحظات امنیتی سازگار است یا خیر.

در عمل، بهتر است کارهایی که نتیجه آن‌ها قابل بررسی و برگشت‌پذیر است، با استقلال بیشتری به ایجنت سپرده شوند؛ اما اقداماتی مانند حذف اطلاعات مهم، انتشار نسخه نهایی، تغییر تنظیمات حساس یا دسترسی به داده‌های محرمانه باید با محدودیت و نظارت متناسب انجام شوند.

نمونه درخواست به ایجنت:

{ وظایف این مرحله را به بخش‌های مستقل تقسیم کن و مشخص کن کدام بخش‌ها را می‌توان به ایجنت‌های جداگانه سپرد. وابستگی میان وظایف و خطرهای احتمالی تداخل را توضیح بده. پیش از هر تغییر مهم در فایل‌های مشترک یا اقدام برگشت‌ناپذیر، از من تأیید بگیر }.

مدیریت پروژه با کمک ایجنت بر چهار اصل اساسی استوار است: برنامه‌ریزی و تقسیم وظایف، حفظ اطلاعات و مدیریت تغییرات، بررسی مستمر پیشرفت و حفظ نظارت انسانی. ایجنت زمانی بیشترین فایده را دارد که بداند چه کاری باید انجام دهد، به اطلاعات موردنیاز دسترسی داشته باشد و نتیجه فعالیتش بر اساس معیارهای مشخص ارزیابی شود.

در نهایت، هدف از مدیریت پروژه با ایجنت این نیست که تمام تصمیم‌ها را به آن واگذار کنیم؛ بلکه باید از توانایی آن برای افزایش سرعت و کاهش کارهای تکراری استفاده کنیم، درحالی‌که کنترل مسیر پروژه و تأیید نتایج را حفظ می‌کنیم.

فصل8: محدودیت‌ها، خطرات و اشتباهات رایج در استفاده از ایجنت‌ها

محدودیت‌های ایجنت‌ها

ایجنت‌های هوش مصنوعی می‌توانند در انجام وظایف مختلف، از تحلیل اطلاعات و برنامه‌نویسی تا مدیریت پروژه و خودکارسازی کارها، کمک کنند؛ اما توانایی آن‌ها نامحدود نیست. شناخت این محدودیت‌ها باعث می‌شود از ایجنت‌ها واقع‌بینانه‌تر استفاده کنیم و تصمیم‌های مهم را صرفاً بر اساس خروجی آن‌ها نگیریم.

یکی از مهم‌ترین محدودیت‌ها احتمال تولید اطلاعات نادرست یا پاسخ‌های غیرقابل اعتماد است. ایجنت ممکن است اطلاعاتی را با لحنی مطمئن ارائه دهد، درحالی‌که آن اطلاعات اشتباه، ناقص یا قدیمی باشند. در برنامه‌نویسی نیز ممکن است کدی تولید کند که از نظر ظاهری درست به نظر برسد، اما هنگام اجرا با خطا مواجه شود یا نتیجه مورد انتظار را ایجاد نکند.

محدودیت دیگر، برداشت اشتباه از درخواست کاربر است. اگر هدف، محدودیت‌ها یا خروجی مورد انتظار به‌روشنی مشخص نشده باشند، ایجنت ممکن است وظیفه را به شکلی متفاوت از خواسته ما انجام دهد. برای مثال، اگر از آن بخواهیم کد یک پروژه را بهبود دهد، ممکن است علاوه بر اصلاح موردنظر، بخش‌های دیگری را نیز تغییر دهد.

همچنین، ایجنت به اطلاعات و Context در دسترس خود وابسته است. اگر اطلاعات مهم پروژه در اختیارش نباشد یا گفت‌وگوی طولانی باعث شود جزئیات قبلی به‌خوبی در دسترس نباشند، ممکن است تصمیم‌های ناسازگار بگیرد یا بخشی از کارهای قبلی را تکرار کند.

از طرف دیگر، عملکرد ایجنت به مدل هوش مصنوعی، ابزارهای در دسترس، سطح دسترسی، منابع محاسباتی و کیفیت اطلاعات وابسته است. بنابراین، نمی‌توان انتظار داشت هر ایجنتی تمام وظایف را با سرعت و دقت یکسان انجام دهد.

راهکار: درخواست‌ها را دقیق بنویسید، اطلاعات ضروری را در اختیار ایجنت قرار دهید و نتایج مهم را با آزمایش، مستندات معتبر یا منابع مستقل بررسی کنید.

خطرات امنیتی و مشکلات اجرایی

برخی ایجنت‌ها فقط پاسخ تولید نمی‌کنند؛ بلکه می‌توانند فایل‌ها را بخوانند و تغییر دهند، کد اجرا کنند، به ابزارهای خارجی متصل شوند یا اقداماتی را در محیط کاربر انجام دهند. همین قابلیت‌ها در کنار مزایای خود، خطراتی نیز ایجاد می‌کنند.

مهم‌ترین خطرات عبارت‌اند از:

- دسترسی بیش‌ازحد: اگر ایجنت به فایل‌ها، حساب‌ها یا ابزارهایی دسترسی داشته باشد که برای وظیفه‌اش ضروری نیستند، احتمال آسیب ناشی از خطا یا سوءاستفاده افزایش می‌یابد.

- تغییر یا حذف ناخواسته اطلاعات: اجرای یک دستور اشتباه یا برداشت نادرست از درخواست ممکن است باعث تغییر فایل‌های مهم، حذف اطلاعات یا مختل‌شدن عملکرد پروژه شود.

- افشای اطلاعات محرمانه: واردکردن رمزهای عبور، کلیدهای API، اطلاعات شخصی یا داده‌های حساس در ابزارهای نامناسب می‌تواند امنیت این اطلاعات را به خطر بیندازد.

- اجرای کد یا دستور ناامن: کد تولیدشده توسط ایجنت لزوماً امن نیست. اجرای آن بدون بررسی، به‌ویژه در محیط اصلی یا روی فایل‌های مهم، ممکن است پیامدهای جدی داشته باشد.

- اعتماد به محتوای نامطمئن: فایل‌ها، صفحات وب یا داده‌های دریافتی ممکن است شامل دستورهای مخربی باشند که تلاش می‌کنند ایجنت را به انجام کاری خارج از هدف اصلی هدایت کنند. این مسئله در سیستم‌های متصل به ابزارها اهمیت بیشتری دارد.

برای کاهش این خطرات، باید دسترسی ایجنت را به حداقل موردنیاز محدود کرد، پیش از تغییرات مهم نسخه پشتیبان تهیه کرد و اقدامات حساس را با نظارت مناسب انجام داد. همچنین، اطلاعات محرمانه نباید بدون بررسی سیاست‌های امنیتی و حریم خصوصی در اختیار ابزار قرار گیرند.

نکته: تأییدکردن یک اقدام توسط کاربر یا اجرای آن در محیط آزمایشی، به‌تنهایی تضمین‌کننده امنیت نیست. دسترسی‌ها، محتوای ورودی و پیامدهای احتمالی اقدام نیز باید بررسی شوند.

اشتباهات رایج در استفاده از ایجنت‌ها

بسیاری از مشکلات هنگام کار با ایجنت‌ها نه‌تنها به محدودیت فناوری، بلکه به شیوه استفاده از آن نیز مربوط می‌شوند. برخی از اشتباهات رایج عبارت‌اند از:

ارائه درخواست‌های مبهم

درخواست‌هایی مانند «برنامه را بهتر کن» یا «یک پروژه کامل بساز» محدوده و معیار موفقیت مشخصی ندارند. در نتیجه، ایجنت ممکن است نتیجه‌ای تولید کند که با انتظار کاربر متفاوت باشد.

سپردن وظایف بزرگ در یک مرحله

درخواست انجام یک پروژه پیچیده به‌صورت یکجا، بررسی نتیجه و شناسایی خطاها را دشوار می‌کند. تقسیم پروژه به مراحل کوچک‌تر، امکان کنترل و اصلاح را افزایش می‌دهد.

اعتماد کامل به خروجی ایجنت

حتی اگر پاسخ یا کد تولیدشده حرفه‌ای به نظر برسد، ممکن است دارای خطا باشد. پذیرش خروجی بدون بررسی می‌تواند باعث انتشار اطلاعات نادرست یا ورود اشکال به پروژه شود.

نادیده‌گرفتن Context و تصمیم‌های قبلی

اگر اطلاعات مهم پروژه ثبت نشوند یا هنگام ادامه کار در اختیار ایجنت قرار نگیرند، احتمال تکرار کارها و ایجاد تغییرات ناسازگار افزایش می‌یابد.

دادن دسترسی‌های غیرضروری

در اختیار قراردادن دسترسی گسترده به فایل‌ها، حساب‌ها یا محیط اجرا، بدون توجه به نیاز واقعی وظیفه، خطرات امنیتی و اجرایی را افزایش می‌دهد.

بررسی‌نکردن تغییرات قبل از ادامه کار

اگر تغییرات هر مرحله بررسی نشوند، ممکن است خطاهای کوچک به مراحل بعدی منتقل شوند و اصلاح آن‌ها در ادامه دشوارتر شود.

برای جلوگیری از این اشتباهات، بهتر است هر وظیفه با هدف روشن آغاز شود، نتیجه آن به‌صورت مستقل بررسی شود و پیش از ادامه پروژه، از درست‌بودن تغییرات مهم اطمینان حاصل کنیم.

روش‌های کاهش خطا و استفاده ایمن از ایجنت

اگرچه نمی‌توان تمام خطاهای ایجنت‌ها را از بین برد، می‌توان با رعایت چند اصل، احتمال بروز مشکلات و میزان آسیب احتمالی را کاهش داد.

۱. تعیین دقیق هدف و محدوده کار: مشخص کنید ایجنت چه کاری باید انجام دهد، چه کارهایی خارج از محدوده وظیفه هستند و چه نتیجه‌ای قابل قبول است.

۲. محدودکردن دسترسی‌ها: فقط مجوزها و ابزارهایی را در اختیار ایجنت قرار دهید که برای انجام وظیفه ضروری هستند. در صورت امکان، کار را ابتدا در یک محیط آزمایشی انجام دهید.

۳. بررسی خروجی و آزمایش نتایج: پاسخ‌های اطلاعاتی را با منابع معتبر و کدها را با اجرای آزمایش‌های مناسب بررسی کنید. موفقیت یک آزمایش نیز لزوماً به معنای نبود تمام خطاها نیست.

۴. تهیه نسخه پشتیبان و ثبت تغییرات: پیش از تغییرات مهم، نسخه‌ای قابل بازگشت از پروژه داشته باشید. ابزارهایی مانند Git به ثبت تغییرات و مقایسه نسخه‌ها کمک می‌کنند.

۵. محافظت از اطلاعات حساس: رمزهای عبور، کلیدهای دسترسی و اطلاعات محرمانه را در اختیار ایجنت قرار ندهید، مگر آنکه روش استفاده از آن‌ها ضروری، مجاز و از نظر امنیتی بررسی‌شده باشد.

۶. درخواست تأیید برای اقدامات حساس: پیش از حذف اطلاعات، انتشار محتوا، تغییر تنظیمات مهم یا اجرای دستورهای پرخطر، سازوکار تأیید و نظارت مناسبی در نظر بگیرید.

۷. حفظ نظارت انسانی: تصمیم‌های مهم باید با توجه به هدف پروژه، شرایط واقعی و پیامدهای احتمالی بررسی شوند. ایجنت می‌تواند در تحلیل و پیشنهاد راه‌حل کمک کند، اما خروجی آن نباید بدون ارزیابی پذیرفته شود.

در مجموع، استفاده ایمن از ایجنت به معنای جلوگیری از هرگونه خطا نیست؛ بلکه به معنای طراحی فرایندی است که در آن خطاها زودتر شناسایی شوند و پیامدهای آن‌ها تا حد امکان محدود بمانند.

ایجنت‌ها ابزارهای توانمندی هستند، اما ممکن است اطلاعات نادرست تولید کنند، درخواست را اشتباه بفهمند، تغییرات ناخواسته ایجاد کنند یا در معرض خطرات امنیتی قرار گیرند. بخش مهمی از این خطرات با درخواست‌های دقیق، مدیریت مناسب Context، محدودکردن دسترسی‌ها، آزمایش نتایج و ثبت تغییرات قابل کنترل است.

اصل اساسی این فصل آن است که توانایی انجام یک کار با توانایی انجام درست، امن و قابل اعتماد آن یکسان نیست. بنابراین، استفاده مؤثر از ایجنت‌ها مستلزم شناخت محدودیت‌ها و حفظ نظارت انسانی، به‌ویژه در وظایف حساس و اقدامات مهم، است.

فصل9: اصول حرفه ای کار با ایجنت ها

نوشتن درخواست‌های دقیق و هدفمند

یکی از مهم‌ترین مهارت‌ها در کار با ایجنت‌های هوش مصنوعی، توانایی تعریف درست وظایف است. هرچقدر درخواست شفاف‌تر باشد، احتمال اینکه ایجنت نتیجه‌ای متناسب با نیاز ما تولید کند بیشتر می‌شود. درخواست خوب فقط مشخص نمی‌کند چه کاری باید انجام شود؛ بلکه اطلاعات لازم، محدوده وظیفه و نتیجه مورد انتظار را نیز توضیح می‌دهد.

برای نوشتن یک درخواست مناسب، بهتر است به این موارد توجه کنیم:

- هدف: مشخص کنیم دقیقاً به چه نتیجه‌ای نیاز داریم.

Context: اطلاعات و پیش‌زمینه لازم را در اختیار ایجنت قرار دهیم.

- محدوده وظیفه: توضیح دهیم ایجنت چه کارهایی را انجام دهد و چه کارهایی را انجام ندهد.

- قالب خروجی: مشخص کنیم نتیجه به چه شکلی ارائه شود؛ برای مثال، جدول، گزارش، فهرست یا کد.

- معیار موفقیت: توضیح دهیم چه شرایطی باید برقرار باشد تا بتوانیم نتیجه را قابل قبول بدانیم.

برای مثال، درخواست «کد را اصلاح کن» مشخص نمی‌کند مشکل چیست، کدام بخش باید تغییر کند و چگونه باید صحت اصلاح را بررسی کرد. در مقابل، درخواست دقیق‌تر می‌تواند چنین باشد:

{ تابع محاسبه میانگین در فایل `statistics.py` هنگام دریافت فهرست خالی خطا می‌دهد. ابتدا علت خطا را بررسی کن، سپس کمترین تغییر لازم را پیشنهاد بده. رفتار تابع برای ورودی‌های معتبر را تغییر نده و پس از اصلاح، آزمایش‌هایی برای ورودی خالی و ورودی معتبر اجرا کن. در پایان، تغییرات و نتایج آزمایش‌ها را گزارش بده }.

در این درخواست، مشکل، محل بررسی، محدودیت تغییرات و روش ارزیابی مشخص شده‌اند.

اصل مهم: درخواست‌های بزرگ یا مبهم را به وظایف کوچک‌تر تقسیم کنید. همچنین، اگر ایجنت اطلاعات کافی ندارد، از آن بخواهید پیش از شروع اجرا، سؤال‌های ضروری را مطرح کند.

مدیریت صحیح وظایف و سطح دسترسی

برای استفاده حرفه‌ای از ایجنت، باید وظایف را متناسب با توانایی ابزار و میزان حساسیت کار تعریف کنیم. همه کارها به یک اندازه پیچیده یا پرخطر نیستند؛ بنابراین، سطح استقلال و دسترسی ایجنت نیز باید متناسب با هر وظیفه تعیین شود.

برای مثال، می‌توان از ایجنت خواست ساختار یک پروژه را بررسی کند، بدون اینکه اجازه تغییر فایل‌ها را داشته باشد. پس از بررسی نتیجه، می‌توان اجازه داد تغییرات پیشنهادی را در فایل‌های مشخص اعمال کند. در مراحل بعدی، پس از اطمینان از عملکرد مناسب، ممکن است اجرای برخی آزمایش‌های کم‌خطر را نیز به آن سپرد.

به‌طور کلی، می‌توان سه شیوه متداول برای مدیریت وظایف در نظر گرفت:

۱. بررسی و پیشنهاد: ایجنت اطلاعات را تحلیل می‌کند و راه‌حل پیشنهاد می‌دهد، اما تغییری اعمال نمی‌کند.

۲. اجرا با تأیید انسان: ایجنت وظیفه را آماده می‌کند، اما پیش از اقدامات مهم یا حساس، تأیید کاربر را دریافت می‌کند.

۳. اجرای خودکار در محدوده مشخص: ایجنت وظایف تعریف‌شده را بدون تأیید جداگانه برای هر مرحله اجرا می‌کند؛ البته دسترسی‌ها، محدودیت‌ها و شرایط توقف باید از قبل مشخص شده باشند.

این دسته‌بندی یک چارچوب کاربردی برای مدیریت کار است و لزوماً به معنای وجود همین سه حالت در تمام ابزارها نیست. امکانات واقعی به ایجنت و تنظیمات آن بستگی دارند.

در انتخاب سطح استقلال، باید پیچیدگی وظیفه، احتمال آسیب، امکان بازگشت از تغییرات و اهمیت اطلاعات را در نظر گرفت. برای کارهای ساده و قابل بازگشت می‌توان استقلال بیشتری در نظر گرفت؛ اما برای اقدامات حساس، بهتر است نظارت و تأیید انسانی افزایش یابد.

نمونه درخواست:

{ فایل‌های مرتبط با این قابلیت را بررسی کن و تغییرات پیشنهادی را توضیح بده. فعلاً هیچ فایلی را تغییر نده. پس از تأیید من، فقط فایل‌های مشخص‌شده را اصلاح کن و از تغییر بخش‌های نامرتبط خودداری کن}.

بررسی و ارزیابی خروجی ایجنت

تولید خروجی پایان کار نیست؛ نتیجه باید بررسی شود تا مشخص شود آیا واقعاً هدف موردنظر را برآورده می‌کند یا خیر. این موضوع به‌ویژه در برنامه‌نویسی، پژوهش، تحلیل داده و کار با اطلاعات حساس اهمیت دارد.

برای ارزیابی خروجی، ابتدا باید معیارهای موفقیت را مشخص کنیم. این معیارها بسته به نوع وظیفه متفاوت‌اند. برای مثال، در یک پروژه برنامه‌نویسی، معیارها می‌توانند شامل اجرای صحیح برنامه، عبور از آزمایش‌ها، رعایت نیازمندی‌ها و نداشتن تغییرات غیرضروری باشند. در یک گزارش پژوهشی نیز می‌توان صحت اطلاعات، اعتبار منابع، پوشش موضوع و رعایت قالب موردنظر را بررسی کرد.

برای ارزیابی دقیق‌تر، می‌توان از روش‌های زیر استفاده کرد:

- بررسی مستقیم: خروجی را با درخواست اولیه و نیازهای پروژه مقایسه کنید.

- آزمایش: در صورت امکان، عملکرد نتیجه را با ورودی‌های مناسب بررسی کنید.

- مقایسه با منابع معتبر: ادعاهای مهم و اطلاعات تخصصی را با منابع قابل اعتماد تطبیق دهید.

- بررسی تغییرات: مشخص کنید چه فایل‌ها، بخش‌ها یا اطلاعاتی تغییر کرده‌اند.

- بررسی موارد استثنا: حالت‌هایی را آزمایش کنید که ممکن است باعث خطا شوند؛ مانند ورودی خالی، داده ناقص یا شرایط غیرمعمول.

برای مثال، اگر ایجنت اعلام کند که یک قابلیت برنامه‌نویسی را با موفقیت پیاده‌سازی کرده است، نباید فقط به گزارش آن اکتفا کنیم. باید کد را بررسی کنیم، آزمایش‌های مناسب را اجرا کنیم و مطمئن شویم که قابلیت جدید با سایر بخش‌های پروژه سازگار است.

همچنین، اگر نتیجه ناقص یا اشتباه باشد، بهتر است مشکل را دقیق توضیح دهیم و از ایجنت بخواهیم آن را اصلاح کند. گفتن «درست نیست» معمولاً به‌اندازه توضیح خطای مشاهده‌شده و نتیجه مورد انتظار مفید نیست.

نمونه درخواست :

«خروجی تولیدشده را با نیازمندی‌های اولیه مقایسه کن. مواردی را که برآورده شده‌اند، موارد ناقص و مشکلات احتمالی را جداگانه گزارش بده. اگر آزمایشی اجرا نکرد.

حفظ کنترل و بهبود فرایند کار

کار با ایجنت یک فرایند تکرارشونده است: درخواست تعریف می‌شود، ایجنت وظیفه را انجام می‌دهد، نتیجه بررسی می‌شود و در صورت نیاز، درخواست یا راه‌حل اصلاح می‌شود. استفاده حرفه‌ای از این چرخه به ما کمک می‌کند از تکرار خطاها جلوگیری کنیم و کیفیت نتایج را به‌تدریج افزایش دهیم.

برای بهبود این فرایند، بهتراست تصمیم‌های مهم، تغییرات اصلی و مشکلات حل‌نشده ثبت شوند. در پروژه‌های برنامه‌نویسی، ابزارهایی مانند Git امکان ثبت تاریخچه تغییرات، مقایسه نسخه‌ها و بازگشت به وضعیت قبلی را فراهم می‌کنند. در پروژه‌های دیگر نیز می‌توان از مستندات یا فهرست وظایف برای ثبت پیشرفت استفاده کرد.

همچنین، لازم است پس از هر مرحله درباره ادامه کار تصمیم بگیریم. اگر نتیجه مناسب است، می‌توان به مرحله بعد رفت. اگر نتیجه ناقص است، باید مشکل را مشخص کرد و اصلاح خواست. اگر ایجنت از هدف اصلی فاصله گرفته است، باید درخواست و محدوده وظیفه را دوباره روشن کنیم.

چند اصل مهم برای حفظ کنترل عبارت‌اند از:

تغییرات مهم را پیش از پذیرفتن بررسی کنید.

اطلاعات و تصمیم‌های کلیدی پروژه را ثبت کنید.

در صورت امکان، از نسخه‌های قابل بازگشت استفاده کنید.

خطاها و بازخوردهای مهم را در مراحل بعدی در نظر بگیرید.

اجازه ندهید ایجنت بدون محدودیت، وظایف جدیدی خارج از هدف پروژه آغاز کند.

در اقدامات حساس، امکان توقف و تأیید انسانی را حفظ کنید.

در نهایت، هدف از بهبود فرایند کار این نیست که تمام مراحل را پیچیده و زمان‌بر کنیم؛ بلکه باید کنترل‌های متناسب با اهمیت هر وظیفه را به کار ببریم. برای یک کار ساده ممکن است بررسی کوتاه کافی باشد، اما برای پروژه‌ای مهم یا حساس، آزمایش، مستندسازی و کنترل دقیق‌تری لازم است.

کار حرفه‌ای با ایجنت‌ها بر چهار اصل استوار است: تعریف دقیق درخواست، مدیریت وظایف و دسترسی‌ها، ارزیابی خروجی و حفظ کنترل بر فرایند کار.

ایجنت می‌تواند بسیاری از فعالیت‌ها را سریع‌تر انجام دهد، اما کیفیت نتیجه تا حد زیادی به نحوه تعریف وظیفه و بررسی خروجی بستگی دارد. هرچه هدف روشن‌تر، محدودیت‌ها مشخص‌تر و معیارهای ارزیابی دقیق‌تر باشند، استفاده از ایجنت مؤثرتر خواهد بود.

در یک فرایند حرفه‌ای، ایجنت مسئول کمک به انجام کار است و انسان مسئول تعیین هدف، نظارت بر مسیر و تصمیم‌گیری درباره پذیرفتن نتیجه نهایی است.

فصل10: جمع بندی و مرجع کاربردی

نقشه راه شروع کار با ایجنت‌ها

برای شروع کار با ایجنت‌های هوش مصنوعی، لازم نیست از همان ابتدا تمام قابلیت‌ها و تنظیمات آن‌ها را یاد بگیریم. بهتر است با یک هدف مشخص و پروژه‌ای کوچک شروع کنیم، مراحل کار را به‌تدریج پیش ببریم و پس از آشنایی با ابزار، وظایف پیچیده‌تر را به آن بسپاریم.

مسیر پیشنهادی برای شروع کار با ایجنت‌ها شامل مراحل زیر است:

مرحله اول: تعیین هدف

مشخص کنید می‌خواهید چه مسئله‌ای را حل کنید یا چه کاری را انجام دهید. هدف باید روشن و قابل ارزیابی باشد؛ برای مثال، ساخت یک برنامه ساده، تحلیل یک مجموعه داده یا خودکارسازی یک کار تکراری.

مرحله دوم: انتخاب ایجنت مناسب

با توجه به نوع پروژه، ابزار موردنیاز، هزینه، سطح دسترسی و امکانات ایجنت، ابزار مناسب را انتخاب کنید. برای مثال، یک ایجنت برنامه‌نویسی برای کار روی کد و یک ایجنت پژوهشی برای جمع‌آوری و تحلیل اطلاعات می‌تواند انتخاب مناسب‌تری باشد.

مرحله سوم: آماده‌سازی محیط و اطلاعات

اطلاعات و فایل‌های موردنیاز را آماده کنید، محیط کار را بررسی کنید و مطمئن شوید ایجنت به منابع ضروری دسترسی دارد. در عین حال، از دادن دسترسی‌های غیرضروری خودداری کنید.

مرحله چهارم: تعریف وظیفه و برنامه‌ریزی

هدف، محدوده کار، محدودیت‌ها و نتیجه مورد انتظار را مشخص کنید. برای پروژه‌های بزرگ، ابتدا از ایجنت بخواهید برنامه‌ای مرحله‌ای ارائه دهد و سپس پس از بررسی برنامه، اجرای آن را آغاز کنید.

مرحله پنجم: اجرا و بررسی مرحله‌ای

اجازه دهید ایجنت وظایف مشخص‌شده را انجام دهد. پس از هر مرحله، نتیجه را بررسی کنید و در صورت نیاز، درخواست اصلاح بدهید. از سپردن تمام مراحل یک پروژه پیچیده به ایجنت بدون بررسی نتایج خودداری کنید.

مرحله ششم: ارزیابی و تکمیل پروژه

در پایان، نتیجه را با نیازهای اولیه مقایسه کنید، آزمایش‌های لازم را انجام دهید و تغییرات اعمال‌شده را بررسی کنید. تنها زمانی پروژه را تکمیل‌شده بدانید که معیارهای موردنظر برآورده شده باشند.

بهترین روش یادگیری، شروع با پروژه‌های کوچک و واقعی است. برای مثال، می‌توانید ابتدا از ایجنت بخواهید یک پوشه را تحلیل کند، سپس یک فایل ساده بسازد و در مراحل بعدی، اصلاح کد و اجرای آزمایش‌ها را تمرین کنید.

نمونه یک فرایند کاری از ابتدا تا انتها

برای درک بهتر نحوه استفاده از ایجنت، فرض کنیم می‌خواهیم یک برنامه ساده برای ثبت عادت‌های روزانه سلامت طراحی کنیم. کاربران باید بتوانند عادت‌هایی مانند نوشیدن آب یا پیاده‌روی را ثبت کنند و وضعیت انجام آن‌ها را ببینند.

در این مثال، مراحل کار را به‌صورت زیر پیش می‌بریم.

مرحله اول: تعریف مسئله و نیازمندی‌ها

ابتدا مشخص می‌کنیم برنامه چه امکاناتی باید داشته باشد. در نسخه اولیه، ثبت عادت، علامت‌گذاری انجام آن و نمایش فهرست عادت‌ها کافی است.

درخواست نمونه:

{ می‌خواهم یک برنامه ساده برای ثبت عادت‌های روزانه سلامت بسازم. کاربران باید بتوانند عادت جدید اضافه کنند، عادت‌های خود را ببینند و انجام هر عادت را ثبت کنند. برای نسخه اولیه، امکانات ضروری را مشخص کن و از پیشنهاد قابلیت‌های غیرضروری خودداری کن }.

مرحله دوم: طراحی برنامه

از ایجنت می‌خواهیم ساختار پیشنهادی پروژه، فناوری‌های مناسب و مراحل پیاده‌سازی را مشخص کند. در این مرحله بهتر است هنوز تغییرات گسترده‌ای در فایل‌ها اعمال نشوند.

درخواست نمونه:

{ بر اساس نیازمندی‌های مشخص‌شده، ساختار ساده‌ای برای پروژه پیشنهاد بده. وظایف را به مراحل کوچک تقسیم کن و برای هر مرحله، معیار تکمیل را مشخص کن. فعلاً فقط برنامه را ارائه بده }.

مرحله سوم: پیاده‌سازی مرحله‌ای

پس از بررسی برنامه، از ایجنت می‌خواهیم هر قابلیت را جداگانه پیاده‌سازی کند. بعد از هر مرحله، کد و تغییرات بررسی می‌شوند و سپس به مرحله بعد می‌رویم.

درخواست نمونه:

{ قابلیت افزودن عادت جدید را بر اساس ساختار تأییدشده پیاده‌سازی کن. فایل‌های نامرتبط را تغییر نده و در پایان، فایل‌های تغییرکرده و نحوه آزمایش این قابلیت را توضیح بده }.

مرحله چهارم: آزمایش و رفع خطا

پس از پیاده‌سازی، بررسی می‌کنیم که برنامه در شرایط مختلف درست کار می‌کند. برای مثال، آیا افزودن عادت جدید موفق است؟ آیا ورودی خالی به‌درستی مدیریت می‌شود؟ آیا ثبت انجام عادت نتیجه صحیحی دارد؟

درخواست نمونه:

{ قابلیت‌های پیاده‌سازی‌شده را بررسی کن و آزمایش‌های مناسبی برای آن‌ها پیشنهاد بده. در صورت امکان، آزمایش‌ها را اجرا کن و نتایج واقعی را گزارش بده. خطاها را از مشکلات احتمالی جدا کن

مرحله پنجم: بررسی نهایی

در پایان، بررسی می‌کنیم که امکانات برنامه با نیازمندی‌های اولیه مطابقت داشته باشند، خطاهای مهم برطرف شده باشند و هیچ تغییر غیرضروری باقی نمانده باشد.

درخواست نمونه:

{ وضعیت نهایی پروژه را با نیازمندی‌های اولیه مقایسه کن. امکانات تکمیل‌شده، موارد ناقص، آزمایش‌های اجراشده و مشکلات باقی‌مانده را گزارش بده. اگر بخشی آزمایش نشده است، آن را به‌عنوان تأییدشده معرفی نکن }.

این مثال نشان می‌دهد که کار با ایجنت فقط به نوشتن یک درخواست محدود نمی‌شود؛ بلکه شامل تعریف هدف، برنامه‌ریزی، اجرا، بررسی و اصلاح مداوم است. همچنین، برای پروژه‌های واقعی ممکن است مراحل بیشتری مانند طراحی رابط کاربری، مدیریت پایگاه داده، بررسی امنیت و انتشار برنامه لازم باشد.

چک‌لیست انتخاب ایجنت و مدیریت پروژه

چک‌لیست زیر به شما کمک می‌کند پیش از شروع پروژه و در طول اجرای آن، نکات ضروری را بررسی کنید.

پیش از شروع پروژه

- هدف و مسئله اصلی پروژه مشخص شده است.
- نوع ایجنت متناسب با نیاز پروژه انتخاب شده است.
- هزینه‌ها و محدودیت‌های ابزار بررسی شده‌اند.
- فایل‌ها و اطلاعات موردنیاز آماده‌اند.
- سطح دسترسی ایجنت متناسب با وظیفه تعیین شده است.
- معیارهای موفقیت پروژه مشخص شده‌اند.

هنگام اجرای پروژه

- وظایف بزرگ به مراحل کوچک‌تر تقسیم شده‌اند.
- درخواست‌ها و محدودیت‌ها به‌روشنی تعریف شده‌اند.
- اطلاعات و تصمیم‌های مهم پروژه حفظ می‌شوند.
- تغییرات ایجنت به‌صورت مرحله‌ای بررسی می‌شوند.
- آزمایش‌ها و نتایج واقعی ثبت می‌شوند.
- پیش از اقدامات حساس، تأیید لازم دریافت می‌شود.

پیش از نهایی‌کردن پروژه

- نتیجه با نیازمندی‌های اولیه مطابقت دارد.
- آزمایش‌های ضروری انجام شده‌اند.
- تغییرات و فایل‌های نهایی بررسی شده‌اند.
- مشکلات حل‌نشده مشخص شده‌اند.
- اطلاعات محرمانه و دسترسی‌ها بررسی شده‌اند.
- نسخه پشتیبان یا امکان بازگشت به نسخه قبلی وجود دارد.

bitwiftit

این چک‌لیست یک راهنمای عمومی است و لازم نیست تمام موارد آن برای هر وظیفه به یک اندازه اجرا شوند. میزان بررسی باید متناسب با پیچیدگی، اهمیت و خطرات پروژه باشد.

واژه‌نامه اصطلاحات و مرجع دستورها

در این بخش، اصطلاحات پرکاربردی را که هنگام کار با ایجنت‌ها با آن‌ها مواجه می‌شویم، به‌صورت خلاصه مرور می‌کنیم.

۱. AI (Artificial Intelligence) — هوش مصنوعی: فناوری ساخت سیستم‌هایی که وظایفی مانند تحلیل اطلاعات، پیش‌بینی و تولید محتوا را انجام می‌دهند.

۲. AI Model — مدل هوش مصنوعی: سامانه‌ای آموزش‌دیده که بر اساس ورودی، خروجی تولید می‌کند.

۳. Agent — ایجنت: سامانه‌ای که با استفاده از مدل هوش مصنوعی و در صورت نیاز ابزارها، برای انجام وظایف مشخص اقدام می‌کند.

۴. Prompt — درخواست یا دستور: متنی که برای هدایت مدل یا ایجنت و مشخص‌کردن وظیفه موردنظر استفاده می‌شود.

۵. Context — زمینه یا اطلاعات زمینه‌ای: اطلاعات و پیش‌زمینه مرتبطی که برای انجام درست وظیفه در اختیار مدل یا ایجنت قرار می‌گیرد.

۶. Tool — ابزار: قابلیت یا برنامه‌ای که ایجنت برای انجام کارهایی مانند خواندن فایل، جست‌وجو یا اجرای کد به کار می‌گیرد.

۷. Workflow — گردش کار: مجموعه‌ای از مراحل و فعالیت‌های مرتبط که برای رسیدن به یک نتیجه انجام می‌شوند.

۸. Reasoning — استدلال: فرایندی که مدل برای تحلیل مسئله، بررسی اطلاعات و رسیدن به پاسخ یا راه‌حل به کار می‌گیرد.

۹. Subagent — ایجنت فرعی: ایجنتی که در برخی سیستم‌ها برای انجام بخشی از وظایف یا بررسی یک موضوع مشخص به کار گرفته می‌شود.

۱۰. Multi-Agent System — سیستم چندایجنتی : سیستمی که در آن چند ایجنت با نقش‌های مختلف برای رسیدن به یک هدف همکاری می‌کنند.

۱۱. Debugging — اشکال‌زدایی: فرایند شناسایی، تحلیل و برطرف‌کردن خطاهای برنامه.

۱۲. Code Review — بازبینی کد: بررسی کد برای شناسایی خطاها، مشکلات امنیتی و موارد قابل بهبود.

۱۳. Repository — مخزن پروژه: محلی برای نگهداری فایل‌های پروژه و در بسیاری از موارد، تاریخچه تغییرات آن‌ها.

۱۴. Git — ابزار مدیریت نسخه: ابزاری برای ثبت، مقایسه و پیگیری تغییرات فایل‌ها، به‌ویژه در پروژه‌های برنامه‌نویسی.

۱۵. API (Application Programming Interface) — رابط برنامه‌نویسی کاربردی: روشی تعریف‌شده برای ارتباط میان نرم‌افزارها یا سرویس‌ها.

۱۶. Terminal — ترمینال: محیط متنی برای واردکردن و اجرای دستورها در سیستم‌عامل.

۱۷. Test — آزمایش: روشی برای بررسی اینکه یک برنامه یا قابلیت مطابق انتظار عمل می‌کند یا خیر.

مرجع کوتاه درخواست‌های عمومی

درخواست‌های عمومی عبارت‌هایی هستند که می‌توان در بسیاری از ابزارهای هوش مصنوعی و ایجنت‌ها، متناسب با قابلیت آن‌ها، استفاده کرد. این عبارت‌ها الزاماً دستورهای اختصاصی یا فرمان‌های قابل اجرا نیستند؛ بلکه روش‌هایی برای بیان وظیفه به شمار می‌روند.

۱. Explore the project — بررسی پروژه: بررسی ساختار پوشه‌ها، فایل‌ها و بخش‌های مختلف پروژه برای آشنایی با وضعیت فعلی آن.

نمونه:

`Explore the project structure and explain the purpose of the main files. Do not modify any files.`

۲. Explain this code — توضیح کد: توضیح عملکرد یک قطعه کد یا فایل و تشریح بخش‌های مهم آن.

نمونه:

`.Explain how this function works and describe its inputs and outputs`

۳. Analyze this error — تحلیل خطا: بررسی پیام خطا و شناسایی علت‌های احتمالی آن.

نمونه:

`.Analyze this error, identify its likely cause, and suggest a solution`

۴. Create a plan — تهیه برنامه: مشخص‌کردن مراحل موردنیاز برای انجام یک وظیفه یا پروژه.

نمونه:

`Create a step-by-step plan for implementing this feature. Do not make changes yet.`

۵. Break down the task — تقسیم وظیفه: تبدیل یک وظیفه بزرگ به چند کار کوچک‌تر و قابل بررسی.

نمونه:

`.Break this project into small tasks and identify their dependencies`

۶. Search the project— جست‌وجو در پروژه: پیدا کردن فایل‌ها، توابع یا بخش‌هایی که با موضوع موردنظر مرتبط هستند.

نمونه:

`Search the project for all references to this function and summarize the results.`

۷. Review the code — بازبینی کد: بررسی کد از نظر خطاهای احتمالی، خوانایی و مشکلات امنیتی.

نمونه:

`Review this code for potential bugs and security issues. Do not modify any files.`

۸. Fix this bug — رفع خطا: تلاش برای شناسایی و برطرف‌کردن یک خطای مشخص در برنامه.

نمونه:

`Investigate this bug, make the smallest necessary change, and explain how to test the fix.`

۹. Run the tests — اجرای آزمایش‌ها: اجرای آزمایش‌های موجود برای بررسی عملکرد برنامه، در صورتی که ایجنت به محیط اجرای مناسب دسترسی داشته باشد.

نمونه:

`Run the relevant tests and report which tests passed or failed.`

۱۰. Summarize the changes — خلاصه تغییرات: ارائه گزارشی از فایل‌ها و بخش‌هایی که تغییر کرده‌اند.

نمونه:

`Summarize all changes made, explain why they were needed, and identify any remaining issues.`

۱۱. Check the requirements — بررسی نیازمندی‌ها: مقایسه نتیجه با خواسته‌ها و معیارهای اولیه پروژه.

نمونه:

`Compare the current implementation with the requirements and list any missing features.`

۱۲. Ask before making changes — درخواست تأیید پیش از تغییر: مشخص‌کردن اینکه ایجنت باید قبل از اعمال تغییرات مهم، از کاربر اجازه بگیرد.

نمونه:

`Analyze the relevant files and propose changes. Ask for my approval before modifying any files.`

این درخواست‌ها قابل تغییر و ترکیب هستند. بهتر است هنگام استفاده از آن‌ها، نام فایل، هدف، محدودیت‌ها و نتیجه مورد انتظار را نیز مشخص کنید.

مرجع کوتاه دستورهای اختصاصی ابزارها

برخی ابزارها علاوه بر درخواست‌های معمول، دستورهای اختصاصی دارند که با نماد یا قالب مشخصی وارد می‌شوند. برای مثال، در Claude Code برخی از دستورهای اسلش عبارت‌اند از:

۱. `/help` — نمایش راهنما: برای مشاهده راهنمای ابزار و اطلاعات مربوط به دستورهای قابل استفاده.

۲. `/clear` — پاک‌کردن زمینه گفت‌وگوی فعلی: برای شروع یک گفت‌وگوی تازه در محیط مربوط. این دستور لزوماً فایل‌های پروژه را حذف نمی‌کند.

۳. `/compact` — فشرده‌سازی Context: برای خلاصه‌کردن زمینه گفت‌وگو و کمک به ادامه کار با اطلاعات فشرده‌تر.

۴. `/init` — آماده‌سازی راهنمای پروژه: برای ایجاد فایل راهنمای اولیه پروژه در محیط پشتیبانی‌شده.

این دستورها نمونه‌هایی از قابلیت‌های اختصاصی یک ابزار هستند و ممکن است کاربرد یا رفتار آن‌ها با توجه به نسخه و تنظیمات تغییر کند. بنابراین، پیش از استفاده باید مستندات رسمی همان ابزار را بررسی کرد.

همچنین، دستورهای Git مانند موارد زیر متعلق به ابزار Git هستند، نه دستورهای اختصاصی ایجنت؛ هرچند ایجنت برنامه‌نویسی ممکن است بتواند از آن‌ها استفاده کند:

- `git status` — نمایش وضعیت فعلی فایل‌ها و تغییرات ثبت‌نشده.

- `git diff` — نمایش تفاوت میان تغییرات فایل‌ها و نسخه مبنا.

- `git log` — نمایش تاریخچه ثبت تغییرات.

- `git diff --staged` — نمایش تغییراتی که برای ثبت در نسخه بعدی آماده شده‌اند.

جمع‌بندی نهایی کتابچه

ایجنت‌های هوش مصنوعی می‌توانند فرایند انجام بسیاری از وظایف را سریع‌تر و منظم‌تر کنند؛ اما استفاده مؤثر از آن‌ها به شناخت نحوه عملکرد، انتخاب ابزار مناسب، تعریف دقیق وظایف و ارزیابی خروجی وابسته است.

برای شروع، نیازی نیست تمام ابزارها یا اصطلاحات تخصصی را به‌صورت کامل بشناسید. بهتر است با یک پروژه کوچک آغاز کنید، مراحل آن را مشخص کنید، از ایجنت برای انجام وظایف محدود کمک بگیرید و نتیجه هر مرحله را بررسی کنید. با افزایش تجربه، می‌توانید به سراغ پروژه‌های پیچیده‌تر، ابزارهای پیشرفته‌تر و سیستم‌های چندایجنتی بروید.

مهم‌ترین نکته این است که ایجنت را جایگزین کامل دانش، قضاوت و مسئولیت انسانی ندانیم. ایجنت ابزاری برای کمک به حل مسئله است؛ ابزاری که زمانی بیشترین ارزش را ایجاد می‌کند که در کنار هدف‌گذاری روشن، بررسی انتقادی و نظارت آگاهانه انسان به کار گرفته شود.

bitwitit